

Change Data Capture Developer Guide

Version 66.0, Spring '26



Summer '26

CONTENTS

Change Data Capture	1
Keep Your External Data Current with Change Data Capture	2
When Do You Use Change Data Capture?	2
Change Event Object Support	3
Select Objects for Change Notifications in the User Interface	4
Select Objects for Change Notifications with Metadata API and Tooling API	4
Change Event Message Structure	5
ChangeEventHeader Fields	8
Change Event Body Fields	10
Merged Change Events	12
Other Types of Change Events: Gap and Overflow Events	13
Gap Events	14
Overflow Events	15
Subscribe to Change Events	17
Change Event Storage and Delivery	18
Subscription Channels	19
Compose Streams of Change Data Capture Notifications with Custom Channels	20
Example Diagrams for Channels and Channel Members	21
Transaction-Based Replication Steps	24
Subscribe with Pub/Sub API	27
Get Compound Fields in Change Events	29
Enrich Change Events with Extra Fields	32
Filter Your Event Streams	41
Subscribe with Apex Triggers	56
Monitor Change Event Publishing and Delivery Usage	66
Security Considerations	68
Required Permissions for Change Event Subscribers	69
Field-Level Security	70
Change Events for Encrypted Salesforce Data	70
Change Event Considerations	72
General Considerations	72
Change Data Capture Allocations	73
Standard Object Notes	80
Change Events for Tasks and Events	81
Change Events for Person Accounts	88
Change Events for Users	92
Change Events for Lead Conversion	94
Change Events for PricebookEntry	100
Change Events for Fields	100

Contents

Sending Data Differences for Fields of Updated Records **101**
Change Events for Custom Field Type Conversions **106**

CHANGE DATA CAPTURE

Receive near-real-time changes of Salesforce records, and synchronize corresponding records in an external data store. Change Data Capture publishes change events, which represent changes to Salesforce records. Changes include creation of a new record, updates to an existing record, deletion of a record, and undeletion of a record.

IN THIS SECTION:

[Keep Your External Data Current with Change Data Capture](#)

Use Change Data Capture to update data in an external system instead of doing periodic exports and imports of data or repeated API calls. Capturing changes with Change Data Capture event notifications ensures that your external data can be updated in real time and stays fresh.

[Change Event Message Structure](#)

A change event message contains header fields and record fields.

[Merged Change Events](#)

For efficiency, sometimes change events for one transaction are merged into one event if the same change occurred in multiple records of the same object type during one second.

[Other Types of Change Events: Gap and Overflow Events](#)

Other types of change events are provided to handle special situations, such as capturing changes not caught in the Salesforce application servers, or handling high loads of changes.

[Subscribe to Change Events](#)

Learn about subscribing to change events including subscription channels, subscription methods, and required permissions.

[Monitor Change Event Publishing and Delivery Usage](#)

To get usage data for event publishing and delivery to CometD and Pub/Sub API clients, `empApi` Lightning components, and event relays, query the PlatformEventUsageMetric object. In API 58.0 and later, enable and use Enhanced Usage Metrics to get granular usage data for various time segments. If Enhanced Usage Metrics isn't enabled, usage data is available for the last 24 hours, ending at the last hour, and for historical daily usage. PlatformEventUsageMetric is available in API version 50.0 and later.

[Security Considerations](#)

Learn about the user permissions required for subscription, field-level security, and Shield Platform Encryption.

[Change Event Considerations](#)

Keep in mind change event considerations and allocations when subscribing to change events.

[Standard Object Notes](#)

Learn about the characteristics of change events for some standard objects and the fields included in the event messages.

[Change Events for Fields](#)

Learn about the change event characteristics for fields.

EDITIONS

Available in: both Salesforce Classic and Lightning Experience

Available in: **Enterprise**, **Performance**, **Unlimited**, and **Developer** editions

SEE ALSO:

[Trailhead: Change Data Capture Basics](#)

Keep Your External Data Current with Change Data Capture

Use Change Data Capture to update data in an external system instead of doing periodic exports and imports of data or repeated API calls. Capturing changes with Change Data Capture event notifications ensures that your external data can be updated in real time and stays fresh.

IN THIS SECTION:

[When Do You Use Change Data Capture?](#)

You can think of Change Data Capture as part of the real-time data replication process for the cloud.

[Change Event Object Support](#)

Change events are available for all custom objects defined in your Salesforce org and a subset of standard objects.

[Select Objects for Change Notifications in the User Interface](#)

To receive notifications on the default standard channel for record changes, select the custom objects and supported standard objects that you're interested in on the Change Data Capture page.

[Select Objects for Change Notifications with Metadata API and Tooling API](#)

Use PlatformEventChannelMember in Metadata API or Tooling API to create or retrieve object event selections for the default standard channel or a custom channel. The default standard channel, ChangeEvents, corresponds to the selections that you configure in Setup in the Change Data Capture page. For a custom channel, the selections are set when you create the channel member. The SelectedEntity field in PlatformEventChannelMember represents a selected event.

When Do You Use Change Data Capture?

You can think of Change Data Capture as part of the real-time data replication process for the cloud.

Data replication includes these stages.

1. Initial (day 0) copy of the entire dataset to the external system.
2. Continuous synchronization of new and updated data to the external system.
3. Reconciliation of duplicate data between the two systems.

Change Data Capture is the continuous synchronization part of replication (step 2). It publishes the deltas of Salesforce data for new and changed records. Change Data Capture requires an integration app for receiving events and performing updates in the external system.

For example, you have a human resource (HR) system with copies of employee custom object records from Salesforce. You can synchronize the employee records in the HR system by receiving change events. You can then process the corresponding insert, update, delete, or undelete operations in the HR system. Because the changes are received in near real time, the data in your HR system stays up to date.

Change Data Capture enables secure and scalable event streaming to downstream systems. An integration app can receive millions of events per day and synchronize data with another system. The event retention of three days enables a CometD or Pub/Sub API subscriber to get past event messages. Encryption and field-level security enable secure event storage and communication.

Use Change Data Capture to:

- Keep external systems in sync with Salesforce data.
- Receive notifications of Salesforce record changes, including create, update, delete, and undelete operations.
- Subscribe using CometD, Pub/Sub API, or Apex triggers.
- Capture field changes for all records.
- Get broad access to all data regardless of sharing rules.
- Deliver only the fields a user has access to based on field-level security.

- Encrypt change event fields at rest.
- Get information about the change in the event header, such as the origin of the change, which allows ignoring changes that your client generates.
- Perform data updates using transaction boundaries.
- Use a versioned event schema.
- Subscribe to mass changes in a scalable way.
- Get access to retained events for up to three days.

We don't recommend using Change Data Capture to:

- Perform audit trails based on record and field changes.
- Update the UI for many users in apps subscribed with CometD or Pub/Sub API. Change Data Capture is intended to keep downstream systems in sync but not individual users. If many users are subscribed with CometD or Pub/Sub API clients, the concurrent client limit can be hit. For more information, see [Change Data Capture Allocations](#).

Change Data Capture Reliability

The temporary storage of change events in the event bus enhances the reliability of event delivery. CometD and Pub/Sub API subscribers can catch up on events that were missed due to an offline subscriber or a connection error. For more information about how to replay events using Pub/Sub API, see [Subscribe RPC Method](#) in the *Pub/Sub API Developer Guide*.

Change events are temporarily persisted to and served from an industry-standard distributed system. A distributed system doesn't have the same semantics or guarantees as a transactional database. Change events are queued and buffered, and Salesforce attempts to publish the events asynchronously. In rare cases, some event messages aren't persisted in the distributed system during the initial or subsequent attempts. In those cases, the events aren't delivered to subscribers and aren't recoverable.

Change Event Object Support

Change events are available for all custom objects defined in your Salesforce org and a subset of standard objects.

For a list of objects that support change events, see [StandardObjectNameChangeEvent](#) in the *Object Reference for Salesforce and Lightning Platform*.



Note: Not all objects may be available in your org. Some objects require specific feature settings and permissions to be enabled.

SEE ALSO:

[Object Reference for Salesforce and Lightning Platform: Standard Objects](#)

[Salesforce Help: Create Partner Users](#)

[Loyalty Management Developer Guide: Standard Objects](#)

Select Objects for Change Notifications in the User Interface

To receive notifications on the default standard channel for record changes, select the custom objects and supported standard objects that you're interested in on the Change Data Capture page.

From Setup, in the Quick Find box, enter *Change Data Capture*, and click **Change Data Capture**. The Available Entities list shows the objects available in your Salesforce org for Change Data Capture. You can select up to five entities, including standard and custom objects. To enable more entities, contact your Salesforce Account Representative to purchase an add-on license. The add-on license removes the limit on the number of entities you can select. Also, it increases the event delivery allocation for CometD and Pub/Sub API clients. With the add-on license, you can select up to 10 entities at a time in the Available Entities list. After selecting the first 10 entities, you can add more.

USER PERMISSIONS

To view the Change Data Capture page:

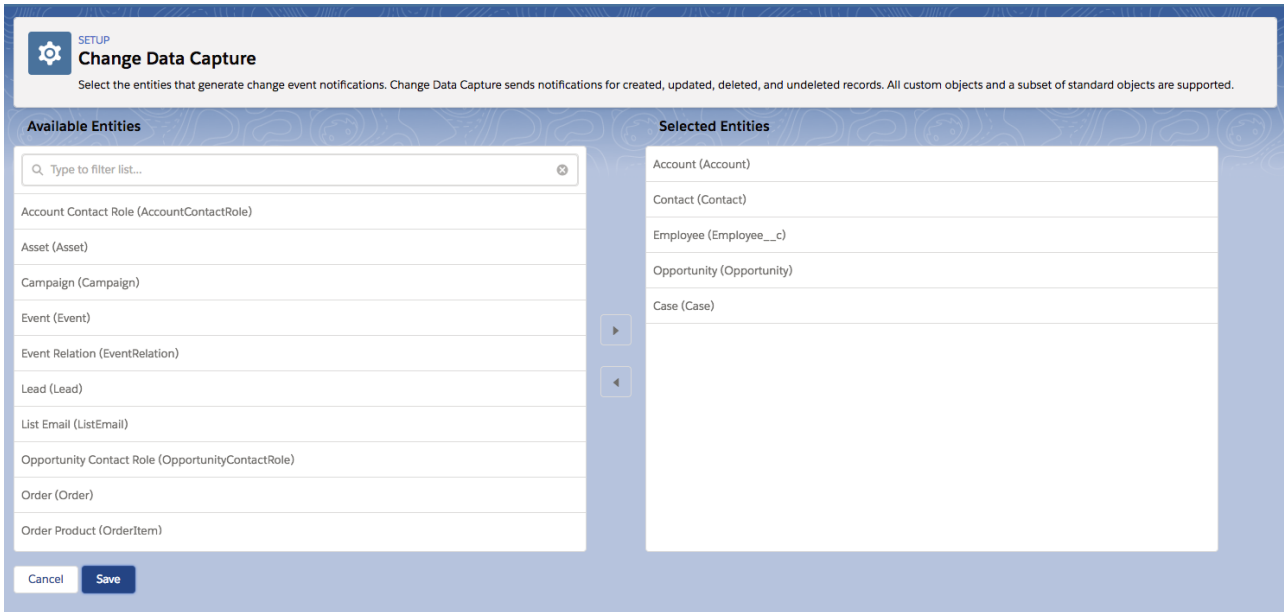
- View Setup and Configuration

To add or modify entity selections:

- Customize Application

 **Note:** The Change Data Capture page shows the object selections for the default standard channel. It doesn't show the selections for custom channels. See [Compose Streams of Change Data Capture Notifications with Custom Channels](#).

Each list entry is in the format "Entity Label (API Name)." Because an entity label can be renamed, the API name is provided in parentheses to better identify the entity.



SEE ALSO:

[Change Data Capture Allocations](#)

Select Objects for Change Notifications with Metadata API and Tooling API

Use PlatformEventChannelMember in Metadata API or Tooling API to create or retrieve object event selections for the default standard channel or a custom channel. The default standard channel, ChangeEvents, corresponds to the selections that you configure in Setup in the Change Data Capture page. For a custom channel, the selections are set when you create the channel member. The SelectedEntity field in PlatformEventChannelMember represents a selected event.

 **Note:** Selections made in PlatformEventChannelMember for a custom channel aren't reflected in the Change Data Capture page. The Change Data Capture page shows selections only for the default standard channel, ChangeEvents, and not for custom channels.

To find out which entities are selected for custom channels, perform a SOQL query on [PlatformEventChannelMember](#) in Tooling API.

To learn how to select objects with Metadata API, see [PlatformEventChannelMember](#) in the *Metadata API Developer Guide*.

To learn how to select objects with Tooling API, see [PlatformEventChannelMember](#) in the *Tooling API Developer Guide*.

To learn about custom channels, see [Compose Streams of Change Data Capture Notifications with Custom Channels](#), [PlatformEventChannel](#) in the *Metadata API Developer Guide*, and [PlatformEventChannel](#) in the *Tooling API Developer Guide*.

Starting with API version 47.0, you define channel member components and channels separately in Metadata API. In API version 45.0 and 46.0, members are included in the PlatformEventChannel component.

SEE ALSO:

[Change Data Capture Allocations](#)

Change Event Message Structure

A change event message contains header fields and record fields.

This event example shows the structure of the payload of an event message received in a Pub/Sub API client.

```
{
  "ChangeEventHeader": {
    "entityName": "...",
    "recordIds": [...],
    "changeType": " ",
    "changeOrigin": " ",
    "transactionKey": " ",
    "sequenceNumber": ,
    "commitTimestamp": ,
    "commitNumber": ,
    "commitUser": " ",
    "nulledFields": [...],
    "diffFields": [...],
    "changedFields": [...]
  },
  "field1": "...",
  "field2": "...",
  ...
}
```



Note: In a Pub/Sub API client, the received event message is in binary Apache Avro format. You can retrieve the schema, replay ID, and payload from the received event separately and decode the payload to obtain the ChangeEventHeader and record fields. This example shows the payload field only. For more information, see [Pub/Sub API as a gRPC API](#) in the Pub/Sub API documentation. Also, the received event in Pub/Sub API contains these ChangeEventHeader fields: `nulledFields` and `diffFields`.

Change Event Fields

The fields that a change event can include correspond to the fields on the associated parent Salesforce object, with a few exceptions. For example, AccountChangeEvent fields correspond to the fields on Account.

The fields that a change event doesn't include are:

- The `IsDeleted` system field.
- The `SystemModStamp` system field.
- Any field whose value isn't on the record and is derived from another record or from a formula, except roll-up summary fields and custom formula fields, which are included. Examples are derived formula fields. Examples of fields with derived values include `LastActivityDate` and `PhotoUrl`.

Each change event also contains header fields. The header fields are included inside the `ChangeEventHeader` field. They contain information about the event, such as whether the change was an update or delete and the name of the object, like `Account`.

API Version and Event Schema

When you subscribe to change events, the subscription uses the latest API version regardless of the API version that the client uses. The event messages received reflect the latest field definitions of the corresponding Salesforce object. When the object schema changes, such as when a field is added or a field type is changed, the schema ID changes. The change event contains the new schema ID in the `schema` field.

You can get the event schema through REST API or Pub/Sub API.

If using Pub/Sub API to subscribe to events, get the event schema with the `GetSchema` RPC method.

```
rpc GetSchema (SchemaRequest) returns (SchemaInfo);
```

For more information, see [GetSchema RPC Method](#) in the [Pub/Sub API Developer Guide](#).

If using a CometD client, get the event schema with REST API. To get the full schema of a change event message, make a GET request to REST API that includes the schema ID sent in the event message:

```
/vXX.X/event/eventSchema/<Schema_ID>?payloadFormat=COMPACT
```

Or make a GET request to this resource.

```
/vXX.X/subjects/<EventName>/eventSchema?payloadFormat=COMPACT
```

`<EventName>` is the name of a change event, such as `AccountChangeEvent`.

The event schema REST API resources return the schema ID in the `uuid` field. To compare the schema with a previous version, retrieve the schema with a previous schema ID and the current schema ID.

The event schema REST API resources are also used for platform events. For more information, see [Platform Event Schema by Event Name](#) and [Platform Event Schema by Schema ID](#) in the [REST API Developer Guide](#).

Change Event Example in Pub/Sub API

This event message is sent for a new account in a Pub/Sub API client.

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "0015f00002J9YEEA3"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "0001ade9-3f74-0b99-dbc4-42e73424b774",
    "sequenceNumber": 1,
    "commitTimestamp": 1712693965000,
  }
}
```

```

    "commitNumber": 1082985383811,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "Name": "Acme",
  "Type": null,
  "ParentId": null,
  "BillingAddress": null,
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": null,
  "AnnualRevenue": null,
  "NumberOfEmployees": null,
  "Ownership": null,
  "TickerSymbol": null,
  "Description": "Sample account record.",
  "Rating": null,
  "Site": null,
  "OwnerId": "0055f000005mc66AAA",
  "CreatedDate": 1712693965000,
  "CreatedById": "0055f000005mc66AAA",
  "LastModifiedDate": 1712693965000,
  "LastModifiedById": "0055f000005mc66AAA",
  "Jigsaw": null,
  "JigsawCompanyId": null,
  "CleanStatus": "Pending",
  "AccountSource": null,
  "DunsNumber": null,
  "Tradestyle": null,
  "NaicsCode": null,
  "NaicsDesc": null,
  "YearStarted": null,
  "SicDesc": null,
  "DandbCompanyId": null,
  "OperatingHoursId": null,
  "CustomerPriority__c": null,
  "SLA__c": null,
  "Active__c": null,
  "NumberOfLocations__c": null,
  "UpsellOpportunity__c": null,
  "SLASerialNumber__c": null,
  "SLAExpirationDate__c": null
  "Custom_Formula_Field_Example_Priority__c": "Low Priority",
  "Custom_Formula_Field_Example_Number__c": 1,
  "Custom_Formula_Field_Example_Email__c": "example@salesforce.com"}
}

```

IN THIS SECTION:

[ChangeEventHeader Fields](#)

Check out the descriptions of the fields that the change event header contains.

[Change Event Body Fields](#)

The body of a change event message includes the fields and values for the corresponding Salesforce record.

ChangeEventHeader Fields

Check out the descriptions of the fields that the change event header contains.

Field Name	Field Type	Description
entityName	string	The API name of the standard or custom object that the change pertains to. For example, Account or MyObject__c.
recordIds	string[]	One or more record IDs for the changed records. Typically, this field contains one record ID. If in one transaction the same change occurred in multiple records of the same object type during one second, Salesforce merges the change notifications. In this case, Salesforce sends one change event for all affected records and the <code>recordIds</code> field contains the IDs for all records that have the same change. Examples of operations with same changes are: • Update of fieldA to valueA in Account records. • Deletion of Account records. • Renaming or replacing a picklist value that results in updating the field value in all affected records. The <code>recordIds</code> field can contain a wildcard value when a change event message is generated for custom field type conversions that cause data loss. In this case, the <code>recordIds</code> value is the three-character prefix of the object, followed by the wildcard character <code>*</code>. For example, for accounts, the value is <code>001*</code>. For more information, see Conversions That Generate a Change Event.
changeType	Enumeration	The operation that caused the change. Can be one of the following values: • CREATE • UPDATE • DELETE • UNDELETE • SNAPSHOT (reserved for future use) For gap events, the change type starts with the <code>GAP_</code> prefix. • GAP_CREATE • GAP_UPDATE • GAP_DELETE • GAP_UNDELETE For overflow events, the change type is <code>GAP_OVERFLOW</code>.

Field Name	Field Type	Description
changeOrigin	string	Only populated for changes done by API apps or from Lightning Experience; empty otherwise. The Salesforce API and the API client ID that initiated the change, if set by the client. Use this field to detect whether your app initiated the change to not process the change again and potentially avoid a deep cycle of changes. The format of the <code>changeOrigin</code> field value is: <pre>com/salesforce/api/<API_Name>/<API_Version>;client=<Client_ID></pre> • <code><API_Name></code> is the name of the Salesforce API used to make the data change. It can take one of these values: <code>soap</code>, <code>rest</code>, <code>bulkapi</code>, <code>xmlrpc</code>, <code>oldsoap</code>, <code>toolingsoap</code>, <code>toolingrest</code>, <code>apex</code>, <code>apexdebuggerrest</code>. • <code><API_Version></code> is the version of the API call that made the change and is in the format <code>XX.X</code>. • <code><Client_ID></code> is a string that contains the client ID of the app that initiated the change. If the client ID is not set in the API call, <code>client=<Client_ID></code> is not appended to the <code>changeOrigin</code> field. Example: <pre>com/salesforce/api/soap/49.0;client=Astro</pre> The client ID is set in the Call Options header of an API call. For an example on how to set the Call Options header, see: • REST API: Sforce-Call-Options Header. (Bulk API and Bulk API 2.0 also use the Sforce-Call-Options header.) • SOAP API: CallOptions Header. (Apex API also uses the CallOptions element.)
transactionKey	string	A string that uniquely identifies each Salesforce transaction. You can use this key to identify and group all changes that were made in the same transaction.
sequenceNumber	int	The sequence of the change within a transaction. The sequence number starts from 1. A lead conversion is an example of a transaction that can have multiple changes. A lead conversion results in the following sequence of changes, all within the same transaction. 1. Create an account 2. Create a contact 3. Create an opportunity 4. Update a lead For more information, see Change Events for Lead Conversion.
commitTimestamp	long	The date and time when the change occurred, represented as the number of milliseconds since January 1, 1970 00:00:00 GMT.
commitUser	string	The ID of the user that ran the change operation.

Field Name	Field Type	Description
<code>commitNumber</code>	<code>long</code>	The system change number (SCN) of a committed transaction, which increases sequentially. This field is provided for diagnostic purposes. The field value is not guaranteed to be unique in Salesforce—it is unique only in a single database instance. If your Salesforce org migrates to another database instance, the commit number might not be unique or sequential.
<code>nulledFields</code>	<code>string[]</code>	Available in Apex triggers and Pub/Sub API only. Not available in CometD. Contains the names of fields whose values were changed to null in an update operation. Use this field to determine if a field was changed to null in an update and isn't an unchanged field. In Pub/Sub API, decode this field before you read its contents. For more information, see Event Deserialization Considerations in the Pub/Sub API Developer Guide .
<code>diffFields</code>	<code>string[]</code>	Available in Apex triggers and Pub/Sub API only. Not available in CometD. Contains the names of fields whose values are sent as a unified diff because they contain large text values. For more information, see Sending Data Differences for Fields of Updated Records . In Pub/Sub API, decode this field before you read its contents. For more information, see Event Deserialization Considerations in the Pub/Sub API Developer Guide .
<code>changedFields</code>	<code>string[]</code>	A list of the fields that were changed in an update operation, including the <code>LastModifiedDate</code> system field. This field is empty for other operations, including record creation. In Pub/Sub API, decode this field before you read its contents. For more information, see Event Deserialization Considerations in the Pub/Sub API Developer Guide .

Change Event Body Fields

The body of a change event message includes the fields and values for the corresponding Salesforce record.

Change Event Messages in Pub/Sub API Clients

Change events received with Pub/Sub API contain all the record fields, including the unchanged fields and empty fields. Check out the details for each type of operation performed.

Create

For a new record, the event message body includes all record and system fields, even if they're empty.

Update

For an updated record, the body includes all record and system fields, even if they're unchanged or empty. Unchanged fields have an empty value even if they have a value on the record. Fields set to null are included with an empty value. To determine which fields have changed, check `changedFields`, after decoding it, in `ChangeEventHeader`. The fields that have changed include fields set to null but if you want to find only the fields that were set to null, check `nulledFields`, after decoding it, in

`ChangeEventHeader`. For details about decoding fields in `ChangeEventHeader`, see [Event Deserialization Considerations](#) in the *Pub/Sub API Developer Guide*.

Delete

For a deleted record, the body doesn't include any values for record or system fields. All record and system fields are included but with empty values.

Undelete

For an undeleted record, the body includes all record and system fields from the original record. If fields are empty, they're included with empty values.

Apex Change Event Messages

Fields in a change event message are statically defined, just like in any other Apex type. As a result, all record fields are available in the change event message received in an Apex trigger, regardless of the operation performed. The event message can contain empty (null) fields.

Create

For a new record, the event message contains all fields, whether populated or empty. It includes fields with default values and system fields, such as `CreatedDate` and `OwnerId`.

Update

For an updated record, the event message contains field values only for changed fields. Unchanged fields are present and empty (null), even if they contain a value in the record. The event message also contains the `LastModifiedDate` system field. The body includes the `LastModifiedById` field only if it has changed—if the user who modified the record is different than the previous user who saved it.

Delete

For a deleted record, all record fields in the event message are empty (null).

Undelete

For an undeleted record, the event message contains all fields from the original record, including empty (null) fields and system fields.

JSON Change Event Messages in CometD Clients

The fields that Salesforce includes in a JSON event message that a CometD client receives depend on the operation performed.

Create

For a new record, the event message body includes all non-empty fields and system fields, such as the `CreatedDate` and `OwnerId` fields.

Update

For an updated record, the body includes only the changed fields. It includes empty fields only if they're updated to an empty value (null). It also includes the `LastModifiedDate` system field. The body includes the `LastModifiedById` field only if it has changed—if the user who modified the record is different than the previous user who saved it.

Delete

For a deleted record, the body doesn't include any fields or system fields.

Undelete

For an undeleted record, the body includes all non-empty fields from the original record, in addition to system fields.

For examples of change event messages, see the [Change Data Capture Basics](#) Trailhead module.

Merged Change Events

For efficiency, sometimes change events for one transaction are merged into one event if the same change occurred in multiple records of the same object type during one second.

When change events are merged, Salesforce sends one change event for all affected records and the `recordIds` field contains the IDs for all records that have the same change.

Examples of operations with same changes are:

- Update of fieldA to valueA in Account records.
- Deletion of Account records.
- Renaming or replacing a picklist value that results in updating the field value in all affected records.

For more information about the `recordIds` field, see [ChangeEventHeader Fields](#).

 **Example:** If you update the `Industry` field to `Apparel` of three Account records in a single update Apex DML statement, one merged change event is sent as shown in this example. The `recordIds` field contains the IDs of the Account records that have the same change.

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "0015f00002JUZPDAA5",
      "0015f00002JUZPXA5",
      "0015f00002JUZPcAAP"
    ],
    "changeType": "UPDATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=devconsole",
    "transactionKey": "00065380-d1a9-a64a-9341-14f6f12f674c",
    "sequenceNumber": 1,
    "commitTimestamp": 1714170102000,
    "commitNumber": 1100823480049,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": [
      "0x400800"
    ]
  },
  "Name": null,
  "Type": null,
  "ParentId": null,
  "BillingAddress": null,
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": "Apparel",
  "AnnualRevenue": null,
  "NumberOfEmployees": null,
  "Ownership": null,
}
```

```

    "TickerSymbol": null,
    "Description": null,
    "Rating": null,
    "Site": null,
    "OwnerId": null,
    "CreatedDate": null,
    "CreatedById": null,
    "LastModifiedDate": 1714170102000,
    "LastModifiedById": null,
    "Jigsaw": null,
    "JigsawCompanyId": null,
    "CleanStatus": null,
    "AccountSource": null,
    "DunsNumber": null,
    "Tradestyle": null,
    "NaicsCode": null,
    "NaicsDesc": null,
    "YearStarted": null,
    "SicDesc": null,
    "DandbCompanyId": null,
    "OperatingHoursId": null,
    "CustomerPriority__c": null,
    "SLA__c": null,
    "Active__c": null,
    "NumberOfLocations__c": null,
    "UpsellOpportunity__c": null,
    "SLASerialNumber__c": null,
    "SLAExpirationDate__c": null
  }

```

After being decoded in a Pub/Sub API client, the `changedFields` field lists the `Industry` field as one of the changed fields.

```

2024-04-26 15:21:43,674 [grpc-default-executor-0] java.lang.Class - 
=====
2024-04-26 15:21:43,674 [grpc-default-executor-0] java.lang.Class -           Changed
Fields
2024-04-26 15:21:43,674 [grpc-default-executor-0] java.lang.Class - 
=====
2024-04-26 15:21:43,674 [grpc-default-executor-0] java.lang.Class - Industry
2024-04-26 15:21:43,674 [grpc-default-executor-0] java.lang.Class - LastModifiedDate
2024-04-26 15:21:43,674 [grpc-default-executor-0] java.lang.Class - 
=====

```

Other Types of Change Events: Gap and Overflow Events

Other types of change events are provided to handle special situations, such as capturing changes not caught in the Salesforce application servers, or handling high loads of changes.

IN THIS SECTION:

[Gap Events](#)

Salesforce sometimes sends gap events instead of change events to inform subscribers about errors, or if it's not possible to generate change events. A gap event contains information about the change in the header, such as the change type and record ID. It doesn't include details about the change, such as record fields.

[Overflow Events](#)

To capture changes more efficiently, overflow events are generated for single transactions that exceed a threshold.

Gap Events

Salesforce sometimes sends gap events instead of change events to inform subscribers about errors, or if it's not possible to generate change events. A gap event contains information about the change in the header, such as the change type and record ID. It doesn't include details about the change, such as record fields.

The conditions that cause gap events include:

- The change event size exceeds the maximum 1 MB message size.
- Some field type conversions of custom fields. For more information, see [Conversions That Generate a Gap Event](#) on page 107.
- When an internal error occurs in Salesforce preventing the change event from being generated.
- Changes that occur outside the application server transaction and are applied directly in the database. For example, archiving of activities or a data cleanup job in the database. To not miss these operations, gap events are generated to notify you about those changes.

Gap events can have one of these `changeType` values in the event header.

- `GAP_CREATE`
- `GAP_UPDATE`
- `GAP_DELETE`
- `GAP_UNDELETE`

 **Note:** A `changeType` value of `GAP_OVERFLOW` means that the event is an overflow event. For more information, see [Overflow Events](#).

Upon receiving a gap event message, your application can retrieve the Salesforce record using the record ID value to get the current data for your system. For more information about handling gap events, see [How to Handle a Gap Event](#) in [Transaction-Based Replication Steps](#).

The gap event's `transactionKey` represents the internal database transaction ID if the change was applied at the database layer, outside an application server transaction. If the gap event was emitted due to other reasons, such as hitting the 1 MB event size limit or an internal error, the `transactionKey` holds the application server transaction ID.

 **Note:** If the same type of change occurs on the same Salesforce entity within the same transaction, sometimes multiple gap events are merged into a single gap event. The IDs of the changed records are included in the `recordIds` header field. For more information, see [Merged Change Events](#).

 **Example:** This sample gap event is for an account creation and contains information about the change in the header. The change type is `GAP_CREATE`.

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
```

```

        "001ZM000001n4n5YAA"
    ],
    "changeType": "GAP_CREATE",
    "changeOrigin": "",
    "transactionKey": "000a50de-05dd-07c4-22fb-44b7f9e72ab5",
    "sequenceNumber": 19,
    "commitTimestamp": 1714417112000,
    "commitNumber": 72784468115,
    "commitUser": "005ZM000000Q6ipYAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
},
{Name": null,
Type": null,
ParentId": null,
BillingAddress": null,
ShippingAddress": null,
Phone": null,
Fax": null,
AccountNumber": null,
Website": null,
Sic": null,
Industry": null,
AnnualRevenue": null,
NumberOfEmployees": null,
Ownership": null,
TickerSymbol": null,
Description": null,
Rating": null,
Site": null,
OwnerId": null,
CreatedDate": null,
CreatedBy": null,
LastModifiedDate": null,
LastModifiedBy": null,
Jigsaw": null,
JigsawCompanyId": null,
AccountSource": null,
SicDesc": null,
External_Account_ID__c": null
}

```

SEE ALSO:

[SOAP API Developer Guide: retrieve\(\)](#)

[Force.com SOQL and SOSL Reference](#)

Overflow Events

To capture changes more efficiently, overflow events are generated for single transactions that exceed a threshold.

The first 100,000 changes generate change events. The set of changes beyond that amount generates one overflow event for each entity type included in that set. An overflow event is generated when a single transaction contains more than 100,000 changes. An overflow event contains only header fields. The `changeType` field header value is `GAP_OVERFLOW` instead of the specific type of change. The object type corresponding to the change is in the `entityName` field. An overflow event doesn't include details about the change, such as the record fields or record ID.

A record creation, deletion, or undeletion counts as one change toward the threshold. In a record update, each field change counts toward the overflow threshold. For example, if three field values are modified in one record update, they count as three operations against the overflow threshold.

Transactions with a high volume of operations aren't frequent, but they can occur in certain situations, such as for a recurring event with hundreds of occurrences and attendees. Another example is a cascade delete of accounts associated with many opportunities, contacts, and activities that results in deleting many more records in the same transaction. If the cascade delete results in the deletion of 120,000 account, opportunity, contact, and activity records in the same transaction, the deletions of the first 100,000 records generate delete change events. The remaining 20,000 records generate one overflow event for each unique entity.

 **Note:** Because changes are sometimes merged in one change event, the number of generated change events isn't always equal to the number of changes. For example, the consecutive deletion of accounts can be merged into one change event. For more information, see the `recordIds` field in [ChangeEventHeader Fields](#). If Apex triggers fire and create other records, more change events are generated in the same transaction. For more information, see [Merged Change Events](#).

For more information about handling overflow events, see [How to Handle an Overflow Event](#) in [Transaction-Based Replication Steps](#).

 **Example:** This overflow event is for an account and contains information about the change in the header. The change type is `GAP_OVERFLOW`. The record ID for the change is always set to `0000000000000000AAA`, which is the empty record ID.

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "0000000000000000AAA"
    ],
    "changeType": "GAP_OVERFLOW",
    "changeOrigin": "com/salesforce/api/soap/61.0;client=Workbench/",
    "transactionKey": "000a5148-405c-21fe-86ce-03205d7404ad",
    "sequenceNumber": 6,
    "commitTimestamp": 1714417568000,
    "commitNumber": 72784848482,
    "commitUser": "005ZM000000M6o1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "Name": null,
  "Type": null,
  "ParentId": null,
  "BillingAddress": null,
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": null,
  "AnnualRevenue": null,
```

```
"NumberOfEmployees": null,
"Ownership": null,
"TickerSymbol": null,
"Description": null,
"Rating": null,
"Site": null,
"OwnerId": null,
"CreatedDate": null,
"CreatedById": null,
"LastModifiedDate": null,
"LastModifiedById": null,
"Jigsaw": null,
"JigsawCompanyId": null,
"AccountSource": null,
"SicDesc": null,
"External_Account_ID__c": null
}
```

Subscribe to Change Events

Learn about subscribing to change events including subscription channels, subscription methods, and required permissions.

IN THIS SECTION:

[Change Event Storage and Delivery](#)

Change events are stored temporarily and subscribers can retrieve them during the retention window. The order of events delivered is based on the order of the corresponding committed transactions. Users with the proper permissions can receive events on a channel.

[Subscription Channels](#)

A subscription channel is a stream of change events that correspond to one or more entities. You can subscribe to a channel to receive change event notifications for record create, update, delete, and undelete operations. Change Data Capture provides predefined standard channels and you can create your own custom channels. Use the subscription channel that corresponds to the change events you want to receive. The channel name is case-sensitive.

[Compose Streams of Change Data Capture Notifications with Custom Channels](#)

Create a custom channel if you have multiple subscribers and each subscriber receives change events from a different set of entities. Also, use a custom channel with event enrichment to isolate sending enriched fields in change events on a specific channel. Custom channels group and isolate change events for each subscriber so subscribers receive only the types of events they need.

[Example Diagrams for Channels and Channel Members](#)

Discover the relationship between channels and channel members with the Entity Relationship Diagram (ERD). Also, understand the benefits of using custom channels through the example diagrams.

[Transaction-Based Replication Steps](#)

To maintain an accurate replica of your Salesforce org's data in another system, subscribe using a transaction-based approach.

[Subscribe with Pub/Sub API](#)

Use Pub/Sub API to subscribe to event messages in an external client to integrate your systems. Simplify your development by using one API to publish, subscribe, and retrieve the event schema. Based on gRPC and HTTP/2, Pub/Sub API enables efficient delivery of binary event messages in the Apache Avro format. You can control the volume of event messages received per Subscribe call based on event processing speed in the client.

[Get Compound Fields in Change Events](#)

Compound fields, such as lead or contact Name, Address, and Geolocation fields, are represented as nested field structures in the event message. In record updates, the `changedFields` header field lists each updated component field using this format: *CompoundField.ComponentField*. The updated component field is included in the event message in a nested field structure.

[Enrich Change Events with Extra Fields](#)

Change event messages include values for new and changed fields, but sometimes unchanged field values are needed for processing or replicating data. For example, use enrichment when your app needs an external ID field for matching records in an external system. Or always include a field that provides important information about the changed record. You can select any field whose type is supported.

[Filter Your Stream of Change Events with Channels](#)

Receive only the change event messages that match a predefined filter on a channel in subscribers. With fewer events delivered to subscribers, event processing is optimized. Also, subscribers make more efficient use of the event delivery allocation. This feature supports Pub/Sub API, CometD (Streaming API), and event relays but not Apex triggers.

[Subscribe with Apex Triggers](#)

With Apex triggers, you can capture and process change events on the Lightning Platform. Change event triggers run asynchronously after the database transaction is completed. Perform resource-intensive business logic asynchronously in the change event trigger, and implement transaction-based logic in the Apex object trigger. By decoupling the processing of changes, change event triggers can help reduce transaction processing time.

SEE ALSO:

[General Considerations](#)

[Streaming API Developer Guide](#)

Change Event Storage and Delivery

Change events are stored temporarily and subscribers can retrieve them during the retention window. The order of events delivered is based on the order of the corresponding committed transactions. Users with the proper permissions can receive events on a channel.

Temporary Storage in the Event Bus

Change events are based on platform events and share some of their characteristics for storage. Change event messages are stored in the event bus for three days. You can retrieve stored event messages from the event bus. Each event message contains the `ReplayId` field, which identifies the event in the stream and enables replaying the stream after a specific event. See [Event Message Durability](#) in the *Pub/Sub API Developer Guide*.

Order of Events

The order of change events stored in the event bus corresponds to the order in which the transactions corresponding to the record changes are committed in Salesforce. If a transaction includes multiple changes, like a lead conversion, a change event is generated for each change with the same `transactionKey` but different `sequenceNumber` in the header. The `sequenceNumber` is the order of the change within the transaction.

When Salesforce receives a change event, it assigns a replay ID value to it and persists it in the event bus. Subscribers receive change events from the event bus in the order of the replay ID.

User Permissions Required

The subscriber must have one or more of the following permissions depending on the subscription channel: View All Data, View All Users, and View All Records for an object. See [Required Permissions for Change Event Subscribers](#).

SEE ALSO:

[Transaction-Based Replication Steps](#)

Subscription Channels

A subscription channel is a stream of change events that correspond to one or more entities. You can subscribe to a channel to receive change event notifications for record create, update, delete, and undelete operations. Change Data Capture provides predefined standard channels and you can create your own custom channels. Use the subscription channel that corresponds to the change events you want to receive. The channel name is case-sensitive.

You can subscribe to channels by supplying the channel endpoint. For example, to subscribe to events for all selected entities, subscribe to `/data/ChangeEvents`. Apex triggers can't subscribe to channels but can subscribe to a single event only. For example, you can create an Apex trigger on `AccountChangeEvent` to subscribe to only Account change events.

Standard Channels

The `ChangeEvents` standard channel contains change events from one or more selected entities in a single stream that you can subscribe to. If you expect change events from more than one entity, use the `ChangeEvents` standard channel. To receive change events on the `ChangeEvents` channel, select the entities for Change Data Capture. For more information, see [Select Objects for Change Notifications in the User Interface](#) and [Select Objects for Change Notifications with Metadata API and Tooling API](#). Then subscribe to the appropriate channel.

If you expect change events for only a single entity, use single-entity channels. With single-entity channels, you can subscribe to change events from only one custom object or standard object. Select the entity for notifications on the Change Data Capture page in Setup or in a custom channel.

Standard Channel for All Selected Entities

```
/data/ChangeEvents
```

Single-Entity Channel for a Standard Object

```
/data/<Standard_Object_Name>ChangeEvent
```

For example, the channel to subscribe to change events for Account records is:

```
/data/AccountChangeEvent
```

Single-Entity Channel for a Custom Object

```
/data/<Custom_Object_Name>__ChangeEvent
```

For example, the channel to subscribe to change events for `Employee__c` custom object records is:

```
/data/Employee__ChangeEvent
```

Custom Channels

Create a custom channel if you have multiple subscribers and each subscriber receives change events from a different set of entities. Also, use a custom channel with event enrichment to isolate sending enriched fields in change events on a specific channel. Custom channels group and isolate change events for each subscriber so subscribers receive only the types of events they need. Entities are automatically selected for change event notifications when you create a custom channel that includes them. A custom channel has the following format.

```
/data/YourChannelName__chn
```

For example, if your channel name is `SalesEvents`, the subscription channel is:

```
/data/SalesEvents__chn
```

SEE ALSO:

[Required Permissions for Change Event Subscribers](#)

[Compose Streams of Change Data Capture Notifications with Custom Channels](#)

[Enrich Change Events with Extra Fields](#)

[Example Diagrams for Channels and Channel Members](#)

[General Considerations](#)

Compose Streams of Change Data Capture Notifications with Custom Channels

Create a custom channel if you have multiple subscribers and each subscriber receives change events from a different set of entities. Also, use a custom channel with event enrichment to isolate sending enriched fields in change events on a specific channel. Custom channels group and isolate change events for each subscriber so subscribers receive only the types of events they need.

For example, if a subscriber uses real-time information about sales objects such as Account, Contact, or Order, you can create a custom channel with these objects. When you subscribe to the custom channel, you receive change events only for these objects. Your subscriber doesn't receive change events of entities selected in another channel.

You can create a custom channel with Metadata API or Tooling API. When you create a custom channel, the objects are selected for notifications when you add a `PlatformEventChannelMember`. Custom channels can't be created or viewed in the user interface on the Change Data Capture page. Use Metadata API to deploy or retrieve channel metadata in your org with a supported tool. Use Tooling API to create channels using REST and query channel metadata with SOQL.

Also, you can package channels to distribute with your apps.

In Metadata API, use the `PlatformEventChannel` metadata type to create a custom channel and the `PlatformEventChannelMember` type to add the selected event entities. For more information, see [PlatformEventChannel](#) and [PlatformEventChannelMember](#) in the [Metadata API Developer Guide](#).

In Tooling API, use the PlatformEventChannel object to create a custom channel and PlatformEventChannelMember to add the selected event entities. For more information, see [PlatformEventChannel](#) and [PlatformEventChannelMember](#) in the [Tooling API](#).

SEE ALSO:

[Subscription Channels](#)

[Required Permissions for Change Event Subscribers](#)

[Change Data Capture Allocations](#)

[Enrich Change Events with Extra Fields](#)

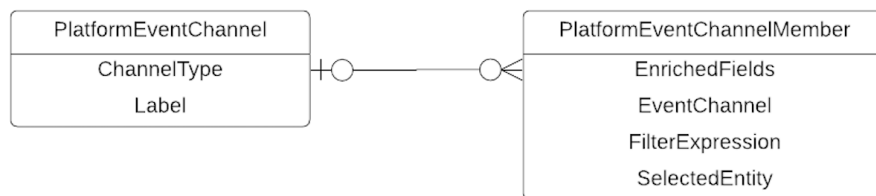
[Example Diagrams for Channels and Channel Members](#)

Example Diagrams for Channels and Channel Members

Discover the relationship between channels and channel members with the Entity Relationship Diagram (ERD). Also, understand the benefits of using custom channels through the example diagrams.

Entity Relationship Diagram for Channel and Channel Member

This ERD shows the channel and channel member entities and the relationships between them. You can access the entities by their corresponding types and objects in Metadata API and Tooling API. The entities in this diagram don't include the FullName field. FullName is the unique name of the Metadata API component or Tooling API object and is used to perform operations on them.



A channel can have zero or more channel members. A channel member can have zero or more enriched fields, and zero or one filter expression. You can add and update enriched fields and filter expressions through the PlatformEventChannelMember entities in Metadata API or Tooling API.

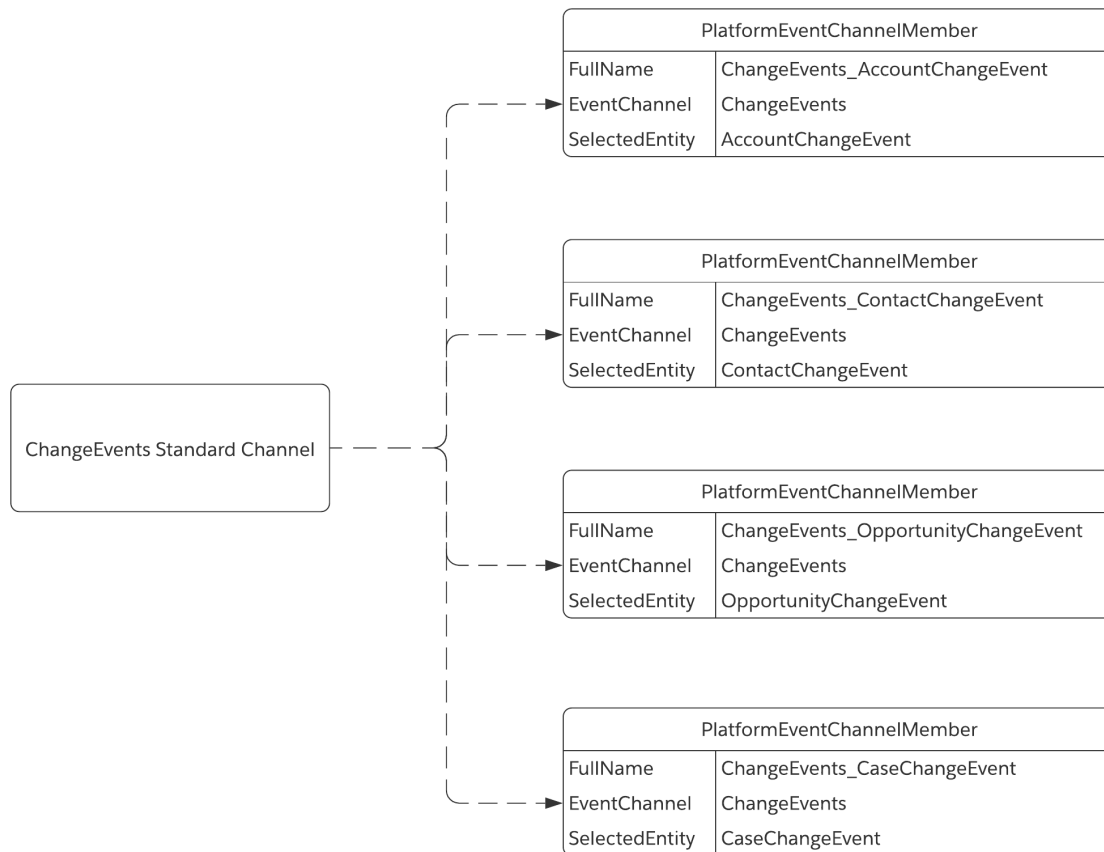
In API version 47.0 and later, the PlatformEventChannel in Metadata API and Tooling API represents a custom channel but not the standard ChangeEvents channel. You can't create the ChangeEvents standard channel. Also, you can't modify the ChangeEvents standard channel attributes: ChannelType and Label.

The diagrams in the next sections show examples of selected entities on the default ChangeEvents channel and on custom channels. The examples show the benefits of using custom channels.

Example for the ChangeEvents Standard Channel

This diagram is an example of four entities selected for the ChangeEvents standard channel (Account, Contact, Opportunity, and Case). The channel members don't contain enriched fields or filter expressions. Even though you can add them to members belonging to the ChangeEvents channel, it's best to add them on custom channels. That way, you isolate change events with enriched fields when using multiple subscribers.

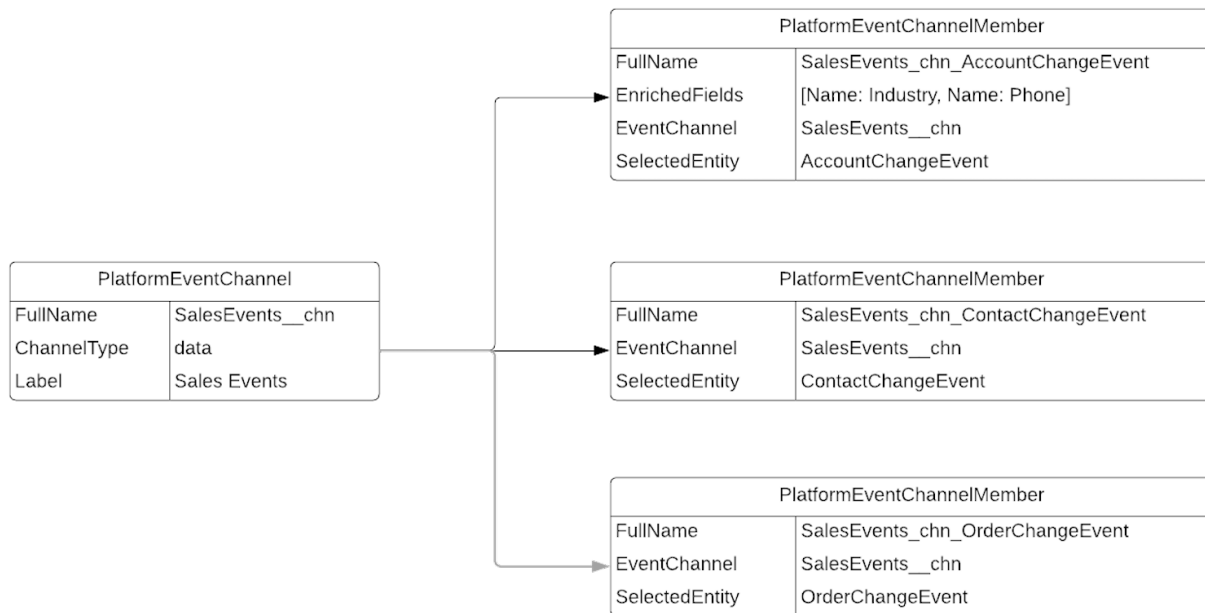
In this diagram, the ChangeEvents channel isn't represented as a PlatformEventChannel entity. The reason is, in API version 47.0 and later, the ChangeEvents standard channel can't be manipulated directly through Metadata API or Tooling API. After all, it's a standard channel.



The AccountChangeEvent selected entity in one channel member is also part of custom channel examples in the next sections. On one custom channel, AccountChangeEvent contains enriched fields. On another custom channel, it contains a filter expression. These fields aren't present on the ChangeEvents channel, so they aren't sent to subscribers on this channel.

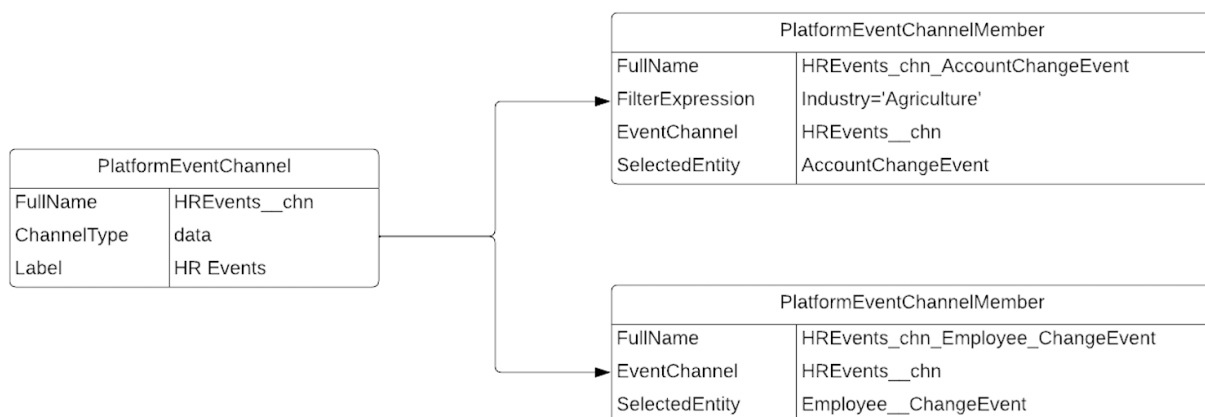
Example for SalesEvents__chn Custom Channel with Enriched Fields

This diagram shows how you can use a custom channel to isolate change events so a subscriber receives only the events they're interested in. In our example, the SalesEvents__chn channel contains a subset of the selected entities of the ChangeEvents channel: Account and Contact. It also contains one other entity: Order. One of the channel member's selected entities, AccountChangeEvent, contains two enriched fields in the EnrichedFields field. The EnrichedFields field is an array containing the name of each enriched field: the Industry and the Phone fields. The enriched field values are available in the account change events on this channel and not on other channels when not specified.



Example for HREvents__chn Custom Channel with a Filter Expression

This diagram shows an example of a custom channel, `HREvents__chn`, designed for entities related to an HR app: `Account` and the `Employee__c` custom object. The `HREvents__chn` channel contains a filter expression for the `AccountChangeEvent` member. The channel delivers account change events that match the criteria specified in the filter expression. Only change events for accounts whose `Industry` field is `Agriculture` are delivered. The channel member for `Employee__ChangeEvent` doesn't have a filter, so all `Employee__c` events are delivered on that channel.



`AccountChangeEvent` is a selected entity in members of two other channels: the standard `ChangeEvents` channel and the custom `SalesEvents__chn`. A subscriber on the `ChangeEvents` channel receives account change events without enriched fields or a filter. A subscriber on the `SalesEvents__chn` custom channel receives account change events enriched with the `Industry` and `Phone` field values.

but without a filter. A subscriber on the HREvents__chn custom channel receives account change events that match the filter criteria—they have the Industry field value of Agriculture. No enriched fields are specified, but the account change events received on the HREvents__chn channel are enriched with the Industry field due to filtering's auto-enrichment feature.

SEE ALSO:

[Enrich Change Events with Extra Fields](#)

[Filter Your Stream of Change Events with Channels](#)

[Metadata API Developer Guide: PlatformEventChannel](#)

[Metadata API Developer Guide: PlatformEventChannelMember](#)

[Tooling API Developer Guide: PlatformEventChannel](#)

[Tooling API Developer Guide: PlatformEventChannelMember](#)

Transaction-Based Replication Steps

To maintain an accurate replica of your Salesforce org's data in another system, subscribe using a transaction-based approach.

Types of Events That Change Data Capture Can Generate: Change Events, Gap Events, and Overflow Events

Generally, Salesforce captures record changes by sending change events, which the subscriber receives to synchronize data in an external system. Sometimes, gap events or overflow events are generated.

Gap events are generated when change events can't be generated. They inform subscribers about errors or operations done outside of Salesforce application servers. Gap events don't contain record data, but they contain the record ID, which enables you to retrieve the record from Salesforce. Ensure that the subscriber expects to receive gap events and handles them properly, as outlined in the next section. The `changeType` field in the gap event header identifies the gap event and the associated operation, and can take one of these values:

- `GAP_CREATE`
- `GAP_UPDATE`
- `GAP_DELETE`
- `GAP_UNDELETE`

For more information about gap events, see [Gap Events](#).

Overflow events are generated when a single transaction involves more than 100,000 changes. The first 100,000 changes generate change events. The set of changes beyond that amount generates one overflow event for each entity type included in that set. Overflow events include header fields but no record data and no record ID. Ensure that the subscriber handles overflow events. The `changeType` field header value is `GAP_OVERFLOW` instead of the specific type of change.

For more information about overflow events, see [Overflow Events](#).

Transaction-Based Replication Approach

Each change event contains a transaction key in the header that uniquely identifies the transaction that the change is part of. Each change event also contains a sequence number that identifies the sequence of the change within a transaction. The sequence number is useful for operations that include multiple steps, such as lead conversion. If not all objects involved in a transaction are enabled for Change Data Capture, there will be a gap in the sequence numbers. We recommend that you replicate all the changes in one transaction as a single commit in your system. One approach is to buffer all changes related to a transaction and commit them all at once.

If you choose not to use a transaction-based replication process, your replicated data can be incomplete if your subscription stops. For example, if your subscription stops in the middle of an event stream for one transaction, only part of the transaction's changes are replicated in your system.

A transaction-based replication process involves these high-level steps.

1. In your subscribed client, allocate a transaction buffer for each transaction key. For example, create a map (`Map<String, List<ChangeEvent>>`) where the key is the `transactionKey` value.
2. Open a subscription to the general `/data/ChangeEvents` channel that captures all enabled events.
3. For each change event received over the channel, check the `changeType` field.
 - a. If the `changeType` field is `GAP_CREATE`, `GAP_UPDATE`, `GAP_DELETE`, or `GAP_UNDELETE`, the event is a gap event. Follow the recommended steps in *How to Handle a Gap Event*.
 - b. If the `changeType` field is `GAP_OVERFLOW`, the event is an overflow event.
 - i. Process the change events that you previously stored in the map. Commit the changes, and then purge the corresponding map entry.
 - ii. For the overflow event, follow the recommended steps in *How to Handle an Overflow Event*.
 - iii. n
4. If the event isn't a gap or overflow event, it's a change event. Deserialize the change event, and add it to the appropriate map entry for the transaction key.
5. When the `transactionKey` value changes in the next change event, commit the changes in the map entry for the previous transaction key, and then purge the map entry.
6. Repeat steps 3 through 5 for each new event received.

How to Handle a Gap Event

If the event that the subscriber receives is a gap event, get the latest data from Salesforce. The gap event includes the ID of the affected record enabling you to retrieve the record. After receiving the gap event, one approach is to mark the corresponding record as dirty and not process any change events for that record until it has been reconciled.

Let's look at an example to examine the steps a subscriber can take to handle a gap event while change events are also received. Records A and B are modified in a transaction and generate two change events. Then a change for record C generates a gap event. The subscriber receives three events: two change events for record A and B and one gap event for record C. The steps for the subscriber are:

1. Handle the change events according to the transaction-based replication process.
2. For the gap event, mark the corresponding record as dirty locally as of the date of the gap event.
3. If you receive change events for new changes for the same record before the data has been reconciled, don't process them. For example, if record C is modified again and a change event is received, ignore it because the corresponding record is marked as dirty. To ensure that the change is after the gap event, compare the `commitTimestamp` fields of both events. To ensure that the change occurred before the data is reconciled, compare the `LastModifiedDate` fields on the change event and the record retrieved in the next step.
4. Reconcile the data for record C. Make a Salesforce API call, such as a REST API call, to retrieve the full data for record C, and save it in your system. Then clear the dirty flag on that record.
5. Record C is modified again and a new change event is received. Process this change event according to the replication process because the record is no longer dirty.



Note: If the same type of change occurs on the same entity within the transaction, sometimes multiple gap events are merged into a single gap event. The IDs of the changed records are included in the `recordIds` header field. Use these IDs to reconcile all the referenced records. For more information, see [Merged Change Events](#).

How to Handle an Overflow Event

If a change results in more than 100,000 events in a single transaction, you receive overflow events for the events sent after the first 100,000. One overflow event is generated for each entity type. Mass changes aren't frequent. They can result from creating or modifying many records, such as changing a recurring calendar event series with many occurrences and invitees. A large change can also result from a cascade delete when deleting records with many related records.

An overflow event doesn't contain the record ID and only a dummy record ID, so one approach for data replication is to retrieve all records of the corresponding entity after an overflow event is received. Then you can update or delete those records in the external system. This approach can be the most process-intensive because it resyncs all the records for an entity. However, it's the simplest approach because it doesn't require figuring out which records changed in a particular timeframe and filtering out the records that resulted in change events. These steps outline the process of reconciling data when the overflow event is received.

1. After you receive an overflow event in your subscriber, unsubscribe from the channel, and stop processing further events. This step is in preparation of a full data synchronization for the entity.
2. Store the Replay ID of the overflow event. This ID is the starting point for the data reconciliation.
3. Reconcile the data for new, updated, and undeleted records.
 - a. Retrieve all records for the entity. Depending on the volume of records stored, this process can take some time.
 - b. Synchronize the data in your system by overwriting it with the retrieved data from Salesforce.
4. Reconcile the data for deleted records by performing one of the following steps.
 - a. Get the non-deleted records from Salesforce, and synchronize.
 - i. Identify all records for that entity in your system that weren't updated through the synchronization that you performed in step 3. These records are the deleted ones.
 - ii. Delete the identified records from your system.
 - b. Or get the deleted records from Salesforce, and synchronize.
 - i. Query all records for the entity with `isDeleted=true`. You get all the soft-deleted records for that entity that are in the Recycle Bin.
 - ii. Identify the records in your system that match the records returned in the previous step.
 - iii. Delete the identified records from your system.
5. Resubscribe to the stored event bus stream starting from the Replay ID you saved earlier.
6. We recommend that you process all change events after that Replay ID. This way, you catch up on any data changes that happened during the synchronization and weren't saved in your system.
7. If you encounter an overflow event for another entity (`entityName` field value), repeat this process for that entity.

Subscribe with Pub/Sub API

Use Pub/Sub API to subscribe to event messages in an external client to integrate your systems. Simplify your development by using one API to publish, subscribe, and retrieve the event schema. Based on gRPC and HTTP/2, Pub/Sub API enables efficient delivery of binary event messages in the Apache Avro format. You can control the volume of event messages received per Subscribe call based on event processing speed in the client.

The Pub/Sub API service is defined in a proto file, with RPC method parameters and return types specified as protocol buffer messages. Pub/Sub API serializes the response of a Subscribe RPC call based on the protocol buffer message type specified in the proto file. For more information, see [What is gRPC?](#) and [Protocol Buffers](#) in the gRPC documentation, and [pubsub_api.proto](#) in the [Pub/Sub API GitHub repository](#).

The `Subscribe` method uses bidirectional streaming, enabling the client to request more events as it consumes events. The client can control the flow of events received by setting the number of requested events in the `FetchRequest` parameter.

```
rpc Subscribe (stream FetchRequest) returns (stream FetchResponse);
```

Salesforce sends platform events to Pub/Sub API clients sequentially in the order they're received. The order of event notifications is based on the replay ID of events. A client can receive a batch of events at once. The total number of events across all batches received in `FetchResponses` per `Subscribe` call is equal to the number of events the client requests. The number of events in each individual batch can vary. If the client uses a buffer for the received events, ensure that the buffer size is large enough to hold all event messages in the batch. The buffer size needed depends on the publishing rate and the event message size. We recommend you set the buffer size to 3 MB.

To learn more about the RPC methods in Pub/Sub API, see [Pub/Sub API RPC Method Reference](#) in the [Pub/Sub API Developer Guide](#).

The channel name is case-sensitive. To subscribe to a standard channel, use this format.

```
/data/Channel1_Name
```

For example, you can subscribe to all events by providing the standard `ChangeEvent` channel.

```
/data/ChangeEvent
```

To subscribe to a custom channel, use this format.

```
/data/Channel1_Name__chn
```

For more information about channels, see [Subscription Channels](#).



Example: After you select Opportunity for change data capture, you can subscribe to opportunity change events by supplying this channel.

```
/data/OpportunityChangeEvent
```

This example shows the fields of a change event received for a new opportunity. This example prints out the payload only. The received event message also contains the schema ID and the event ID, in addition to the payload.

```
{
  "ChangeEventHeader": {
    "entityName": "Opportunity",
    "recordIds": [
      "006SM0000001Tb3YAE"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/55.0;client=SfdcInternalAPI/",
    "transactionKey": "00000466-3d4f-bbe3-9c2b-5ab0fb45cc02",
    "sequenceNumber": 1,
  }
}
```

```

    "commitTimestamp": 1652977933000,
    "commitNumber": 1652977933433528300,
    "commitUser": "005SM000000146PYAQ",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "AccountId": "001SM0000000iibYAA",
  "IsPrivate": null,
  "Name": "Acme - 400 Widgets",
  "Description": null,
  "StageName": "Prospecting",
  "Amount": 40000,
  "Probability": 10,
  "ExpectedRevenue": null,
  "TotalOpportunityQuantity": null,
  "CloseDate": 1653955200000,
  "Type": null,
  "NextStep": null,
  "LeadSource": null,
  "IsClosed": false,
  "IsWon": false,
  "ForecastCategory": "Pipeline",
  "ForecastCategoryName": "Pipeline",
  "CampaignId": null,
  "HasOpportunityLineItem": false,
  "Pricebook2Id": null,
  "OwnerId": "005SM000000146PYAQ",
  "CreatedDate": 1652977933000,
  "CreatedBy": "005SM000000146PYAQ",
  "LastModifiedDate": 1652977933000,
  "LastModifiedBy": "005SM000000146PYAQ",
  "LastStageChangeDate": null,
  "ContactId": null,
  "ContractId": null,
  "LastAmountChangedHistoryId": null,
  "LastCloseDateChangedHistoryId": null
}

```

Pub/Sub API is used for system integration and isn't intended for end-user scenarios. The binary event format enables efficient delivery of lightweight messages. As a result, after decoding the event payload, some fields aren't human readable and require additional processing. For example, `CreatedDate` is in Epoch time and can be converted to another date format for readability. Also, `nulledFields`, `diffFields`, and `changedFields` require further processing. For more information, see [Event Deserialization Considerations](#) in the *Pub/Sub API Developer Guide*.

The event schema is versioned—when the schema changes, the schema ID changes as well. For more information about retrieving the event schema, see [Get the Event Schema with Pub/Sub API](#).

Write a Pub/Sub API client to subscribe to change events.

- To learn how to write a client in Java or Python, check out [Quick Starts](#) in the *Pub/Sub API Developer Guide*.
- For code examples in other languages that are community-supported, see the [Pub/Sub API GitHub repository](#).

Get Compound Fields in Change Events

Compound fields, such as lead or contact Name, Address, and Geolocation fields, are represented as nested field structures in the event message. In record updates, the `changedFields` header field lists each updated component field using this format:

CompoundField.ComponentField. The updated component field is included in the event message in a nested field structure.

 **Note:** The name of the component field can differ from the field name in the corresponding Salesforce object. For example, in a change event, the `Street` nested component field of `BillingAddress` is `BillingStreet` in the Account object. To find out the field names and structure in a change event, get the event schema. For more information about the event schema, see [Change Event Message Structure](#). For more information about Salesforce objects, see [Standard Objects](#) in the *Object Reference for Salesforce and Lightning Platform*.

Compound Field in a New Record

 **Example:** This example shows a change event received after an account is created with a `BillingAddress` compound field. The `BillingAddress` field contains its component fields as nested fields.

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "0015f00002JUXA8AAP"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "00006bec-ce66-0611-9018-30a98446c9f2",
    "sequenceNumber": 1,
    "commitTimestamp": 1714156685000,
    "commitNumber": 1100670838951,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "Name": "Acme",
  "Type": null,
  "ParentId": null,
  "BillingAddress": {
    "Street": "415 Mission Street",
    "City": "San Francisco",
    "State": "CA",
    "PostalCode": "94105",
    "Country": "United States",
    "Latitude": null,
    "Longitude": null,
    "GeocodeAccuracy": null
  },
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
```

```

    "Industry": null,
    "AnnualRevenue": null,
    "NumberOfEmployees": null,
    "Ownership": null,
    "TickerSymbol": null,
    "Description": null,
    "Rating": null,
    "Site": null,
    "OwnerId": "0055f000005mc66AAA",
    "CreatedDate": 1714156685000,
    "CreatedById": "0055f000005mc66AAA",
    "LastModifiedDate": 1714156685000,
    "LastModifiedById": "0055f000005mc66AAA",
    "Jigsaw": null,
    "JigsawCompanyId": null,
    "CleanStatus": "Pending",
    "AccountSource": null,
    "DunsNumber": null,
    "Tradestyle": null,
    "NaicsCode": null,
    "NaicsDesc": null,
    "YearStarted": null,
    "SicDesc": null,
    "DandbCompanyId": null,
    "OperatingHoursId": null,
    "CustomerPriority__c": null,
    "SLA__c": null,
    "Active__c": null,
    "NumberOfLocations__c": null,
    "UpsellOpportunity__c": null,
    "SLASerialNumber__c": null,
    "SLAExpirationDate__c": null
  }
}

```

Compound Field in an Updated Record



Example: This example shows a change event received after updating the `Street` component field of `BillingAddress`. The `Street` field is nested under `BillingAddress`.

```

{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "0015f00002JUXA8AAP"
    ],
    "changeType": "UPDATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "00006bf0-edfd-6e54-a3f9-e8cefdb2c2b7",
    "sequenceNumber": 1,
    "commitTimestamp": 1714156703000,
    "commitNumber": 1100671026205,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],

```

```

    "diffFields": [],
    "changedFields": [
      "0x400000",
      "4-0x01"
    ]
  },
  "Name": null,
  "Type": null,
  "ParentId": null,
  "BillingAddress": {
    "Street": "415 Mission Street Suite B",
    "City": null,
    "State": null,
    "PostalCode": null,
    "Country": null,
    "Latitude": null,
    "Longitude": null,
    "GeocodeAccuracy": null
  },
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": null,
  "AnnualRevenue": null,
  "NumberOfEmployees": null,
  "Ownership": null,
  "TickerSymbol": null,
  "Description": null,
  "Rating": null,
  "Site": null,
  "OwnerId": null,
  "CreatedDate": null,
  "CreatedBy": null,
  "LastModifiedDate": 1714156703000,
  "LastModifiedBy": null,
  "Jigsaw": null,
  "JigsawCompanyId": null,
  "CleanStatus": null,
  "AccountSource": null,
  "DunsNumber": null,
  "Tradestyle": null,
  "NaicsCode": null,
  "NaicsDesc": null,
  "YearStarted": null,
  "SicDesc": null,
  "DandbCompanyId": null,
  "OperatingHoursId": null,
  "CustomerPriority__c": null,
  "SLA__c": null,
  "Active__c": null,
  "NumberofLocations__c": null,

```

```

    "UpsellOpportunity__c": null,
    "SLASerialNumber__c": null,
    "SLAExpirationDate__c": null
  }

```

After decoding the `changedFields` header field in a Pub/Sub API client, the updated `Street` field is listed as `BillingAddress.Street`.

```

2024-04-26 11:38:24,375 [grpc-default-executor-0] java.lang.Class - 
=====
2024-04-26 11:38:24,375 [grpc-default-executor-0] java.lang.Class -      Changed
Fields
2024-04-26 11:38:24,375 [grpc-default-executor-0] java.lang.Class - 
=====
2024-04-26 11:38:24,375 [grpc-default-executor-0] java.lang.Class - LastModifiedDate
2024-04-26 11:38:24,375 [grpc-default-executor-0] java.lang.Class - BillingAddress.Street
2024-04-26 11:38:24,375 [grpc-default-executor-0] java.lang.Class - 
=====

```

SEE ALSO:

[Object Reference for Salesforce and Lightning Platform: Compound Fields](#)

Enrich Change Events with Extra Fields

Change event messages include values for new and changed fields, but sometimes unchanged field values are needed for processing or replicating data. For example, use enrichment when your app needs an external ID field for matching records in an external system. Or always include a field that provides important information about the changed record. You can select any field whose type is supported.

Event enrichment is supported for subscribers that use Pub/Sub API, CometD (Streaming API), or event relays. Fields that you select for enrichment are included in change events for update and delete operations. Enriched fields aren't included in change events for create and undelete operations because these events contain all the populated fields. Also, because no change event is generated for a hard-deleted record, which is a record deleted from the Recycle Bin, enriched fields aren't available. If the enriched fields have an empty value, they aren't included in the event messages in CometD clients and event relays, and are included with null values in Pub/Sub API clients. If the enriched fields are updated to null, they're included in the event as changed fields and not as enriched fields.



Note: If your client-side parsing code expects only changed fields in the event payload, the presence of enriched fields can require a change to the code. Check your client-side code, and modify it if necessary. You can determine which fields are changed by using the `changedFields` header field. For more information, see [ChangeEventHeader Fields](#).

Event enrichment is available for channels that support multiple entities, such as the standard `/data/ChangeEvents` channel, or custom channels, such as `/data/SalesEvents__chn`. You can't add enrichment directly to single-entity channels, such as `/data/<Entity>ChangeEvent`. For example, say that you want to add the `Account Industry` field for enrichment. You can do that to the custom channel `SalesEvents__chn`, assuming `AccountChangeEvent` is a member of that channel. Then, if you subscribe to `/data/SalesEvents__chn`, the `Industry` field is included in account change events on that channel. If you subscribe to another channel that isn't enriched with this field, such as `/data/ChangeEvents`, or another custom channel, account change events don't include the `Industry` field.

We recommend that you configure event enrichment on a custom channel and not the standard `/data/ChangeEvents` channel. This way, other subscribers that receive change events on the standard channel don't receive unchanged fields that they don't expect. If you create a custom channel and configure event enrichment on it, you isolate the fields sent to only the clients that anticipate those fields. To learn how to create a custom channel, see [Compose Streams of Change Data Capture Notifications with Custom Channels](#).

As part of the fields that a change event object contains, these field types are supported for enriched fields.

- Address
- Auto Number
- Checkbox
- Currency
- Date, Date/Time, Time
- Email
- External Lookup Relationship
- Geolocation
- Hierarchical Relationship on User
- Lookup Relationship
- Master-Detail Relationship
- Name
- Number
- Percent
- Phone (and Fax)
- Picklist
- Roll-Up Summary
- Text
- TextArea
- URL

**Note:**

- Formula fields aren't supported for enriched fields because they aren't supported for change events.
- Only the TextArea field type is supported, and not TextArea (Long), TextArea (Rich), or TextArea (Encrypted).
- Compound fields, such as Name, Address, and Geolocation fields, are supported for enriched fields. You can specify an entire compound field for enrichment but not the individual constituent fields. For example, you can enrich an event with the Lead `Name` field. The enriched change event contains the constituent fields as part of the `Name` field, including `FirstName`, `MiddleName`, `LastName`, and `Suffix`. In CometD clients, all non-empty constituent fields are returned as part of the compound field in the enriched change event. In Pub/Sub API clients, all constituent fields are returned as part of the compound field including the null fields.
- For a relationship field, you can select only the field as an enriched field. You can't traverse the fields on the related object. The enriched change event contains the ID of the related record. For example, to enrich a contact change event with the ID of the related account, select the `Account` relationship field as the name of the enriched field for `ContactChangeEvent`, and not `Account.Name`. For custom relationship fields, specify the relationship field name with the `__c` suffix, such as `RelField__c`.
- You can add up to 10 enriched fields in each channel member. A compound field counts as one field. This allocation is per channel member. For example, if you have a channel with two channel members, the channel can be enriched with 20 fields total, 10 for each channel member.

Select fields to enrich your change event messages by using the `PlatformEventChannelMember` object in Tooling API or Metadata API.

IN THIS SECTION:

[Example: Add Event Enrichment Fields with Tooling API](#)

To add event enrichment fields, use the PlatformEventChannelMember Tooling API object, and specify the fields, the channel, and channel member.

[Example: Add Event Enrichment Fields with Metadata API](#)

To add event enrichment fields, use the PlatformEventChannelMember metadata type, and specify the fields, the channel, and channel member.

[Example: Delivered Enriched Event Messages](#)

Check out example event messages that contain enriched fields for update and delete operations.

[Event Enrichment Considerations](#)

Keep in mind these considerations when using enriched change events.

Example: Add Event Enrichment Fields with Tooling API

To add event enrichment fields, use the PlatformEventChannelMember Tooling API object, and specify the fields, the channel, and channel member.



Note: To carry out similar steps in Trailhead and earn a badge, check out [Create a Custom Channel and Enrich Change Events](#).

If the channel member you're enriching is part of a custom channel, create the custom channel first, as shown in this example. You can skip this step if using the ChangeEvents standard channel, or if you created the custom channel earlier.

Make a POST request to this REST endpoint:

```
/services/data/v66.0/tooling/objects/PlatformEventChannel
```

Request body for the custom channel:

```
{
  "FullName": "SalesEvents__chn",
  "Metadata": {
    "channelType": "data",
    "label": "Custom Channel for Sales App"
  }
}
```

To add enrichment fields, perform a REST request that creates a PlatformEventChannelMember component by using Tooling API. In this example, the component contains three enriched fields in the `enrichedFields` array for AccountChangeEvent on the SalesEvents custom channel. Before you create this channel member, create a custom Text(20) field for Account with the label `External Account ID`.

Make a POST request to this REST endpoint (API version 51.0 or later is supported for enrichment fields):

```
/services/data/v66.0/tooling/objects/PlatformEventChannelMember
```

Request body with enrichment fields added in a channel member:

```
{
  "FullName": "SalesEvents_AccountChangeEvent",
  "Metadata": {
    "enrichedFields": [
      {
        "name": "External_Account_ID__c"
      }
    ]
  }
}
```

```

    },
    {
      "name": "Industry"
    },
    {
      "name": "BillingAddress"
    }
  ],
  "eventChannel": "SalesEvents__chn",
  "selectedEntity": "AccountChangeEvent"
}
}

```

Query Enriched Fields

To find out which channel members and fields you configured, query the `EnrichedField` object in Tooling API. For example, this query returns the selected enriched field and the channel member ID.

```
SELECT ChannelMemberId,Field FROM EnrichedField ORDER BY ChannelMemberId
```

You can perform a query using the Query Editor in the Developer Console and by checking **Use Tooling API**. For more information, see [Developer Console Query Editor](#) in *Salesforce Help*.

Alternatively, you can run a query using REST API. Perform a GET request to the following URI. The URI includes the query with spaces replaced with `+`.

```
/services/data/v66.0/tooling/query/?q=SELECT+ChannelMemberId,Field+FROM+EnrichedField+ORDER+BY+ChannelMemberId
```

In these query results, the rows returned are for the same channel member. They contain these enriched fields: `Industry`, the `External_Account_ID__c` custom field, whose value is an ID, and `BillingAddress`.

ChannelMemberId	Field
0v8RM00000000JsYAI	Industry
0v8RM00000000JsYAI	00NRM000001gEx32AE
0v8RM00000000JsYAI	BillingAddress

Update a Channel Member with Enriched Fields

If there's an existing channel member for the same selected entity and channel, you can't create a duplicate channel member with a POST request. Instead, update the channel member with a PATCH request. Alternatively, you can delete the channel member and recreate it with the enriched fields.

To update a channel member, follow these steps.

1. If you're using a custom channel, get the channel ID by running this query:

```
SELECT Id FROM PlatformEventChannel WHERE DeveloperName=Channel_Name
```

`DeveloperName` doesn't contain the `__chn` suffix of a custom channel name. For example, for the `SalesEvents__chn` channel, the query would be:

```
SELECT Id FROM PlatformEventChannel WHERE DeveloperName='SalesEvents'
```

2. Get the channel member ID with this Tooling API query. For a custom channel, replace *Channel_ID* with the ID you got in the previous step, or for the standard ChangeEvents channel, replace *Channel_ID* with ChangeEvents. Replace **Entity**ChangeEvent with the selected entity name.

```
SELECT Id,DeveloperName,EventChannel,SelectedEntity FROM PlatformEventChannelMember
WHERE EventChannel='Channel_ID' AND SelectedEntity='EntityChangeEvent'
```

For example, for AccountChangeEvent on custom channel ID 0YLRM00000000434AA, the query looks as follows.

```
SELECT Id,DeveloperName,EventChannel,SelectedEntity FROM PlatformEventChannelMember
WHERE EventChannel='0YLRM00000000434AA' AND SelectedEntity='AccountChangeEvent'
```

Or for the standard ChangeEvents channel, the full URI would be:

```
SELECT Id,DeveloperName,EventChannel,SelectedEntity FROM PlatformEventChannelMember
WHERE EventChannel='ChangeEvents' AND SelectedEntity='AccountChangeEvent'
```

3. Make a PATCH request to this URI and append the channel member ID you got in the previous step.

```
/services/data/v66.0/tooling/subjects/PlatformEventChannelMember/Channel_Member_ID
```

In the request body, include the JSON definition of the channel member. For example, to update AccountChangeEvent on the channel member ID of 0v8RM00000000JsYAI and set the enriched fields to be the Phone field only, make a PATCH request to this URI:

```
/services/data/v66.0/tooling/subjects/PlatformEventChannelMember/0v8RM00000000JsYAI
```

With this request body:

```
{
  "FullName": "SalesEvents_chn_AccountChangeEvent",
  "Metadata": {
    "enrichedFields": [
      {
        "name": "Phone"
      }
    ],
    "eventChannel": "SalesEvents__chn",
    "selectedEntity": "AccountChangeEvent"
  }
}
```

If the channel member was previously configured with enriched fields, the update clears them and replaces them with the fields specified in the request body. This example specifies only one enriched field, the Phone field. If the channel member didn't contain enriched fields, the update adds the specified enriched fields.

For PATCH requests, include the full definition of a PlatformEventChannelMember. Partial definitions with only the enriched fields aren't supported.

SEE ALSO:

[Tooling API Developer Guide: PlatformEventChannel](#)

[Tooling API Developer Guide: PlatformEventChannelMember](#)

Example: Add Event Enrichment Fields with Metadata API

To add event enrichment fields, use the PlatformEventChannelMember metadata type, and specify the fields, the channel, and channel member.

If the channel member you are enriching is part of a custom channel, create the custom channel first, as shown in this example. You can skip this step if using the ChangeEvents standard channel, or if you created the custom channel earlier.

This sample metadata component is for a custom channel.

```
<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannel xmlns="http://soap.sforce.com/2006/04/metadata">
  <channelType>data</channelType>
  <label>Custom Channel for Sales Events</label>
</PlatformEventChannel>
```

This package.xml references the previous definition. The custom channel name is SalesEvents__chn.

```
<?xml version="1.0" encoding="UTF-8"?>
<Package xmlns="http://soap.sforce.com/2006/04/metadata">
  <types>
    <members>SalesEvents__chn</members>
    <name>PlatformEventChannel</name>
  </types>
  <version>66.0</version>
</Package>
```

To add enrichment fields, deploy the PlatformEventChannelMember component containing the enriched fields. In this example, the component contains three enriched fields for AccountChangeEvent on the SalesEvents custom channel. Before you create this channel member, create a custom Text(20) field for Account with the label External Account ID.

```
<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannelMember xmlns="http://soap.sforce.com/2006/04/metadata">
  <enrichedFields>
    <name>External_Account_ID__c</name>
  </enrichedFields>
  <enrichedFields>
    <name>Industry</name>
  </enrichedFields>
  <enrichedFields>
    <name>BillingAddress</name>
  </enrichedFields>
  <eventChannel>SalesEvents__chn</eventChannel>
  <selectedEntity>AccountChangeEvent</selectedEntity>
</PlatformEventChannelMember>
```

Use this package.xml manifest file to deploy or retrieve the channel member definition. Only API version 51.0 or later is supported for enriched fields.

```
<?xml version="1.0" encoding="UTF-8"?>
<Package xmlns="http://soap.sforce.com/2006/04/metadata">
  <types>
    <members>SalesEvents_AccountChangeEvent</members>
    <name>PlatformEventChannelMember</name>
  </types>
  <version>66.0</version>
</Package>
```

 **Note:** If the member is already part of the channel, you can update it with enriched fields by redeploying the PlatformEventChannelMember component with an updated definition.

SEE ALSO:

[Metadata API Developer Guide: PlatformEventChannel](#)

[Metadata API Developer Guide: PlatformEventChannelMember](#)

Example: Delivered Enriched Event Messages

Check out example event messages that contain enriched fields for update and delete operations.

This change event for an account update includes these enriched fields: the `External_Account_ID__c` custom field, `BillingAddress`, and `Industry`. The `changedFields` field indicates which fields changed. In this example, only the `Fax` field and the `LastModifiedDate` system field changed, but the field values for `External_Account_ID__c`, `BillingAddress`, and `Industry` are also included because they're enriched fields.

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "001ZM000001QkdOYAS"
    ],
    "changeType": "UPDATE",
    "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
    "transactionKey": "00097360-44a0-7c2e-a172-97381ae22f82",
    "sequenceNumber": 1,
    "commitTimestamp": 1714172795000,
    "commitNumber": 72657170033,
    "commitUser": "005ZM000000M6o1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": [
      "0x400080"
    ]
  },
  "Name": null,
  "Type": null,
  "ParentId": null,
  "BillingAddress": {
    "Street": "415 Mission Street",
    "City": "San Francisco",
    "State": "CA",
    "PostalCode": "94105",
    "Country": "United States",
    "Latitude": null,
    "Longitude": null,
    "GeocodeAccuracy": null
  },
  "ShippingAddress": null,
  "Phone": null,
  "Fax": "4155551212",
  "AccountNumber": null,
}
```

```

    "Website": null,
    "Sic": null,
    "Industry": "Biotechnology",
    "AnnualRevenue": null,
    "NumberOfEmployees": null,
    "Ownership": null,
    "TickerSymbol": null,
    "Description": null,
    "Rating": null,
    "Site": null,
    "OwnerId": null,
    "CreatedDate": null,
    "CreatedById": null,
    "LastModifiedDate": 1714172795000,
    "LastModifiedById": null,
    "Jigsaw": null,
    "JigsawCompanyId": null,
    "AccountSource": null,
    "SicDesc": null,
    "External_Account_ID__c": "1ABC"
  }

```

After being decoded in a Pub/Sub API client, the `changedFields` field lists the fields changed, including the `Fax` field.

```

2024-04-26 16:15:52,375 [grpc-default-executor-0] java.lang.Class - 
=====
2024-04-26 16:15:52,375 [grpc-default-executor-0] java.lang.Class -           Changed Fields

2024-04-26 16:15:52,375 [grpc-default-executor-0] java.lang.Class - 
=====
2024-04-26 16:15:52,375 [grpc-default-executor-0] java.lang.Class - Fax
2024-04-26 16:15:52,375 [grpc-default-executor-0] java.lang.Class - LastModifiedDate
2024-04-26 16:15:52,375 [grpc-default-executor-0] java.lang.Class - 
=====

```

A change event message for a delete operation includes the enriched fields, `External_Account_ID__c`, `BillingAddress`, and `Industry`.

```

{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "001ZM000001QkdOYAS"
    ],
    "changeType": "DELETE",
    "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
    "transactionKey": "00097379-d9df-704f-ed2f-cld2ca3ac266",
    "sequenceNumber": 1,
    "commitTimestamp": 1714172911000,
    "commitNumber": 72657195312,
    "commitUser": "005ZM000000M6o1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },

```

```

    "Name": null,
    "Type": null,
    "ParentId": null,
    "BillingAddress": {
      "Street": "415 Mission Street",
      "City": "San Francisco",
      "State": "CA",
      "PostalCode": "94105",
      "Country": "United States",
      "Latitude": null,
      "Longitude": null,
      "GeocodeAccuracy": null
    },
    "ShippingAddress": null,
    "Phone": null,
    "Fax": null,
    "AccountNumber": null,
    "Website": null,
    "Sic": null,
    "Industry": "Biotechnology",
    "AnnualRevenue": null,
    "NumberOfEmployees": null,
    "Ownership": null,
    "TickerSymbol": null,
    "Description": null,
    "Rating": null,
    "Site": null,
    "OwnerId": null,
    "CreatedDate": null,
    "CreatedById": null,
    "LastModifiedDate": null,
    "LastModifiedById": null,
    "Jigsaw": null,
    "JigsawCompanyId": null,
    "AccountSource": null,
    "SicDesc": null,
    "External_Account_ID__c": "1ABC"
  }

```

Event Enrichment Considerations

Keep in mind these considerations when using enriched change events.

No Apex Trigger Support

Event enrichment isn't available in Apex change event triggers.

Latest Enriched Field Value Returned When Replaying an Event Message

Change event messages are stored in the event bus for a temporary duration. Enriched fields aren't stored with the event message in the event bus. When you retrieve an event from the event bus using a replay option, enriched fields are retrieved from the database and added to the event message before delivery. As a result, if an enriched field is updated after the event was stored, the replayed event message contains the latest value and not the original value for the enriched field. The only exception is when the enriched fields are part of the changed fields for the event. In this case, their values reflect the correct changed values.

Duplicate Replay ID for Ungrouped Enriched Event Messages

Salesforce sometimes groups event messages when the same change occurs in multiple records of one object during a transaction. For more information, see the `recordIds` field in [ChangeEventHeader Fields](#). However, if change event messages are enriched, single event messages are sent because the external ID field values could be different for each record. Because these event messages are first grouped and then ungrouped, they contain duplicate `ReplayId` values and only one record ID in the `recordIds` field. You can still replay those events using the `ReplayId` option in Streaming API (CometD) or Pub/Sub API. Also, because change event messages aren't optimistically grouped before being delivered to subscribers, event allocation usage could be higher.

CampaignMember Change Event

If a `CampaignMember` is deleted from a campaign, the change event message doesn't include enriched fields because it's a hard deletion—the record no longer exists in the database. The system can't query the enriched field value for that record. However, if a `CampaignMember` is deleted as part of a cascade delete on the campaign, this deletion is a soft deletion, and the records are in the Recycle Bin. The system can query the soft-deleted record and obtain the enriched fields.

Gap and Overflow Events

Enriched fields aren't supported for gap or overflow events.

Filter Your Stream of Change Events with Channels

Receive only the change event messages that match a predefined filter on a channel in subscribers. With fewer events delivered to subscribers, event processing is optimized. Also, subscribers make more efficient use of the event delivery allocation. This feature supports Pub/Sub API, CometD (Streaming API), and event relays but not Apex triggers.



Note: If you use Government Cloud and your org was created before January 14, 2022, contact Salesforce to enable this feature. Government Cloud orgs created on or after January 14, 2022 have this feature enabled. This feature is available in all other clouds.

IN THIS SECTION:[Change Event Filters](#)

Filter a stream of change events by adding a filter expression on a channel member. A filter expression can contain Salesforce entity fields and event header fields, which are part of `ChangeEventHeader`. A change data capture channel can have one or more channel members, and each channel member can have its own filter.

[Filter Expressions in Channel Members](#)

Add a filter expression in a channel member that's associated with a custom or the standard `ChangeEvents` channel. We recommend that you use a custom channel so that the filtered stream is isolated from the standard event stream and subscribers expect the stream to be filtered. The channel holds the filtered stream of events that match the filter expression for the specified change event.

[Subscribe to the Channel and Receive the Filtered Event Stream](#)

After configuring the filter, subscribe to the channel, and receive the event messages that match the filter expression. The channel to subscribe to is `/data/ChannelName__chn`. Only Pub/Sub API and CometD clients support stream filtering. Because Apex triggers don't support channels, you can't use them to subscribe to filtered event streams.

[Get Custom Channels and Channel Members](#)

You can find which channels and channel members are set up in your Salesforce org by performing SOQL queries through Tooling API.

SEE ALSO:

[Subscribe with Pub/Sub API](#)

Change Event Filters

Filter a stream of change events by adding a filter expression on a channel member. A filter expression can contain Salesforce entity fields and event header fields, which are part of `ChangeEventHeader`. A change data capture channel can have one or more channel members, and each channel member can have its own filter.

The filter can still be evaluated even if the subscriber has no view access to the fields in the filter expression. For more information, see field-level security in [Field Considerations](#).

Auto-Enrichment of Filtered Fields

Each Salesforce entity field referenced in a filter expression is automatically enriched, which means that the change event message always contains the filtered field. Without enrichment, change events contain only new and changed fields of Salesforce records. With a filter and auto-enrichment, fields referenced in the filter expression are included in the change event message, provided that they're not empty and the subscriber has view access to them. For more information about event enrichment, see [Enrich Change Events with Extra Fields](#).

Change Event Header Fields

You can add `ChangeEventHeader` fields in a filter expression, except for fields that are arrays. For a list of header fields, see [ChangeEventHeader Fields](#).

The unsupported array fields are:

- `recordIds`
- `nulledFields`
- `diffFields`
- `changedFields`

`ChangeEventHeader` fields contain information about the change, such as the change type and the ID of the user who made the change. For example, you can use the `ChangeEventHeader.changeType` field in a filter expression to receive events only for record updates.

IN THIS SECTION:

[Filter Expression Format](#)

The filter expression format is based on SOQL and supports a subset of SOQL operators and field types. The filter expression can contain one or more field expressions, joined by a logical operator.

[Field Considerations](#)

Keep these considerations in mind for the fields in a filter expression.

[Event Delivery Usage for Filtered Streams](#)

The event delivery allocation applies to the number of events delivered after the filter is applied and not before filtering. Because a filter can reduce the number of events delivered to a subscriber, using a filter helps lower a subscriber's usage of the event delivery allocation.

Filter Expression Format

The filter expression format is based on SOQL and supports a subset of SOQL operators and field types. The filter expression can contain one or more field expressions, joined by a logical operator.

Single-field expression:

```
<FieldName> <Comparison Operator> <Value>
```

Example of multiple-field expressions joined by logical operators:

```
<FieldName> <Comparison Operator> <Value> AND (<FieldName> <Comparison Operator> <Value>  
OR <FieldName> <Comparison Operator> <Value>) ...
```

Text field values are included within single quotes. Examples of a single-field expression filtering on a Text field:

```
Industry = 'Agriculture'  
Industry LIKE 'A%'
```

Example of a single-field expression filtering on a Date field:

```
LastViewedDate > 2021-11-03T09:30:11-08:00
```

Example of a single-field expression filtering on a Time field (the UTC time zone designator Z is required):

```
OpenTime__c >= 14:30:00Z
```

Example of a single-field expression filtering on a ChangeEventHeader field:

```
ChangeEventHeader.changeType = 'UPDATE'
```

Examples of a multiple-field expression:

```
Industry = 'Agriculture' AND NumberOfEmployees > 1000
```

Example of a multiple-field expression using parentheses and the AND and OR logical operators:

```
NumberOfEmployees > 1000 AND (Industry = 'Agriculture' OR Industry = 'Banking')
```

Supported Field Types

All field types supported for enriched fields are supported in filter expressions.

- Address
- Auto Number
- Checkbox
- Currency
- Date, Date/Time, Time
- Email
- External Lookup Relationship
- Geolocation
- Hierarchical Relationship on User
- Lookup Relationship
- Master-Detail Relationship
- Name
- Number
- Percent
- Phone (and Fax)

- Picklist
- Picklist (Multi-select)
- Roll-Up Summary
- Text
- TextArea
- URL

**Note:**

- Only the TextArea field type is supported, and not TextArea (Long), TextArea (Rich), or TextArea (Encrypted).
- For Picklist (Multi-select) fields, the selected picklist values are in a delimited string. You can use any of the supported comparison operators, but not `INCLUDES` or `EXCLUDES` because they aren't supported.
- Formula fields aren't supported in filter expressions because they aren't supported for change events.

Standard and custom compound fields are supported in filter expressions when you specify their component fields in the filter expression but not the compound field itself. This table contains compound fields and examples of component fields that you can use in a filter expression.

Compound field	Examples of component fields to use in the filter expression
Name	Name component fields include the <code>FirstName</code> and <code>LastName</code> of a contact, lead, or a person account: <pre>Name.FirstName = 'John' Name.LastName = 'Smith'</pre> If you have person accounts enabled in Salesforce but reference a business account, or if you don't have person accounts enabled, specify the account <code>Name</code> field with a text value: <pre>Name = 'John Smith'</pre>
Address	<pre>BillingAddress.City = 'San Francisco'</pre> For more information, see Address Compound Fields in the <i>Object Reference for Salesforce and Lightning Platform</i>.
Geolocation	<pre>My_Location__c.Latitude > 40</pre> For more information, see Geolocation Compound Field in the <i>Object Reference for Salesforce and Lightning Platform</i>.

Supported Comparison Operators

These comparison operators are supported in filter expressions.

- `=`
- `!=`
- `>`

- <
- >=
- <=
- LIKE

Considerations for the LIKE Operator

The `LIKE` operator is supported for Text fields. The text string value must be enclosed in single quotes. The `LIKE` operator can match partial text string values when used with the `%` and `_` wildcards. The `%` wildcard matches zero or more characters. The `_` wildcard matches exactly one character.

For example, this expression matches messages with Industry values that start with 'A', such as 'Agriculture' and 'Apparel'. But it doesn't match Industry values that don't start with 'A', such as 'Education'.

```
Industry LIKE 'A%'
```

This expression matches messages with Industry values that start with 'Agricultur' and end with any single character. For example, 'Agriculture' is a match.

```
Industry LIKE 'Agricultur_'
```

Supported Logical Operators

These logical operators are supported in filter expressions.

- AND
- OR
- NOT

Considerations for the NOT Operator

Use the `NOT` operator to negate an expression. For example, this expression states that the industry isn't Banking.

```
NOT Industry = 'Banking'
```

In this next expression, the `NOT` operator negates two conditions evaluated with the `AND` operator. The filter matches events that have the industry set to a value other than Banking or `NumberOfEmployees` is less than or equal to 1,000. If an event has both the industry set to Banking and the `NumberOfEmployees` is greater than 1,000, it doesn't match the filter criteria and isn't delivered.

```
NOT (Industry = 'Banking' AND NumberOfEmployees>1000)
```

If there's more than one expression, including the expression with the `NOT` operator, parentheses around `NOT` and its expression are required. In this example, two field expressions are joined by the `AND` operator. `NOT` is used only for the first expression. It must be enclosed within parentheses because there are two expressions. The entire filter expression states that the industry is not Banking and `NumberOfEmployees` is greater than 1,000.

```
(NOT (Industry = 'Banking')) AND (NumberOfEmployees>1000)
```

This example also requires enclosing the `NOT` operator in parentheses. This filter expression matches events that have a last viewed date greater than 2021-10-21T09:30:11 in the Pacific time zone and the industry is not Banking or `NumberOfEmployees` is less than or equal to 1,000.

```
LastViewedDate>2021-10-21T09:30:11-08:00 AND (NOT (Industry = 'Banking' AND  
NumberOfEmployees>1000))
```

Filter Expression Allocations

- You can add up to 10 fields in a filter expression.
- The filter expression's maximum length is 131,072 characters.
- Each channel member can contain one filter expression.

SEE ALSO:

[Salesforce Object Query Language \(SOQL\) Reference](#)

Field Considerations

Keep these considerations in mind for the fields in a filter expression.

Text Field Considerations

- Enclose Text field values in single quotes. For example, `MyTextField__c='Hello'` is valid, but `MyTextField__c=Hello` isn't valid.
- Text values are case-insensitive except for custom fields that are marked as case-sensitive through the Unique field attribute. For example, for a field not marked as case-sensitive, `MyTextField__c='ABC'` and `MyTextField__c='abc'` are considered the same. Events with any combination of uppercase and lowercase letters of the field value match the filter and are delivered.
- If an administrator changes the case sensitivity of a custom Text field, the change isn't reflected in the filter in an active subscription until you stop and restart the subscription.
- A Text value can contain spaces and tabs between words. Because leading and trailing spaces and tabs in Text field values are stripped in the received event messages, don't include them in the filter string. If you do, the filter comparison fails.
- Text fields support all comparison operators. Comparisons of Text fields using `<`, `<=`, `>`, and `>=` are lexicographic, similar to SOQL.
- If a Text field value includes special characters such as a double quote (`"`), you can escape the characters, with some exceptions. You can't escape the backslash (`\`), underscore (`_`), and percent (`%`) characters. For more information, see [Quoted String Escape Sequences](#) in the *SOQL and SOSL Reference*.

Checkbox Field Considerations

- Checkbox fields support only the `=` and `!=` comparison operators. Using another operator causes an error.
- Comparing a Checkbox field to null is equivalent to comparing it to a value of false.

Date and Time Field Considerations

- For Date/Time fields, the supported formats include the time zone offset preceded by `+` or `-`: `YYYY-MM-DDThh:mm:ss+hh:mm` and `YYYY-MM-DDThh:mm:ss-hh:mm`, and the format that includes the UTC time zone designator `Z`: `YYYY-MM-DDThh:mm:ssZ`.
- Time field values require that the UTC time zone designator `Z` be included in this format: `hh:mm:ssZ`. Time field values are saved and retrieved in UTC.
- You can compare Date and Date/Time fields to hardcoded date values only, such as `2021-07-09` or `2021-07-09T10:30:11-08:00`. You can't compare them to date literals such as `TOMORROW`. For more information, see [Date Formats and Date Literals](#) in the *SOQL and SOSL Reference*.

Number Field Considerations

- If a filter expression contains a Number field with a value greater than 2147483647, when you attempt to save the channel member containing the filter expression you get a `FIELD_INTEGRITY_EXCEPTION` with an error message that starts with `"A number format error occurred"`. The error is due to a limitation in SOQL, which is described in this [known issue](#). To save the filter expression, append `.0` to the value so that it becomes a decimal value. For example, `"filterExpression" : "MyNumberField__c = 1657093404000.0"`.

Null Field Considerations

- When comparing a field to null, only the `=` and `!=` operators are supported.

Relationship Field Considerations

- A filter expression can contain relationship fields that are included in change events, such as `LastModifiedById`. Traversed relationship fields, such as `LastModifiedBy.Name`, aren't supported in filter expressions because those fields aren't included in change events. For example, a filter expression can contain the field expression `LastModifiedById='005RM000001dTr0YAE'`, but not `LastModifiedBy.Name='Joe Smith'`.

General Field Considerations

- Deleting fields—If a field is referenced in a filter expression, you can't delete it. If you delete it, you get an error.
 - Deleting a custom object—If a filter expression references fields of a custom object, you can't delete the custom object.
 - Renaming fields—If you rename a field that's referenced in a filter expression, the filter continues to be applied correctly. The system maps the old field name to the renamed field. It's not necessary to update the field name in the filter expression. If you rename a field label, the field name doesn't change, and filtering continues to work correctly.
-  **Note:** If a filter expression was created before Winter '23, renamed fields work only after you update the filter expression and save the channel member again.
- Namespace prefix—If a filter expression was created before an org had a namespace, and the filter expression didn't contain the namespace prefix in the field names, the filter expression is automatically updated with the namespace prefix and continues to work.
 - Changing field types—You can't change the type of a field that's referenced in a filter expression. If you change it, you get an error.
 - Field name case in the filter expression—The names of fields used in a filter expression are case-insensitive. The case of field names in the filter expression and the change event schema can differ.
 - Null enriched fields—Fields in a filter expression are also enriched fields. If an enriched field is null, it's excluded from the change event message. In the filter expression, the field is evaluated as null.
 - Field-level security—When filter expressions are evaluated, field-level security is ignored. Filter expressions are evaluated on all fields included even if the subscriber doesn't have access to the fields. Events delivered in the filtered stream include only the fields that the subscriber has access to, and they exclude the fields the subscriber doesn't have access to.

Event Delivery Usage for Filtered Streams

The event delivery allocation applies to the number of events delivered after the filter is applied and not before filtering. Because a filter can reduce the number of events delivered to a subscriber, using a filter helps lower a subscriber's usage of the event delivery allocation.

For example, a client subscribes to a channel to receive account change events, and the event bus contains 100 such events to deliver. But the channel member for `AccountChangeEvent` has a filter that matches only accounts whose `Industry` field is set to `Agriculture`. Out of the 100 account change events, 15 match this field value and are delivered. The event delivery usage is in this case 15 events and not 100. For more information about the event delivery allocation, see [Change Data Capture Allocations](#).

Filter Expressions in Channel Members

Add a filter expression in a channel member that's associated with a custom or the standard ChangeEvents channel. We recommend that you use a custom channel so that the filtered stream is isolated from the standard event stream and subscribers expect the stream to be filtered. The channel holds the filtered stream of events that match the filter expression for the specified change event.

Let's walk through the steps to create a channel, a channel member, and a filter expression. Then we can subscribe to the channel to validate receiving the filtered event stream.

IN THIS SECTION:

[Add a Filter with Tooling API](#)

Create a channel and channel member in Tooling API using API version 56.0 or later.

[Add a Filter with Metadata API](#)

We recommend using Metadata API as part of the application lifecycle management process to develop, test, deploy, and release your apps to production. If you want to create the channel and filter expression, we recommend that you use Tooling API with REST.

Add a Filter with Tooling API

Create a channel and channel member in Tooling API using API version 56.0 or later.

You can use your preferred REST API tool to perform these steps. We recommend using Postman with the Salesforce Platform APIs collection, which contains handy templates for Salesforce API calls. See [Quick Start: Connect Postman to Salesforce](#) in Trailhead.

The steps are based on a custom channel. We recommend using a custom channel instead of the standard channel so that the filtered stream is isolated from the standard event stream and subscribers expect the stream to be filtered. You can alternatively use the ChangeEvents standard channel. If you do so, you can skip steps 1 and 2, and in step 4, adjust the `FullName` field and use this `eventChannel` value: `"eventChannel": "ChangeEvents"`.

1. To create a channel, send a POST request to this URL.

```
/services/data/v66.0/tooling/objects/PlatformEventChannel
```

If you're using Postman, expand **Event Platform** > **Custom Channels** > **Platform Event**, and then click **Create channel**.

2. Use this example request body. The `channelType` is `data` because this is a channel for change events.

```
{
  "FullName": "FilteredChannel__chn",
  "Metadata": {
    "channelType": "data",
    "label": "My Custom Filtered Channel"
  }
}
```

You receive a response similar to this example response.

```
{
  "id" : "0YLRM000000004m4AA",
  "success" : true,
  "errors" : [ ],
  "warnings" : [ ],
```

USER PERMISSIONS

To create or update PlatformEventChannel and PlatformEventChannel/Member objects:

- Customize Application

To use REST API:

- API Enabled

```
"infos" : [ ]
}
```

3. Add a channel member that specifies the Industry field as an enriched field, a change event type of AccountChangeEvent, and a filter expression.

```
/services/data/v66.0/tooling/subjects/PlatformEventChannelMember
```

If you're using Postman, expand **Event Platform** > **Custom Channels** > **Platform Event**, and then click **Create channel member**.

4. Use this example request body.

```
{
  "FullName": "FilteredChannel_chn_AccountChangeEvent",
  "Metadata": {
    "eventChannel": "FilteredChannel__chn",
    "filterExpression": "Industry='Agriculture' AND NumberOfEmployees>1000",
    "selectedEntity": "AccountChangeEvent"
  }
}
```

You receive a response similar to this example response.

```
{
  "id" : "0v8RM0000004VAKYA2",
  "success" : true,
  "errors" : [ ],
  "warnings" : [ ],
  "infos" : [ ]
}
```

To update a filter expression, perform a PATCH request to

`/services/data/v66.0/tooling/subjects/PlatformEventChannelMember/<ChannelMemberID>`, and pass in the entire request body with the new filter expression. You can update only the filter expression and enriched fields of a channel member. All other fields aren't updateable.

If your Salesforce org has a namespace, prepend the namespace prefix to each field used in `filterExpression` and the `selectedEntity` value in the PlatformEventChannelMember request body. For example, if the namespace is `ns`, the request body in this example becomes:

```
{
  "FullName": "FilteredChannel_chn_AccountChangeEvent",
  "Metadata": {
    "eventChannel": "FilteredChannel__chn",
    "filterExpression": "ns__Industry='Agriculture' AND ns__NumberOfEmployees>1000",
    "selectedEntity": "AccountChangeEvent"
  }
}
```

You can add another filter to the same channel for another change event by adding another channel member. For example, if you want to filter lead change events, add a second channel member for the `FilteredChannel__chn` channel. In this member, specify `selectedEntity` as `LeadChangeEvent`, and specify the filter expression. For example:

```
{
  "FullName": "FilteredChannel_chn_LeadChangeEvent",
  "Metadata": {
```

```

    "eventChannel": "FilteredChannel__chn",
    "filterExpression": "AnnualRevenue>1000000",
    "selectedEntity": "LeadChangeEvent"
  }
}

```

SEE ALSO:

[Tooling API Developer Guide: PlatformEventChannel](#)

[Tooling API Developer Guide: PlatformEventChannelMember](#)

Add a Filter with Metadata API

We recommend using Metadata API as part of the application lifecycle management process to develop, test, deploy, and release your apps to production. If you want to create the channel and filter expression, we recommend that you use Tooling API with REST.

Create a channel and channel member in Metadata API using API version 56.0 or later.

To create a channel and channel member with Metadata API, you can use tools such as Visual Studio Code with the Salesforce Extension pack or Salesforce CLI. For more information, see [Metadata API Developer Tools](#) and [Quick Start: Metadata API](#) in the *Metadata API Developer Guide*.

The steps are based on a custom channel. We recommend using a custom channel instead of the standard channel so that the filtered stream is isolated from the standard event stream and subscribers expect the stream to be filtered. You can alternatively use the ChangeEvents standard channel. If you do so, you can skip the custom channel definition, and for the PlatformEventChannelMember definition, adjust the file name and use this eventChannel value: <eventChannel>ChangeEvents</eventChannel>.

This sample custom channel definition is for the FilteredChannel__chn channel. The file name is FilteredChannel__chn.platformEventChannel.

```

<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannel xmlns="http://soap.sforce.com/2006/04/metadata">
  <channelType>data</channelType>
  <label>My Custom Filtered Channel</label>
</PlatformEventChannel>

```

Next, add a channel member. This channel member specifies the enriched fields of Industry and NumberOfEmployees, the filter expression, and the selected entity of AccountChangeEvent. The file name is FilteredChannel__chn_AccountChangeEvent.platformEventChannelMember.

```

<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannelMember xmlns="http://soap.sforce.com/2006/04/metadata">
  <eventChannel>FilteredChannel__chn</eventChannel>
  <filterExpression><![CDATA[Industry='Agriculture' AND
NumberOfEmployees>1000]]></filterExpression>
  <selectedEntity>AccountChangeEvent</selectedEntity>
</PlatformEventChannelMember>

```



Note: If the filter expression contains the < and & special characters, they aren't allowed in XML data in their literal form. Escape those characters as < and &, or enclose the entire filter expression value within the <![CDATA[. . .]]> section. Although no special characters are present in the previous example, <![CDATA[. . .]]> is included for convenience. For more information, see [CDATA sections](#) in the Extensible Markup Language (XML) specification.

USER PERMISSIONS

To deploy and retrieve metadata types:

- Customize Application

To update metadata types:

- Modify Metadata Through Metadata API Functions

To use Metadata API:

- API Enabled

If your Salesforce org has a namespace, prepend the namespace prefix to each field used in `filterExpression` and the `selectedEntity` value in the `PlatformEventChannelMember` request body. For example, if the namespace is `ns`, the request body in this example becomes:

```
<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannelMember xmlns="http://soap.sforce.com/2006/04/metadata">
  <eventChannel>FilteredChannel__chn</eventChannel>
  <filterExpression><![CDATA[ns__Industry='Agriculture' AND
ns__NumberOfEmployees>1000]]></filterExpression>
  <selectedEntity>AccountChangeEvent</selectedEntity>
</PlatformEventChannelMember>
```

This `package.xml` file references the channel and channel member.

```
<?xml version="1.0" encoding="UTF-8"?>
<Package xmlns="http://soap.sforce.com/2006/04/metadata">
  <types>
    <members>FilteredChannel__chn</members>
    <name>PlatformEventChannel</name>
  </types>
  <types>
    <members>FilteredChannel__chn_AccountChangeEvent</members>
    <name>PlatformEventChannelMember</name>
  </types>
  <version>66.0</version>
</Package>
```

To update a filter expression, redeploy the package with an updated value for the `filterExpression` field in the `PlatformEventChannelMember` component. You can update only the filter expression and enriched fields of a channel member. All other fields aren't updateable.

You can add another filter on another change event by adding a channel member to the same channel. For example, to filter lead change events, add a second channel member for the `FilteredChannel__chn` channel. In this member, specify `selectedEntity` as `LeadChangeEvent`, and specify the filter expression and enriched fields. This `PlatformEventChannelMember` definition is an example component with a file name of `FilteredChannel__chn_LeadChangeEvent.platformEventChannelMember`.

```
<?xml version="1.0" encoding="UTF-8"?>
<PlatformEventChannelMember xmlns="http://soap.sforce.com/2006/04/metadata">
  <eventChannel>FilteredChannel__chn</eventChannel>
  <filterExpression><![CDATA[AnnualRevenue>1000000]]></filterExpression>
  <selectedEntity>LeadChangeEvent</selectedEntity>
</PlatformEventChannelMember>
```

This `package.xml` file references both channel members.

```
<?xml version="1.0" encoding="UTF-8"?>
<Package xmlns="http://soap.sforce.com/2006/04/metadata">
  <types>
    <members>FilteredChannel__chn</members>
    <name>PlatformEventChannel</name>
  </types>
  <types>
    <members>FilteredChannel__chn_AccountChangeEvent</members>
    <members>FilteredChannel__chn_LeadChangeEvent</members>
    <name>PlatformEventChannelMember</name>
  </types>
```

```
<version>66.0</version>
</Package>
```

SEE ALSO:

[Metadata API Developer Guide: PlatformEventChannel](#)

[Metadata API Developer Guide: PlatformEventChannelMember](#)

Subscribe to the Channel and Receive the Filtered Event Stream

After configuring the filter, subscribe to the channel, and receive the event messages that match the filter expression. The channel to subscribe to is `/data/ChannelName__chn`. Only Pub/Sub API and CometD clients support stream filtering. Because Apex triggers don't support channels, you can't use them to subscribe to filtered event streams.



Note: Before subscribing to the channel, follow the steps in the previous sections to create the `FilteredChannel__chn` channel, and configure a filter expression for `AccountChangeEvent` with Tooling API or Metadata API.

In this example, we use a Pub/Sub API Java client.

1. To set up the Pub/Sub API Java client, follow the steps in [Java Quick Start for Publishing and Subscribing to Events](#) in the [Pub/Sub API Developer Guide](#).
2. In [Step 3: Configure Client Parameters](#), supply the configuration parameters in `arguments.yaml`. Also, make sure you supply the values.
 - a. `TOPIC: /data/FilteredChannel__chn`
 - b. `PROCESS_CHANGE_EVENT_HEADER_FIELDS: true`
 This value ensures that the bitmap fields, such as `changedFields`, in `ChangeEventHeader` are expanded. For more information, see [Event Deserialization Considerations](#) in the *Pub/Sub API Developer Guide*.
3. In a Terminal window, navigate to the top-level `java` folder.
4. To run the Subscribe RPC example, enter: `./run.sh genericpubsub.Subscribe`.
5. Let's create some accounts and update one account. Each account creation and update generates a change event that Salesforce publishes.
 - a. Click App Launcher, and enter Accounts in the search box.
 - b. Right-click Accounts, and select to open the link in a new tab.
 - c. In the new tab, search for Accounts in the App Launcher.
 - d. Create an account with these values.
 - Account Name: Acme North
 - Industry: Agriculture
 - NumberOfEmployees: 1500
 - e. Create a second account with these values.
 - Account Name: Acme South
 - Industry: Agriculture
 - NumberOfEmployees: 20
 - f. Create a third account with these values.

- Account Name: Acme West
 - NumberOfEmployees: 1100
- g. Update the 'Acme North' account with this value.
- Type: Prospect

As a refresher, here's the filter expression that was set in the previous section.

```
Industry='Agriculture' AND NumberOfEmployees>1000
```

From the change event messages that Salesforce published in response to the accounts created, only the first event of account Acme North matches the filter criteria set up in the previous example. The first event message is delivered to the client. Also, the last change event corresponding to the Acme North account update is delivered, as the account continues to match the criteria. Notice that even though only the Type field was updated, the change event includes the NumberOfEmployees and Industry fields because they're auto-enriched.

The second and third events don't match the filter criteria and aren't delivered to the client. In the second event, the NumberOfEmployees field is less than 1,000, and the third event has a blank (null) Industry.

This example shows the received change event messages after subscribing to the `/data/FilteredChannel__chn` channel.

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "001ZM000002JNaXYAW"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
    "transactionKey": "00007816-6798-4836-5275-80afa466c4d4",
    "sequenceNumber": 1,
    "commitTimestamp": 1722531023000,
    "commitNumber": 77390604774,
    "commitUser": "005ZM000000M6o1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "Name": "Acme North",
  "Type": null,
  "ParentId": null,
  "BillingAddress": null,
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": "Agriculture",
  "AnnualRevenue": null,
  "NumberOfEmployees": 1500,
  "Ownership": null,
  "TickerSymbol": null,
  "Description": null,
  "Rating": null,
```

```

    "Site": null,
    "OwnerId": "005ZM000000M6o1YAC",
    "CreatedDate": 1722531023000,
    "CreatedById": "005ZM000000M6o1YAC",
    "LastModifiedDate": 1722531023000,
    "LastModifiedById": "005ZM000000M6o1YAC",
    "Jigsaw": null,
    "JigsawCompanyId": null,
    "AccountSource": null,
    "SicDesc": null,
    "External_Account_ID__c": null
  }
  with schema name: AccountChangeEvent

{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "001ZM000002JNaXYAW"
    ],
    "changeType": "UPDATE",
    "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
    "transactionKey": "000077fe-1655-b8fc-44ab-27df66fccbf6",
    "sequenceNumber": 1,
    "commitTimestamp": 1722531115000,
    "commitNumber": 77390686560,
    "commitUser": "005ZM000000M6o1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": [
      "0x400004"
    ]
  },
  "Name": null,
  "Type": "Prospect",
  "ParentId": null,
  "BillingAddress": null,
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": "Agriculture",
  "AnnualRevenue": null,
  "NumberOfEmployees": 1500,
  "Ownership": null,
  "TickerSymbol": null,
  "Description": null,
  "Rating": null,
  "Site": null,
  "OwnerId": null,
  "CreatedDate": null,
  "CreatedById": null,

```

```

    "LastModifiedDate": 1722531115000,
    "LastModifiedById": null,
    "Jigsaw": null,
    "JigsawCompanyId": null,
    "AccountSource": null,
    "SicDesc": null,
    "External_Account_ID__c": null
  }
  with schema name: AccountChangeEvent
2024-08-01 09:52:08,030 [grpc-default-executor-1] java.lang.Class -
=====
2024-08-01 09:52:08,030 [grpc-default-executor-1] java.lang.Class -      ChangedFields

2024-08-01 09:52:08,030 [grpc-default-executor-1] java.lang.Class -
=====
2024-08-01 09:52:08,030 [grpc-default-executor-1] java.lang.Class - Type
2024-08-01 09:52:08,030 [grpc-default-executor-1] java.lang.Class - LastModifiedDate
2024-08-01 09:52:08,030 [grpc-default-executor-1] java.lang.Class -
=====

```

Get Custom Channels and Channel Members

You can find which channels and channel members are set up in your Salesforce org by performing SOQL queries through Tooling API.

To perform SOQL queries, make a REST query call or use the Query Editor in the Developer Console with the **Tooling API** option selected. To make REST calls using Postman, set up Postman with the Salesforce Platform APIs collection. See [Quick Start: Connect Postman to Salesforce](#) in Trailhead.

Perform a GET request to this endpoint with the SOQL query appended.

USER PERMISSIONS

To query PlatformEventChannel and PlatformEventChannel/Member Tooling objects:

- View Setup and Configuration

To use REST with Tooling API:

- API Enabled

```
/services/data/v66.0/tooling/query?q=<query>
```

This query returns all the custom channels.

```
SELECT Id, DeveloperName, ChannelType, MasterLabel FROM PlatformEventChannel
```

If you're using Postman, expand **Event Platform** > **Custom Channels**, and then click **List event channels**.

Sample result:

Id

0YLRM000000004m4AA

DeveloperName

FilteredChannel

ChannelType

data

MasterLabel

My Custom Filtered Channel

And this query returns all the channel members.

```
SELECT Id, DeveloperName, EventChannel, FilterExpression, SelectedEntity FROM
PlatformEventChannelMember
```

If you're using Postman, expand **Event Platform > Custom Channels**, and then click **List channel members**.

Sample result (the `SelectedEntity` field references the ID of the custom platform event):

Id

0v8RM0000004VAKYA2

DeveloperName

FilteredChannel_chn_AccountChangeEvent

EventChannel

0YLRM000000004m4AA

FilterExpression

```
Industry='Agriculture' AND NumberOfEmployees>1000
```

SelectedEntity

AccountChangeEvent

Subscribe with Apex Triggers

With Apex triggers, you can capture and process change events on the Lightning Platform. Change event triggers run asynchronously after the database transaction is completed. Perform resource-intensive business logic asynchronously in the change event trigger, and implement transaction-based logic in the Apex object trigger. By decoupling the processing of changes, change event triggers can help reduce transaction processing time.

IN THIS SECTION:

[Change Event Triggers](#)

Because change events are based on platform events, they share characteristics for subscription with Apex platform event triggers. Also, change event messages in triggers contain header and record fields, and some additional fields not present in JSON event messages.

[Apex Trigger Quick Start](#)

Create an Apex trigger that captures change event messages.

[Apex Trigger Example](#)

This sample trigger demonstrates a common use for change event triggers and provides a more complex trigger example than the quick start.

[Test Change Event Triggers](#)

Before you can package or deploy Apex change event triggers to production, you must provide Apex tests and sufficient code coverage.

[Change Event Trigger Considerations](#)

Keep these considerations in mind when working with change events in Apex triggers.

[Obtain Apex Trigger Subscribers](#)

To get information about the triggers that are subscribed to change events, query the `EventBusSubscriber` standard object using SOQL. `EventBusSubscriber` contains information about Apex triggers but not CometD or Pub/Sub API subscribers.

Change Event Triggers

Because change events are based on platform events, they share characteristics for subscription with Apex platform event triggers. Also, change event messages in triggers contain header and record fields, and some additional fields not present in JSON event messages.

A change event trigger:

- Is an after-insert trigger (defined with the `after insert` keyword)? It fires after the change event message is published.

For example, this empty trigger definition is for the Account standard object.

```
trigger AccountChangeEventTrigger on AccountChangeEvent (after insert) {
}
```

And this empty trigger definition is for the Employee__c custom object.

```
trigger EmployeeChangeEventTrigger on Employee__ChangeEvent (after insert) {
}
```

- Runs under the Automated Process entity. As a result:
 - Debug logs corresponding to the trigger execution are created by Automated Process. The logs aren't available in the Developer Console's log tab, except for Apex tests. Set up a trace flag entity for the Automated Process entity on the Debug Logs page in Setup.
 - The system fields of records that the trigger processes, such as CreatedById and LastModifiedById, also reference the Automated Process entity.
- Executes asynchronously outside the Apex transaction that published the change event.
- Is subject to Apex synchronous governor limits.
- Has a maximum batch size of 2,000 event messages.

To override a change event trigger's default running user and batch size, use PlatformEventSubscriberConfig in Tooling API or Metadata API. PlatformEventSubscriberConfig also configures platform event triggers. For more information, see [Configure the User and Batch Size for Your Platform Event Trigger](#) in the *Platform Events Developer Guide*.

Apex Change Event Message Fields

Each change event captured in the trigger contains header and record fields. Fields in a change event message are statically defined, just like in any other Apex type. As a result, all record fields are present in the change event message, whether changed or not. Unchanged fields are null in the Apex change event message. For details, see [Change Event Body Fields](#).

To obtain a header field, access the `ChangeEventHeader` field on the event object. For example, this code snippet gets the change event header and writes two header field values to the debug log.

```
EventBus.ChangeEventHeader header = event.ChangeEventHeader;
String changeEntity = header.entityName;
String changeOperation = header.changeType;
```

All header fields are provided in the `EventBus.ChangeEventHeader` Apex class. For more information, see [ChangeEventHeader Class](#) in the *Apex Developer Guide*. The Apex class also contains these two headers, which aren't present in JSON event messages.

nulledfields

Contains the names of fields whose values were changed to null in an update operation. Use this field in Apex change event messages to determine if a field was changed to null in an update and isn't an unchanged field.



Note: Starting in API version 47.0, the `changedfields` header is present in Apex and JSON event messages. It contains all fields that were changed in an update operation, whether populated or set to null.

difffields

Contains the names of fields whose values are sent as a unified diff because they contain large text values.

SEE ALSO:

[Other Types of Change Events: Gap and Overflow Events](#)

Apex Trigger Quick Start

Create an Apex trigger that captures change event messages.

IN THIS SECTION:

[Prerequisites](#)

Before subscribing to change events with an Apex trigger, set up debug logs and select the Account object for notifications.

[Add an Apex Trigger](#)

The quick start adds a simple change event trigger that shows how to access header and record fields in a change event message.

Prerequisites

Before subscribing to change events with an Apex trigger, set up debug logs and select the Account object for notifications.

Set Up Debug Logs

To obtain debug logs for change event trigger execution, set up debug logs for the Automated Process entity.

1. From Setup, enter *Debug Logs* in the Quick Find box, then select **Debug Logs**.
2. Click **New**.
3. For Traced Entity Type, select **Automated Process**.
4. Select the time period to collect logs for and the debug level.
5. Click **Save**.

Enable Change Notifications for the Object

On the Change Data Capture page in Setup, enable change notifications for the Account object. See [Select Objects for Change Notifications in the User Interface](#).

Add an Apex Trigger

The quick start adds a simple change event trigger that shows how to access header and record fields in a change event message.

Before you add and test the trigger, set up debug logging for the Automated Process entity and enable Account for Change Data Capture. See [Prerequisites](#).

1. In the Developer Console, select **File > New > Apex Trigger**.
2. In the Name field, enter a name for the trigger: *MyAccountChangeTrigger*.
3. From the dropdown, select the change event object for Account: **AccountChangeEvent**.

The trigger is created with the after `insert` keyword.

4. Replace the default content of the trigger with the following code.

```
trigger MyAccountChangeTrigger on AccountChangeEvent (after insert) {
    List<Task> tasks = new List<Task>();

    // Iterate through each event message.
    for (AccountChangeEvent event : Trigger.New) {
        // Get some event header fields
        EventBus.ChangeEventHeader header = event.ChangeEventHeader;
        System.debug('Received change event for ' + header.entityName +
            ' for the ' + header.changeType + ' operation.');
```

```
        // Get account record fields
        System.debug('Account Name: ' + event.Name);
        System.debug('Account Phone: ' + event.Phone);

        // Create a followup task
        if (header.changetype == 'CREATE') {
            Task tk = new Task();
            tk.Subject = 'Follow up on new account for record or group of records: ' +
                header.recordIds;
            // Explicitly set the task owner ID to a valid user ID so that
            // it is not Automated Process.
            // For simplicity, we set it to the CommitUser header field,
            // which is available for all operations.
            tk.OwnerId = header.CommitUser;
            tasks.add(tk);
        }
        else if ((header.changetype == 'UPDATE')) {
            // For update operations, iterate over the list of changed fields
            System.debug('Iterate over the list of changed fields.');
```

```
            for (String field : header.changedFields) {
                if (null == event.get(field)) {
                    System.debug('Deleted field value (set to null): ' + field);
                } else {
                    System.debug('Changed field value: ' + field +
                        '. New Value: ' + event.get(field));
                }
            }
        }
    }

    // Insert all tasks in bulk.
    if (tasks.size() > 0) {
        insert tasks;
    }
}
```

This simple trigger writes header and field values to the debug log for each received change event message. The trigger uses the `changedFields` header field to determine which fields changed in an update operation. The trigger also creates a follow-up task for new accounts.



Note: The `changedFields` property is available in Apex saved using API version 47.0 or later.

5. To test the trigger, create an account with a name and phone.
6. Edit the account, change the name, delete the phone value, and save the record.
7. In Setup, enter *Debug Logs* in the Quick Find box, then select **Debug Logs**.
8. To view the debug logs corresponding to the record creation, click the second log in the list (logs are ordered by most recent first). The output of the `System.debug` statements looks similar to the following.

```
...|DEBUG|Received change event for Account for the CREATE operation.
...|DEBUG|Account Name: Quick Start Account
...|DEBUG|Account Phone: 4155551212
```

9. To view the debug logs corresponding to the record update, click the first log in the list. The output of the `System.debug` statements looks similar to the following. The account name's new value is listed. The phone field's value was deleted, and its value was set to null. Also, because the system updates the `LastModifiedDate` field when the record is updated, this field is listed in the `changedFields` field and is part of the debug output.

```
...|DEBUG|Received change event for Account for the UPDATE operation.
...|DEBUG|Account Name: Quick Start Account Updated
...|DEBUG|Account Phone: null
...|DEBUG|Iterate over the list of changed fields.
...|DEBUG|Changed field value: Name. New Value: Quick Start Account Updated
...|DEBUG|Deleted field value (set to null): Phone
...|DEBUG|Changed field value: LastModifiedDate. New Value: 2019-08-14 23:20:15
```

Apex Trigger Example

This sample trigger demonstrates a common use for change event triggers and provides a more complex trigger example than the quick start.

Using an Apex Change Event Trigger to Predict Account Status

Because Apex change event triggers run asynchronously, they typically contain time-intensive processes that run after the database transaction is completed. This trigger example covers a more common real-world scenario than the trigger given in the quick start. It captures case changes and predicts the account trust status using an Apex class. The prediction is based on counting the account cases and checking case fields. In other scenarios, you might have complex prediction algorithms that are more resource intensive.

The trigger calls a method in an Apex class to perform the trust prediction on the accounts related to all cases in this trigger. An account status of Red means that the account trust level is low because too many cases are associated with this account among other criteria. An account status of Green means that the trust level is good. The associated class is listed after this trigger. This example assumes that you have the following prerequisites.

- A platform event named `Red_Account__e` with two fields: `Account_Id__c` of type Text and `Rating__c` of type Text
- The `SLAViolation__c` custom field on the Case object of type Picklist with values Yes and No

```
trigger CaseChangeEventTrigger on CaseChangeEvent (after insert) {

    List<CaseChangeEvent> changes = Trigger.new;

    Set<String> caseIds = new Set<String>();
```

```

for (CaseChangeEvent change : changes) {
    // Get all Record Ids for this change and add to the set
    List<String> recordIds = change.ChangeEventHeader.getRecordIds();
    caseIds.addAll(recordIds);
}

// Perform heavy (slow) computation determining Red Account
// status based on these Case changes
RedAccountPredictor predictor = new RedAccountPredictor();
Map<String, boolean> accountsToRedAccountStatus =
    predictor.predictForCases(new List<String>(caseIds));

// Publish platform events for predicted red accounts
List<Red_Account__e> redAccountEvents = new List<Red_Account__e>();
for (String acctId : accountsToRedAccountStatus.keySet()) {
    String rating = accountsToRedAccountStatus.get(acctId) ? 'Red' : 'Green';
    if (rating=='Red') {
        redAccountEvents.add(new Red_Account__e(Account_Id__c=acctId,
            Rating__c=rating));
    }
}
System.debug('RED_ACCT: ' + redAccountEvents);
if (redAccountEvents.size() > 0) {
    EventBus.publish(redAccountEvents);
}
}

```

The RedAccountPredictor class performs the prediction of the account trust level. The first method that the trigger calls is predictForCases, which calls other methods in this class. The method returns a map of account ID to a Boolean value for account status.

```

public class RedAccountPredictor {

    private static final Integer MAX_CASES_EXPECTED = 2;

    public RedAccountPredictor() { }

    // First method to be called for performing account status prediction.
    // Get the account IDs related to the passed-in case IDs
    // and call a predictor method.
    // Return a map of account ID to account status Boolean.
    public Map<String, boolean> predictForCases(List<String> caseId) {
        List<Case> casesMatchingIds =
            [SELECT Id, Account.Id FROM Case WHERE Id IN :caseId];
        if (null != casesMatchingIds && casesMatchingIds.size() > 0) {
            List<String> accountIds = new List<String>();
            for (Case c : casesMatchingIds) {
                accountIds.add(c.Account.Id);
            }
            return predictForAccounts(accountIds);
        } else {
            return new Map<String, boolean>();
        }
    }
}

```

```

// Perform slow, resource intensive calculation to determine.
// If Account is in RED status (think Einstein predictions, etc.)
public Map<String, boolean> predictForAccounts(List<String> acctIds) {
    List<Case> casesForAccounts =
        [SELECT Id, Account.Id, Status, CaseNumber, Priority,
         IsEscalated, SLAViolation__c
         FROM Case
         WHERE AccountId IN :acctIds AND Status !='Closed'];
    Map<String, List<Case>> accountsToCases = new Map<String, List<Case>>();
    for (Case c : casesForAccounts) {
        if (null == c.Account.Id) continue;
        if (!accountsToCases.containsKey(c.Account.Id)) {
            accountsToCases.put(c.Account.Id, new List<Case>());
        }
        accountsToCases.get(c.Account.Id).add(c);
    }
    Map<String, boolean> results = new Map<String, boolean>();
    for (String acctId : accountsToCases.keySet()) {
        results.put(acctId, predict(accountsToCases.get(acctId)));
    }
    return results;
}

// Perform the account status prediction.
// Return true if account is red; otherwise, return false.
private boolean predict(List<Case> casesForAccount) {
    boolean isEscalated = false;
    boolean hasSlaViolation = false;
    boolean hasHighPriority = false;
    boolean allStatusesResolved = true;

    for (Case openCase : casesForAccount) {
        isEscalated |= openCase.IsEscalated;
        hasSlaViolation |= (openCase.SLAViolation__c == 'Yes');
        hasHighPriority |= openCase.Priority == 'High';
        allStatusesResolved &= (openCase.Status == 'Closed'
                                || openCase.Status == 'Part Received');
    }
    if (allStatusesResolved) {
        return false;
    }
    if (casesForAccount.size() > MAX_CASES_EXPECTED) {
        return true;
    } else if (isEscalated || hasSlaViolation) {
        return true;
    } else if (hasHighPriority) {
        return true;
    }
    return false;
}
}

```

Test Change Event Triggers

Before you can package or deploy Apex change event triggers to production, you must provide Apex tests and sufficient code coverage.

Enable All Change Data Capture Entities for Notifications

To enable the generation of change event notifications for all supported Change Data Capture entities for an Apex test, call this method.

```
Test.enableChangeDataCapture();
```

Call the `Test.enableChangeDataCapture()` method at the beginning of your test method before performing DML operations and calling `Test.getEventBus().deliver()` or `Test.stopTest()`.

The `Test.enableChangeDataCapture()` method ensures that Apex tests can fire change event triggers regardless of the entities selected in Setup. This method doesn't affect the Change Data Capture entity selections for the org.

Deliver Test Change Events

To test your change event trigger, perform DML operations and then call the `Test.getEventBus().deliver()` method. The method delivers the event messages from the test event bus to the corresponding change event trigger and causes the trigger to fire. Finally, validate that the trigger executed as expected. For example, if the trigger creates or updates records, you can query those records with SOQL.

This test method outlines the order of statements that must be executed in a test, starting with enabling Change Data Capture entities.

```
@isTest static void testChangeEventTrigger() {
    // Enable all Change Data Capture entities for notifications.
    Test.enableChangeDataCapture();

    // Insert one or more test records
    // ...

    // Deliver test change events
    Test.getEventBus().deliver();

    // Verify the change event trigger's execution
    // ...
}
```

Alternatively, use the `Test.startTest()`, `Test.stopTest()` method block to fire a change event trigger. After `Test.stopTest()` executes, all test change event messages generated from DML operations are delivered to the associated trigger. The DML statements can be within the block or outside the block as long as they precede `Test.stopTest()`.

```
@isTest static void testChangeEventTriggerWithStopTest() {
    // Enable all Change Data Capture entities for notifications.
    Test.enableChangeDataCapture();

    Test.startTest();
    // Insert one or more test records
    // ...
    Test.stopTest();
    // The stopTest() call delivers the test change events and fires the trigger

    // Verify the change event trigger's execution
}
```

```
// ...
}
```

 **Note:** In test context, only up to 500 change event messages can be delivered as a result of record changes. If you exceed this limit, the Apex test stops execution with a fatal error.

Apex Test Example Based on Quick Start Trigger

The `testNewAccount` method in this test class shows you how to write a test for the `MyAccountChangeTrigger` trigger provided in the [Add an Apex Trigger](#) on page 58 quick start. The test method first enables all entities for change notifications. It creates a test account and then calls the `Test.getEventBus().deliver();` method. Next, the test verifies that the trigger's execution by querying Task records and validating that one task was created. The query returns only tasks that the trigger created in test context. For that reason, the test expects only one task. Next, the test updates the account and verifies that no new task is created.

```
@isTest
public class TestMyAccountChangeTrigger {
    @isTest static void testNewAndUpdatedAccount() {
        // Enable all Change Data Capture entities for notifications.
        Test.enableChangeDataCapture();

        // Insert an account to generate a change event.
        Account newAcct = new Account(Name='TestAccount', Phone='4155551212');
        insert newAcct;

        // Call deliver to fire the trigger and deliver the test change event.
        Test.getEventBus().deliver();

        // VERIFICATIONS
        // Check that the change event trigger created a task.
        Task[] taskList = [SELECT Id,Subject FROM Task];
        System.assertEquals(1, taskList.size(),
            'The change event trigger did not create the expected task.');
```

```
        // Update account record
        Account queriedAcct = [SELECT Id,Phone,Website FROM Account WHERE Id=:newAcct.Id];

        // Debug
        System.debug('Retrieved account record: ' + queriedAcct);
        // Update one field and empty another
        queriedAcct.Website = 'www.salesforce.com';
        queriedAcct.Phone = null;
        update queriedAcct;

        // Call deliver to fire the trigger for the update operation
        Test.getEventBus().deliver();

        // VERIFICATIONS
        // Check that the change event trigger did NOT create a task.
        // We should still have only 1 task.
        Task[] taskList2 = [SELECT Id,Subject FROM Task];
        System.assertEquals(1, taskList2.size(),
            'The change event trigger created a task unepextedly.');
```

```
    }
}
```

This test class is an alternative example that uses the `Test.startTest()`, `Test.stopTest()` method block to deliver test change events and fire the trigger. For more information about these methods, see [Using Limits, startTest, and stopTest](#) in the [Apex Developer Guide](#).

```
@isTest
public class TestMyAccountChangeTriggerWithStopTest {
    @isTest static void testNewAccount() {
        // Enable all Change Data Capture entities for notifications.
        Test.enableChangeDataCapture();

        Test.startTest();
        // Insert an account to generate a change event.
        insert new Account(Name='TestAccount', Phone='4155551212');
        Test.stopTest();
        // The stopTest() call fires the trigger with the test account change event.

        // VERIFICATIONS
        // Check that the change event trigger created a task.
        Task[] taskList = [SELECT Id,Subject FROM Task];
        System.assertEquals(1, taskList.size(),
            'The change event trigger did not create the expected task.');
```



Note: If multiple DML operations are performed on a single record within the `Test.startTest()`, `Test.stopTest()` block, only one change event is generated. The change event contains the latest data and the initial change type. For more information, see [Change Event Generation in a Transaction with Multiple Changes for the Same Record](#).

Properties of Change Events in Test Context

Test change events messages are published to the test event bus, which is separate from the Salesforce event bus. They aren't persisted in Salesforce and aren't delivered to event channels outside the test class. Properties of test change event messages, like the replay ID, are reset in test context and reflect only the values of test event messages. For more information, see [Event and Event Bus Properties in Test Context](#) in the [Platform Events Developer Guide](#).

Change Event Trigger Considerations

Keep these considerations in mind when working with change events in Apex triggers.

Triggers for Non-Enabled Objects

You can save an Apex trigger for a change event object even if the object isn't selected for notifications on the Change Data Capture page. When the object isn't selected, the trigger doesn't fire. To ensure that the trigger fires, select the object for notifications. Any type of change event fires a change event trigger, including gap events and overflow events.

No Formula Field Support

Formula fields aren't supported in Change Data Capture. They're null in a change event trigger, regardless of whether they were changed. For information on which field values aren't included, so are null in a trigger, see [Change Event Fields](#).

Null Name Field for Person Accounts

For a person account, the Name compound field in the `AccountChangeEvent` received in the trigger is null. The `FirstName` and `LastName` fields, which are included in the Name field, contain the person account first name and last name values. In contrast, the `ContactChangeEvent` Name field contains the concatenated values of the salutation, first name, and last name.

Infinite Trigger Loop

If your trigger updates records of the same object as the one that corresponds to the received change event, then the trigger can fire recursively and exceed limits. To avoid infinite trigger recursion, ensure that you limit your updates so they don't occur every time the trigger refires.

Obtain Apex Trigger Subscribers

To get information about the triggers that are subscribed to change events, query the `EventBusSubscriber` standard object using SOQL. `EventBusSubscriber` contains information about Apex triggers but not CometD or Pub/Sub API subscribers.



Example: This example SOQL query selects several fields from `EventBusSubscriber`. For more information about `EventBusSubscriber` fields, see [EventBusSubscriber](#) in the *Object Reference for Salesforce and Lightning Platform*.

```
SELECT ExternalId, Name, Topic, Position, Status, Tip, Type FROM EventBusSubscriber
```

The returned result shows that there are two Apex triggers subscribed to change events. One trigger is subscribed to `AccountChangeEvent` and one to `ContactChangeEvent`.

ExternalId	Name	Topic	Position	Status	Tip	Type
01q2J000000g0kb	MyAccountChangeTrigger	AccountChangeEvent	226751	Running	-1	ApexTrigger
01q2J000000g0kg	MyContactChangeTrigger	ContactChangeEvent	226752	Running	-1	ApexTrigger

You can filter the query results by using a `WHERE` clause. For example, this query filters by the topic `ContactChangeEvent`.

```
SELECT ExternalId, Name, Topic, Position, Status, Tip, Type FROM EventBusSubscriber
WHERE Topic='ContactChangeEvent'
```

The query returns only the trigger subscribers to `ContactChangeEvent`, in this case, one trigger.

ExternalId	Name	Topic	Position	Status	Tip	Type
01q2J000000g0kg	MyContactChangeTrigger	ContactChangeEvent	226752	Running	-1	ApexTrigger

Monitor Change Event Publishing and Delivery Usage

To get usage data for event publishing and delivery to CometD and Pub/Sub API clients, `empApi` Lightning components, and event relays, query the `PlatformEventUsageMetric` object. In API 58.0 and later, enable and use Enhanced Usage Metrics to get granular usage data for various time segments. If Enhanced Usage Metrics isn't enabled, usage data is available for the last 24 hours, ending at the last hour, and for historical daily usage. `PlatformEventUsageMetric` is available in API version 50.0 and later.



Important: We recommend that you use Enhanced Usage Metrics. With Enhanced Usage Metrics, you can query usage data at a granular level. You can break down usage metrics by event name, client ID, event type, and usage type. And you can get usage data by various time segments, including daily, hourly, and 15-minute periods. See [Enhanced Usage Metrics](#) in the *Platform Events Developer Guide*.

Use `PlatformEventUsageMetric` to get visibility into your event usage and usage trends. The usage data gives you an idea of how close you are to your allocations and when you need more allocations. The usage metrics stored in `PlatformEventUsageMetric` are separate

from the REST API limits values. Use the REST API limits to track your monthly delivery usage against your allocations. The monthly event delivery usage that the limits API returns is common for platform events and change data capture events in CometD and Pub/Sub API clients, `empApi`, Lightning components, and event relays. `PlatformEventUsageMetric` breaks down usage of platform events and change data capture events so you can track their usage separately.

Because dates are stored in Coordinated Universal Time (UTC), convert your local dates and times to UTC for the query. For the date format to use, see [Date Formats and Date Literals](#) in the *SOQL and SOSL Reference*.

 Note:

- Usage data is stored for at least 45 days. Usage data is updated hourly and is available only when usage is nonzero for a 24-hour period. Usage data isn't available for 1-hour intervals or any other arbitrary interval. The only supported intervals are the last 24 hours and daily data. Also, usage data isn't available for standard-volume platform events.
- After a Salesforce major upgrade, usage data can be inaccurate for the day and the last 24 hours within the upgrade window. New usage data overwrites the data for the hour that the 5-minute upgrade occurs in. The new usage data includes metrics that start after the upgrade for that hour. For more information about Salesforce upgrades, see [Salesforce Upgrades and Maintenance](#) in *Help* and [Salesforce Status](#).

For change events, you can query usage data for these metrics. The first value is the metric name value that you supply in the query.

- `CHANGE_EVENTS_PUBLISHED`—Number of change data capture events published
- `CHANGE_EVENTS_DELIVERED`—Number of change data capture events delivered to CometD and Pub/Sub API clients, `empApi`, Lightning components, and event relays

For platform events, you can query usage data for these metrics. The first value is the metric name value that you supply in the query.

- `PLATFORM_EVENTS_PUBLISHED`—Number of platform events published
- `PLATFORM_EVENTS_DELIVERED`—Number of platform events delivered to CometD and Pub/Sub API clients, `empApi`, Lightning components, and event relays

Obtain Usage Metrics for the Last 24 Hours

To get usage metrics for the last 24 hours, ending at the last hour, perform a query by specifying the start and end date and time in UTC, and the metric name.

For the last 24-hour period, the end date is the current date in UTC, with the time rounded down to the previous hour. The start date is 24 hours before the end date. Dates have hourly granularity.



Example: Based on the current date and time of August 4, 2020 11:23 in UTC, the last hour is 11:00. The query includes these dates.

- Start date in UTC format: 2020-08-03T11:00:00.000Z
- End date in UTC format: 2020-08-04T11:00:00.000Z

This query returns the usage for the number of platform events delivered between August 3, 2020 at 11:00 and August 4, 2020 at 11:00.

```
SELECT Name, StartDate, EndDate, Value FROM PlatformEventUsageMetric
WHERE Name='CHANGE_EVENTS_DELIVERED'
AND StartDate=2020-08-03T11:00:00.000Z AND EndDate=2020-08-04T11:00:00.000Z
```

The query returns this result for the last 24-hour usage.

Name	StartDate	EndDate	Value
CHANGE_EVENTS_DELIVERED	2020-08-03T11:00:00.000+0000	2020-08-04T11:00:00.000+0000	575

The time span between `StartDate` and `EndDate` is 24 hours for the stored 24-hour usage. Therefore, you can specify either `StartDate` or `EndDate` in the query and you get the same result.

Obtain Historical Daily Usage Metrics

To get daily usage metrics for 1 or more days, perform a query by specifying the start date and end date in UTC, and metric name.



Example: To get usage metrics for a period of 3 days, from July 19 to July 22, 2020, use these start and end dates. Time values are 0.

- Start date for the query: 2020-07-19T00:00:00.000Z
- End date for the query: 2020-07-22T00:00:00.000Z

This query selects usage metrics for the number of platform events delivered for a 3-day period.

```
SELECT Name, StartDate, EndDate, Value FROM PlatformEventUsageMetric
WHERE Name='CHANGE_EVENTS_DELIVERED'
AND StartDate>=2020-07-19T00:00:00.000Z and EndDate<=2020-07-22T00:00:00.000Z
```

The query returns these results for the specified date range.

Name	StartDate	EndDate	Value
CHANGE_EVENTS_DELIVERED	2020-07-19T00:00:00.000+0000	2020-07-20T00:00:00.000+0000	575
CHANGE_EVENTS_DELIVERED	2020-07-20T00:00:00.000+0000	2020-07-21T00:00:00.000+0000	899
CHANGE_EVENTS_DELIVERED	2020-07-21T00:00:00.000+0000	2020-07-22T00:00:00.000+0000	1,035

General Considerations

If you query the `Id` of `PlatformEventUsageMetric`, the `Id` value returned isn't a valid record ID. For example, this query returns an `Id` field value of 0000000000000000AAA.

```
SELECT Id, Name, StartDate, EndDate, Value FROM PlatformEventUsageMetric WHERE
Name='CHANGE_EVENTS_DELIVERED'
```

As a result, you can't use `PlatformEventUsageMetric` in batch Apex with `QueryLocator` because `QueryLocator` requires valid record IDs to be passed in to the `execute` method. Using `PlatformEventUsageMetric` with batch Apex and `QueryLocator` causes unexpected results. Instead, use an iterable with batch Apex and `PlatformEventUsageMetric`. For more information, see [Using Batch Apex](#) in the *Platform Events Developer Guide*.

SEE ALSO:

[Object Reference for Salesforce and Lightning Platform: PlatformEventUsageMetric](#)

Security Considerations

Learn about the user permissions required for subscription, field-level security, and Shield Platform Encryption.

IN THIS SECTION:[Required Permissions for Change Event Subscribers](#)

Change Data Capture ignores sharing settings and sends change events for all records of a Salesforce object. To receive change events on a channel, the subscribed user must have one or more permissions depending on the entities associated with the change events. The permissions apply to Pub/Sub API and CometD subscribers but not to Apex triggers. Apex triggers run with system privileges under the Automated Process entity, so they don't require those permissions.

[Field-Level Security](#)

Change Data Capture respects your org's field-level security settings. Delivered events contain only the fields that a subscribed user is allowed to view. Before delivering a change event for an object, the subscribed user's field permissions are checked. If a subscribed user has no access to a field, the field isn't included in the change event message that the subscriber receives.

[Change Events for Encrypted Salesforce Data](#)

If Salesforce record fields are encrypted with Shield Platform Encryption, changes in encrypted field values generate change events. Change events are stored in the event bus for up to three days. To ensure that the events stored in the event bus are encrypted and not in clear text, create an event bus tenant secret and enable encryption.

Required Permissions for Change Event Subscribers

Change Data Capture ignores sharing settings and sends change events for all records of a Salesforce object. To receive change events on a channel, the subscribed user must have one or more permissions depending on the entities associated with the change events. The permissions apply to Pub/Sub API and CometD subscribers but not to Apex triggers. Apex triggers run with system privileges under the Automated Process entity, so they don't require those permissions.

Change Event Permissions

To receive change events for	Required permission
A specific standard or custom object:	View All Records for the object
User:	View All Users
Standard objects that don't have the View All Records permission, such as Task and Event:	View All Data
All entities on a channel:	View All Data (AND View All Users, if User is one of the entities)

Permission Enforcement

For the standard `/data/ChangeEvents` channel and custom channels, user permissions are enforced on event delivery. Users can subscribe to the `/data/ChangeEvents` channel or to any custom channel regardless of their entity permissions. Users receive only change events associated with entities for which they have the necessary permissions and don't receive change events they don't have permissions for. If permissions change after subscription, the changes are enforced within 10 minutes for Pub/Sub API subscribers. For CometD subscribers, the changes aren't enforced until you restart the subscription.

For the single-entity standard channels, which include change events for one standard or custom object, user permissions are enforced initially on subscription. If users don't have sufficient permissions for the corresponding object, the subscription is denied and an error is returned. If permissions change after successful subscription and users no longer have access to the entity, they stop receiving the corresponding change events.

For more information about user permissions, see [View All and Modify All Permissions Overview](#) in *Salesforce Help*.

SEE ALSO:

[Subscription Channels](#)

[Pub/Sub API Guide: Channel Membership and User Permission Changes](#)

Field-Level Security

Change Data Capture respects your org's field-level security settings. Delivered events contain only the fields that a subscribed user is allowed to view. Before delivering a change event for an object, the subscribed user's field permissions are checked. If a subscribed user has no access to a field, the field isn't included in the change event message that the subscriber receives.

When describing a change event of a Salesforce object, the describe call checks the user's field-level security settings for that object. The describe call returns only the fields that the user has access to in the describe result of the change event. You can describe a change event through SOAP API or REST API by using the change event name as the sObject name, such as AccountChangeEvent. See [describeSObjects\(\)](#) in the [SOAP API Developer Guide](#) and [sObject Describe](#) in the [REST API Developer Guide](#).

When getting the change event schema corresponding to a Salesforce object, the returned schema includes all object fields, even the fields that the user doesn't have access to. See [Get the Event Schema](#) in the *Platform Events Developer Guide*.

Change Events for Encrypted Salesforce Data

If Salesforce record fields are encrypted with Shield Platform Encryption, changes in encrypted field values generate change events. Change events are stored in the event bus for up to three days. To ensure that the events stored in the event bus are encrypted and not in clear text, create an event bus tenant secret and enable encryption.

To enable encryption of change events, first create an event bus tenant secret on the Key Management page in Setup. Then enable encryption of change events on the Encryption Policy page.



Warning: You must create an event bus tenant secret before enabling encryption. From Setup, the encryption setting is available only after you create an event bus tenant secret. In Metadata API, if you enable encryption using PlatformEncryptionSettings without having the tenant secret, you get an error.

IN THIS SECTION:

[Generate an Event Bus Tenant Secret](#)

To enable encryption of change events, first generate an event bus tenant secret.

[Enable Encryption of Change Events](#)

After you create an event bus tenant secret, a setting becomes available in the Encryption Settings page that starts encryption of change events.

Capturing Changes and Encrypting the Event Payload

After capturing record changes, Change Data Capture creates a change event and stores it in the event bus. Because data changes are captured internally on the application servers in decrypted form, they must be encrypted before storing the corresponding change event that contains them. The entire event payload is encrypted using the data encryption key that is based on the Event Bus tenant secret type.

SEE ALSO:

- [Salesforce Help: Strengthen Your Data's Security with Shield Platform Encryption](#)
- [Salesforce Help: Which User Permissions Does Shield Platform Encryption Require?](#)

Generate an Event Bus Tenant Secret

To enable encryption of change events, first generate an event bus tenant secret.

Prerequisites:

Only authorized users can generate tenant secrets from the Platform Encryption page. Ask your Salesforce admin to assign the Manage Encryption Keys permission to you.

Before generating an Event Bus tenant secret, you must have an active Fields and Files (Probabilistic) or Fields (Deterministic) tenant secret. For instructions, see [Generate a Tenant Secret with Salesforce](#) in Salesforce Help.

USER PERMISSIONS

- To manage tenant secrets:
- Manage Encryption Keys

Steps:

1. From Setup, in the Quick Find box, enter *Platform Encryption*, and then select **Key Management**.
2. In the Key Management Table, select **Event Bus**.
3. Click **Generate Tenant Secret** or, to upload a customer-supplied tenant secret, click **Bring Your Own Key**.

Key Management Table

Key Management Help

These keys encrypt event bus data.

Fields and Files (Probabilistic)

Search Index

Event Bus

Generate Tenant Secret | Bring Your Own Key

Actions	Version	Tenant Secret Type	Status	Key Material Source	Key Derivation	Created By	Last Modified By
Export	1	Event Bus	Active	HSM	✓	Admin User, 12/13/2024, 8:41 AM	Admin User, 12/13/2024, 8:41 AM

Note:

- You can generate or rotate an event bus tenant secret once every 7 days.
- You can also generate a tenant secret through SOAP API or REST API using the TenantSecret object and the Type field value of EventBus. For more information, see [TenantSecret](#) in the [Object Reference for Salesforce and Lightning Platform](#).

SEE ALSO:

- [Salesforce Help: Generate a Tenant Secret with Salesforce](#)

Enable Encryption of Change Events

After you create an event bus tenant secret, a setting becomes available in the Encryption Settings page that starts encryption of change events.

Prerequisites:

[Generate an Event Bus Tenant Secret](#)

Steps:

1. From Setup, in the Quick Find box, enter *Platform Encryption*, and then select **Encryption Settings**.
2. Select **Encrypt Change Data Capture Events and Platform Events**.



Note: When you enable encryption for change events, you also enable it for platform events. For more information, see [Encrypting Platform Event Messages at Rest in the Event Bus](#) in the *Platform Events Developer Guide*.

Capturing Changes and Encrypting the Event Payload

After capturing record changes, Change Data Capture creates a change event and stores it in the event bus. Because data changes are captured internally on the application servers in decrypted form, they must be encrypted before storing the corresponding change event that contains them. The entire event payload is encrypted using the data encryption key that is based on the Event Bus tenant secret type.

When Shield Platform Encryption is enabled, Change Data Capture encrypts the fields of all Salesforce objects that it tracks. Change Data Capture ignores the object and field selections set up for Shield Platform Encryption. Fields of all objects for which changes are tracked are encrypted before event storage, even objects not selected for Shield Platform Encryption. For example, suppose that only the Mailing Address of contacts is encrypted with Shield Platform Encryption. If data changes occur in accounts and contacts, change events for both accounts and contacts are encrypted.

Delivering Change Events

Before delivering a change event to a subscribed client, the change event payload is decrypted using the data encryption key. The change event is sent over a secure channel using HTTPS and TLS, which ensures that the data is protected and encrypted while in transit. If the encryption key was rotated and a new key is issued, stored events are not re-encrypted but they are decrypted before delivery using the archived key. If a key is destroyed, stored events can't be decrypted and aren't delivered.



Note: Classic Encryption is not supported.

Change Event Considerations

Keep in mind change event considerations and allocations when subscribing to change events.

IN THIS SECTION:

[General Considerations](#)

[Change Data Capture Allocations](#)

Learn about the allocations for change events including the number of custom channels, selected entities in a channel, and event delivery.

General Considerations

No Change Events Generated for Some Actions

These actions don't generate change events for affected records.

- Hard-deleting a record, that is, deleting a record from the Recycle Bin.
- Any action related to state and country/territory picklists that you perform in Setup on the State and Country/Territory Picklists page.
- Changing the type of an opportunity stage picklist value.

- When a custom picklist field is defined on Contact in a person account org, the field is present on Account with the `__pc` suffix. Replacing or renaming a value of the custom picklist doesn't generate account change events but only contact change events for the affected records. But if the custom picklist field is defined on Account, the field isn't present on Contact and only account change events are generated, as expected.

Change Event Generation in a Transaction with Multiple Changes for the Same Record

If multiple DML operations are performed on a record within the same transaction boundary, only one change event is generated for the initial change type. The change event contains the data that is committed in Salesforce at the end of the transaction. No change events are generated for the additional operations because they're internal to the transaction. For example, a case record is created and an after-insert trigger queries this case before the transaction is committed. The trigger changes the case priority from `Medium` to `High` and performs an update operation. After the transaction is committed, one change event is generated with a `changeType` of `CREATE` and priority of `High`.

Event Replay ID Field

See the [Event Replay ID Field](#) Pub/Sub API documentation.

Formula Field Support

Custom formula fields are included in change events, but updates to the formula field don't always trigger a change event. Formula fields don't appear in the changed record in Apex Triggers on `ChangeEvent` objects. Derived fields are not supported in Change Data Capture. For information on which fields are included and excluded in a change event, see [Change Event Fields](#).

Formula Field Numeric Digit Limits and Gap Events

When including numeric formula fields, ensure that the value does not exceed the Salesforce limit of 18 total digits (integer and decimal parts combined). If a formula field has a high number of decimal places configured, the system may pad the value with zeros to meet the configured precision, potentially causing the total number of digits to exceed 18 even if the original value has fewer digits. Change events containing values that exceed this limit are not captured and a gap event is generated instead.

Geolocation Compound Fields

When a geolocation compound field (of type `location`) is changed in a custom object, all its component fields are published in the change event whether they were changed or not. In contrast, when a geolocation field is changed in a standard object, only the changed field is published. For more information, see [Geolocation Compound Fields](#) in the *Salesforce Object Reference*.

Data Cloud Channel for Data Streams

In Data Cloud, when you create a data stream by using the Salesforce CRM connector, the data stream uses Change Data Capture for synchronizing the record data if the selected objects for the data stream support change events. The selections are added to the `DataCloudEntities` standard channel. Make sure that you don't modify the selections for this standard channel by using Metadata API or Tooling API. Updating the channel selections by using the API can result in unexpected synchronizations of Salesforce objects that the Data Cloud administrator didn't intend. For more information, see [Create a Salesforce CRM Data Stream](#) in *Salesforce Help*.

Change Data Capture Allocations

Learn about the allocations for change events including the number of custom channels, selected entities in a channel, and event delivery.

IN THIS SECTION:

[Common Change Event Allocations](#)

Common allocations include the concurrent CometD client allocation and the maximum event message size.

[Default Change Event Allocations for Event Delivery](#)

If your org has no add-on licenses, default allocations apply for event publishing and delivery that can't be exceeded. The default allocations are enforced to ensure fair sharing of resources in the multitenant environment and to protect the service.

Get the Number of Selected Entities

To get the number of selected entities, perform a SOQL query in Tooling API on PlatformEventChannelMember.

Allocations for AppExchange Released Managed Packages

The allocation for selected entities doesn't include selections that installed AppExchange released managed packages make in a custom channel that's part of the package. The maximum number of entity selections of 5 applies to selections that you make, or selections that an unmanaged or managed package makes, except for AppExchange released managed packages.

Increase Your Event Delivery Allocation with an Add-On License

To increase your event delivery allocation for Pub/Sub API, CometD, `empApi` Lightning components, and event relays, purchase an add-on for additional change events. The add-on also increases the event publishing allocation. The add-on moves your event delivery usage to a monthly usage-based entitlement model and allows for spikes in usage. To purchase an add-on, contact your Salesforce Account Representative.

Monitor Event Usage Against Your Allocations

Check your event publishing and delivery usage and maximum allocation in Setup, or use REST API or Apex.

SEE ALSO:

[Platform Events Developer Guide: Platform Event Allocations](#)

Common Change Event Allocations

Common allocations include the concurrent CometD client allocation and the maximum event message size.

Description	Performance and Unlimited Editions	Enterprise Edition	Developer Edition
Maximum number of concurrent CometD clients (subscribers) across all channels and for all event types	2,000	1,000	20
The maximum event message size that Salesforce can publish is 1 MB. If your entity has hundreds of custom fields or many long text area fields, you can reach this limit. If so, the change event message isn't delivered and is replaced by a gap event message. For more information, see Gap Events .	1 MB	1 MB	1 MB



Note: The concurrent client allocation applies to CometD and to all types of events: platform events, change events, PushTopic events, and generic events. It doesn't apply to non-CometD clients, such as Apex triggers, flows, and Process Builder processes. Flows and Process Builder processes apply only to platform events and not to change events. The `empApi` Lightning component uses CometD and consumes the concurrent client allocation like any other CometD client. Each logged-in user using `empApi` counts as one concurrent client. If the user has multiple browser tabs using `empApi`, the streaming connection is shared and is counted as one client for that user. A client that exceeds the concurrent client allocation receives an error and can't subscribe. When one of the clients disconnects and a connection is available, the new client can subscribe. For more information, see [Streaming API Error Codes](#) in the [Streaming API Developer Guide](#).

Default Change Event Allocations for Event Delivery

If your org has no add-on licenses, default allocations apply for event publishing and delivery that can't be exceeded. The default allocations are enforced to ensure fair sharing of resources in the multitenant environment and to protect the service.

- The event delivery allocation is how many event messages can be delivered in a 24-hour period to Pub/Sub API and CometD subscribers, `empApi` Lightning components, and event relays. It excludes non-API subscribers, such as Apex triggers, flows, and Process Builder processes. Published event messages that are delivered to non-API subscribers, such as Apex triggers, flows, and Process Builder processes, don't count against the delivery allocation.
- The event delivery allocation is shared between high-volume platform events and Change Data Capture events.

 **Note:** Even though Apex triggers don't count against the event delivery limit, their event processing rate depends on the subscriber processing time and volume of events received. A higher processing time and event volume means that it takes longer for the subscriber to reach the tip of the event stream.

Event Delivery Usage Combined for All Subscribers

The number of delivered events to clients is counted for each subscribed client, including event relays. If you have multiple client subscribers, your usage is added across all subscribers. For example, you have an Unlimited Edition org with a default allocation of 50,000 events in a 24-hour period. Within a few hours, 20,000 event messages are delivered to two subscribed clients. So you consumed 40,000 events and are still entitled to 10,000 events within the 24-hour period.

How Is Event Delivery Usage Calculated?

The daily event delivery limit is a rolling limit. It's calculated for the number of delivered events in the last 24 hours. As time goes by, the usage is updated. The event delivery limit is checked when a new event is received.

To learn more about how event usage is calculated against your event allocations, see [Learn About Daily Rate Limits](#) in the [App Development Without Limits](#) Trailhead module.

Table 1: Default Allocations

Description	Subscriber Clients	Performance and Unlimited Editions	Enterprise Edition	Developer Edition
Maximum number of entities, including standard and custom objects, that you can select across all channels, including the default standard channel and custom channels. This allocation doesn't apply to entity selections made by AppExchange packages in a custom channel. If the same entity is selected in multiple channels, it's counted once toward the allocation. To increase this allocation by purchasing an add-on, see Which Allocations Can Be Increased?	Not applicable	5	5	5
Maximum number of custom channels This allocation is separate from the one for custom platform event channels.	Not applicable	100	100	100

Description	Subscriber Clients	Performance and Unlimited Editions	Enterprise Edition	Developer Edition
Event Delivery: maximum number of delivered event messages in the last 24 hours, shared by all clients. To increase this allocation by purchasing an add-on, see Which Allocations Can Be Increased?.	This allocation applies to: ✓ Pub/Sub API ✓ CometD ✓ empApi Lightning component ✓ Event relays This allocation doesn't apply to: ✗ Apex triggers ✗ Flows ✗ Process Builder processes	50,000	25,000	10,000



Note: Salesforce publishes change events in response to record changes, so it doesn't enforce a publishing limit for Change Data Capture because users don't control the total events published.

How to Avoid Exceeding Event Allocations

Proactively monitor your event usage. For more information, see [Monitor Event Usage Against Your Allocations](#) and [Enhanced Usage Metrics](#) in the *Platform Events Developer Guide*. When your event delivery usage gets close to the allocation, try these methods to reduce the consumption of delivered events.

- Use stream filtering to reduce the amount of events delivered to the subscriber and receive only relevant events. For more information, see [Filter Your Stream of Change Events with Channels](#).
- Use custom channels instead of the default `ChangeEvent`s channel to only receive the events related to the entities you're interested in. See [Compose Streams of Change Data Capture Notifications with Custom Channels](#).
- Make sure you don't have unnecessary subscribers. Each event delivered to a subscriber counts against the event delivery allocation.

What to Do If You Exceed the Event Delivery Allocation

If you exceed the default event delivery allocation, an error is returned and the subscription is disconnected.

- The error you receive in a CometD client is: `403::Organization total events daily limit exceeded`. The error is returned in the `Bayeux /meta/connect` channel when a CometD subscriber first connects or in an existing subscriber connection. For more information, see [Streaming API Error Codes](#) in the [Streaming API Developer Guide](#).
- The error code that you receive in a Pub/Sub API client is: `sfdc.platform.eventbus.grpc.subscription.limit.exceeded`. And the error message is: `You have exceeded the event delivery limit for your org.`

When the client reaches the event delivery allocation, perform one of these steps.

- Keep the subscriber disconnected for a temporary time. While the subscriber is disconnected, the event usage for the last 24 hours decreases after some time. The events received in Salesforce during the disconnected state are stored for the retention period of 72 hours. After usage decreases, resume the subscription from where it left off and receive events. You can retrieve stored event messages with Pub/Sub API and CometD using the Replay ID.
- If you reach the event delivery limit often and your event volume is high, consider purchasing an add-on to increase your event allocations by contacting your Salesforce Account Representative. See [Which Allocations Can Be Increased?](#) on page 77.

Which Allocations Can Be Increased?

You can increase the event delivery allocation and remove the limit on the maximum number of entities selected for change notifications. To do so, purchase a Change Data Capture add-on by contacting your Salesforce Account Representative. The add-on moves your event delivery usage to a monthly entitlement model and allows for spikes in usage. See [Increase Your Event Delivery Allocation with an Add-On License](#) on page 78.

SEE ALSO:

[Increase Your Event Delivery Allocation with an Add-On License](#)

[Monitor Event Usage Against Your Allocations](#)

[Platform Events Developer Guide: Platform Event Allocations](#)

[Salesforce Developers Blog: How to Work Within Platform Events Delivery Limits](#)

[Pub/Sub API Developer Guide](#)

[Salesforce Help: Event Relay](#)

[Streaming API Developer Guide](#)

Get the Number of Selected Entities

To get the number of selected entities, perform a SOQL query in Tooling API on PlatformEventChannelMember.

To verify the current usage of selected entities, perform this Tooling API query through REST or in the Developer Console Query Editor.

```
SELECT COUNT_DISTINCT(SelectedEntity) FROM PlatformEventChannelMember
```

The query gets the number of unique entities selected across all channels. The query can return a number higher than the selected entities allocation if entity selections are made by AppExchange packages. The latter selections aren't counted against the selected entities allocation.

The `SelectedEntity` field of PlatformEventChannelMember in Metadata API and Tooling API represents the entities selected through the user interface or the API. For more information, see [Select Objects for Change Notifications with Metadata API and Tooling API](#).

For more information about the Tooling API query REST resource, see [REST Resources](#) in the *Tooling API Developer Guide*.

Allocations for AppExchange Released Managed Packages

The allocation for selected entities doesn't include selections that installed AppExchange released managed packages make in a custom channel that's part of the package. The maximum number of entity selections of 5 applies to selections that you make, or selections that an unmanaged or managed package makes, except for AppExchange released managed packages.

If you install an AppExchange released managed package, the selections made by the AppExchange package in a custom channel don't count against your org's allocation. You can install the AppExchange package even if the org reaches the maximum number of selected entities default allocation. Also, installing the AppExchange package doesn't alter the current usage for the number of selected entities.

This statement holds true for first- and second-generation packages. For package developers, the entity selection allocation is still enforced in the package development org.

SEE ALSO:

[Salesforce AppExchange](#)

Increase Your Event Delivery Allocation with an Add-On License

To increase your event delivery allocation for Pub/Sub API, CometD, `empApi` Lightning components, and event relays, purchase an add-on for additional change events. The add-on also increases the event publishing allocation. The add-on moves your event delivery usage to a monthly usage-based entitlement model and allows for spikes in usage. To purchase an add-on, contact your Salesforce Account Representative.

Check out the benefits and facts about an add-on license.

- The add-on increases the 24-hour allocation of delivered event messages by 100,000 per day (3 million a month) as a usage-based entitlement.
- The add-on removes the limit on the maximum number of entities selected for change notifications.
- The daily delivery usage isn't as strictly enforced as the default allocation. The add-on allows for spikes in usage through a grace allocation. The grace allocation is higher than the allocation that you purchased through the add-on license. As long as the daily event delivery usage is within the grace allocation, your subscribers aren't stopped and can continue receiving events. Salesforce reserves the right to adjust grace allocations at any time.
- The entitlement is reset every month after your contract start date.
- Entitlement usage is computed only for production orgs. It isn't available in sandbox or trial orgs. For more information, see [Usage-based Entitlement Fields](#).
- Salesforce monitors event overages based on a calendar month, starting with your contract start date. If you exceed the monthly entitlement, Salesforce contacts you to discuss your event usage needs. The entitlement used for monitoring monthly event overages is the daily allocation multiplied by 30.

Table 2: Example: Usage-Based Entitlement with One Change Data Capture Add-On License

Description	Subscriber Clients	Performance and Unlimited Editions	Enterprise Edition
Maximum number of entities, including standard and custom objects, that you can select for Change Data Capture.	Not Applicable	No limit	No limit
Event Delivery: entitlement for delivered event messages, shared by all clients. You can exceed this entitlement by a certain amount before receiving an error. Salesforce uses the monthly entitlement for event overage monitoring. The monthly entitlement is returned in the <code>limits</code> REST API resource.	This entitlement applies to: ✓ Pub/Sub API ✓ CometD ✓ <code>empApi</code> Lightning component ✓ Event relays This allocation doesn't apply to: ✗ Apex triggers	Last 24 hours: 150,000 (50 K included with org license + 100 K from add-on license + grace amount) Monthly entitlement: 4.5	Last 24 hours: 125,000 (25 K included with org license + 100 K from add-on license + grace amount) Monthly entitlement:

Description	Subscriber Clients	Performance and Unlimited Editions	Enterprise Edition
	✖ Flows ✖ Process Builder processes	million (1.5 million included with org license + 3 million from add-on license)	3.75 million (0.75 million included with org license + 3 million from add-on license)

To increase the entitlement for event delivery, contact your Salesforce Account Representative to purchase the add-on for additional change events.

Monitor Event Usage Against Your Allocations

Check your event publishing and delivery usage and maximum allocation in Setup, or use REST API or Apex.

Check your change event delivery usage in the user interface, which is shared with platform events. From Setup, in the Quick Find box, enter *Platform Events*, and then select **Platform Events**. The usage is shown in the Event Allocations section. The publishing usage doesn't apply to change events.

Event Allocations 		
Item	Usage	Allocation
High-Volume Platform Event Hourly Publishing Allocation	81	50,000
High-Volume Platform Event and Change Event Daily Delivery Allocation	81	10,000

If your org purchased the add-on for platform events or change data capture, the grace allocation is displayed in addition to the allocation for daily event delivery. This value corresponds to the `DailyDeliveredPlatformEvents` REST API `limits` value. The monthly event delivery usage is also displayed. It corresponds to the `MonthlyPlatformEvents` REST API `limits` value.

Event Allocations 			
Item	Usage	Allocation	Grace Allocation
High-Volume Platform Event Hourly Publishing Allocation	48,011	275,000	Not Applicable
High-Volume Platform Event and Change Event Daily Delivery Allocation	126,142	125,000	525,000
High-Volume Platform Event and Change Event Monthly Delivery Allocation	4,000,052	3,750,000	Not Applicable

Learn about other ways to check event usage with REST API, Apex, and in the Company Information page.

Allocation	Default Allocations	Add-On License
Event Delivery: number of delivered event notifications to CometD and Pub/Sub API clients, <code>empApi</code> Lightning components, and event relays	If your org hasn't purchased the add-on, check your usage in one of these ways. Daily event delivery usage in the last 24 hours using REST API: Check the <code>DailyDeliveredPlatformEvents</code> value with the REST API <code>limits</code> resource.	If your org has purchased the add-on, check your usage in one of these ways. - Daily event delivery in the last 24 hours as mentioned in the previous column. - Monthly event delivery usage: From Setup, in the Quick Find box, enter <i>Platform Events</i>, and then select

Allocation	Default Allocations	Add-On License
	Daily event delivery usage in the last 24 hours using Apex: Use the <code>System.OrgLimit</code> class and check the <code>DailyDeliveredPlatformEvents</code> value. The daily event delivery usage is updated within a few minutes after event delivery.	Platform Events. The monthly event delivery usage is displayed in the Event Allocations section. In the REST API <code>limits</code> resource, this value corresponds to <code>MonthlyPlatformEvents</code> in API version 47.0 and earlier. This value in the UI and API is updated within a few minutes after event delivery. Usage-based entitlement: From Setup, in the Quick Find box, enter <i>Company Information</i>, and then select Company Information. The usage is shown under the Usage-based Entitlements related list. In the REST API <code>limits</code> resource, this value corresponds to <code>MonthlyPlatformEventsUsageEntitlement</code> in API version 48.0 and later. This value in the UI and API is updated once a day.

Monitor Hourly Event Usage

To monitor event delivery usage by the hour, retrieve the daily event delivery usage using REST API every hour. For more information, see [Monitor Hourly Event Delivery Usage with REST API](#) in the *Platform Events Developer Guide*.

Track Event Usage Trends with SOQL Queries by Using PlatformEventUsageMetric

Perform a SOQL query on `PlatformEventUsageMetric` to get visibility into your event usage and usage trends. With enhanced usage metrics, you can view separate and combined metrics for platform events and change data capture events. Break down usage metrics by event name, client ID, event type, and usage type, and get usage data by granular time segments. `PlatformEventUsageMetric` data is available for CometD and Pub/Sub API clients, `empApi`, Lightning components, and event relays. For more information, see [Enhanced Usage Metrics](#) in the *Platform Events Developer Guide*.

SEE ALSO:

[REST API Developer Guide: Limits](#)

[REST API Developer Guide: List Org Limits](#)

[Apex Reference Guide: OrgLimit Class](#)

Standard Object Notes

Learn about the characteristics of change events for some standard objects and the fields included in the event messages.

IN THIS SECTION:

[Change Events for Tasks and Events](#)

You can receive change events for single and recurring tasks and calendar events, including events with invitees.

[Change Events for Person Accounts](#)

Because a person account record combines fields from an account and a contact, changing a person account results in two change events: one for the account and one for the contact, provided that both objects are selected for change data capture. The two change events are generated for all changes to a person account, including create, update, delete, and undelete operations.

[Change Events for Users](#)

The user and email preferences in change events include only the preferences that are enabled (set to true) without their Boolean values. Preferences that are disabled (set to false) are not included in the event payload.

[Change Events for Lead Conversion](#)

Converting a lead results in the creation of an account, a contact, and optionally an opportunity, and also a lead update. When converting a lead, the change event for the lead update includes fields specific to the conversion.

[Change Events for PricebookEntry](#)

The Create Change Events payload does not include the system fields (sCreatedBy, CreatedDate, LastModifiedBy, and LastModifiedDate). This is due to the highly customized nature of the PricebookEntry object, which prevents these specific fields from being captured at the time the event is generated.

Change Events for Tasks and Events

You can receive change events for single and recurring tasks and calendar events, including events with invitees.

IN THIS SECTION:

[Recurring Activities](#)

The activity series record is tracked in a single change event. Each occurrence in the series is tracked by an individual change event.

[Event Invitees](#)

Change events are generated for event invitees in addition to the calendar event record. When a Salesforce user is invited to a calendar event, a child calendar event record is created for the invitee. A child calendar event is an Event record with the IsChild field set to true and OwnerId set to the invitee's user ID.

[Updating Recurring Calendar Events](#)

If a critical change is made to a recurring calendar event, such as changing the recurrence pattern or the recurrence start date, the series is deleted and recreated.

[Shared Activities and Parent Records for Tasks and Events](#)

If Shared Activities is enabled, the relationships between a task and its parent records (for example, contacts and lead), which correspond to TaskRelation objects, are tracked through change events.

Recurring Activities

The activity series record is tracked in a single change event. Each occurrence in the series is tracked by an individual change event.



Example: These two change events are delivered to a Pub/Sub API client when creating a recurring calendar event. The first change event is for the event series record, which represents the recurrence pattern, with `GroupEventType` set to 3. The

second change event is for the first occurrence. The `ActivityDateTime` and `ActivityDate` are in Epoch time. The other occurrences are omitted in this example.

```
// Change event generated for the event series record.
{
  "ChangeEventHeader": {
    "entityName": "Event",
    "recordIds": [
      "00UZM000000wuBw2AI"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
    "transactionKey": "000a48c7-f73c-5d0b-7650-123ab3a20e70",
    "sequenceNumber": 1,
    "commitTimestamp": 1714408223000,
    "commitNumber": 72779532308,
    "commitUser": "005ZM000000M6o1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "WhoId": null,
  "WhatId": null,
  "Subject": "Product Planning",
  "Location": "San Francisco",
  "IsAllDayEvent": false,
  "ActivityDateTime": 1715014800000,
  "ActivityDate": 1714953600000,
  "DurationInMinutes": 60,
  "Description": "Let's meet to discuss product requirements.",
  "AccountId": null,
  "OwnerId": "005ZM000000M6o1YAC",
  "Type": null,
  "IsPrivate": false,
  "ShowAs": "Busy",
  "IsChild": false,
  "IsGroupEvent": false,
  "GroupEventType": "3",
  "CreatedDate": 1714408223000,
  "CreatedById": "005ZM000000M6o1YAC",
  "LastModifiedDate": 1714408223000,
  "LastModifiedById": "005ZM000000M6o1YAC",
  "RecurrenceActivityId": null,
  "IsRecurrence": false,
  "RecurrenceStartDateTime": null,
  "RecurrenceEndDateOnly": null,
  "RecurrenceTimeZoneSidKey": null,
  "RecurrenceType": null,
  "RecurrenceInterval": null,
  "RecurrenceDayOfWeekMask": null,
  "RecurrenceDayOfMonth": null,
  "RecurrenceInstance": null,
  "RecurrenceMonthOfYear": null,
  "ReminderDateTime": null,
}
```

```

    "IsReminderSet": false,
    "IsRecurrence2Exclusion": false,
    "Recurrence2PatternText": "RRULE:FREQ=WEEKLY;BYDAY=MO;WKST=SU;INTERVAL=1;COUNT=13",

    "Recurrence2PatternVersion": "1",
    "ActivityRecurrence2Id": "828ZM0000000001YAA",
    "ActivityRecurrence2ExceptionId": null
  }

  // Change event generated for the first occurrence.
  {
    "ChangeEventHeader": {
      "entityName": "Event",
      "recordIds": [
        "00UZM000000wuBx2AI"
      ],
      "changeType": "CREATE",
      "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
      "transactionKey": "000a48c7-f73c-5d0b-7650-123ab3a20e70",
      "sequenceNumber": 2,
      "commitTimestamp": 1714408224000,
      "commitNumber": 72779532308,
      "commitUser": "005ZM000000M6o1YAC",
      "nulledFields": [],
      "diffFields": [],
      "changedFields": []
    },
    "WhoId": null,
    "WhatId": null,
    "Subject": "Product Planning",
    "Location": "San Francisco",
    "IsAllDayEvent": false,
    "ActivityDateTime": 1715014800000,
    "ActivityDate": 1714953600000,
    "DurationInMinutes": 60,
    "Description": "Let's meet to discuss product requirements.",
    "AccountId": null,
    "OwnerId": "005ZM000000M6o1YAC",
    "Type": null,
    "IsPrivate": false,
    "ShowAs": "Busy",
    "IsChild": false,
    "IsGroupEvent": false,
    "GroupEventType": "0",
    "CreatedDate": 1714408224000,
    "CreatedById": "005ZM000000M6o1YAC",
    "LastModifiedDate": 1714408224000,
    "LastModifiedById": "005ZM000000M6o1YAC",
    "RecurrenceActivityId": null,
    "IsRecurrence": false,
    "RecurrenceStartDateTime": null,
    "RecurrenceEndDateOnly": null,
    "RecurrenceTimeZoneSidKey": null,
    "RecurrenceType": null,

```

```

"RecurrenceInterval": null,
"RecurrenceDayOfWeekMask": null,
"RecurrenceDayOfMonth": null,
"RecurrenceInstance": null,
"RecurrenceMonthOfYear": null,
"ReminderDateTime": null,
"IsReminderSet": false,
"IsRecurrence2Exclusion": false,
"Recurrence2PatternText": "RRULE:FREQ=WEEKLY;BYDAY=MO;WKST=SU;INTERVAL=1;COUNT=13",

"Recurrence2PatternVersion": "1",
"ActivityRecurrence2Id": "828ZM0000000001YAA",
"ActivityRecurrence2ExceptionId": null
}

```

Event Invitees

Change events are generated for event invitees in addition to the calendar event record. When a Salesforce user is invited to a calendar event, a child calendar event record is created for the invitee. A child calendar event is an Event record with the `IsChild` field set to true and `OwnerId` set to the invitee's user ID.

A child calendar event isn't created for an invitee who isn't a Salesforce user, such as a contact, a lead, or a resource. For each invitee added, an `EventRelation` record is created to represent the relationship to the calendar event. In a recurring series, child calendar events are created for invitees in each occurrence.

For example, if a calendar event is created with two invitees, three calendar event records are created in Salesforce: one record for the calendar event, and two records for the invitees. The three records result in three change events being generated on the channel for the Event standard object. In addition, two `EventRelation` records are created and generate two change events on the channel for `EventRelation`.



Example: These events are delivered to a Pub/Sub API client when creating a calendar event and inviting one user. The first change event is for the calendar event. The second change event is the child calendar event for the invitee, with `IsChild` set to true and `OwnerId` set to the invitee's user ID. The third change event is for the `EventRelation` record representing the relationship between the invitee and the calendar event.

```

// Change event generated for the calendar event record.
{
  "ChangeEventHeader": {
    "entityName": "Event",
    "recordIds": [
      "00UZM00000165Ps2AI"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
    "transactionKey": "000a4b85-7abf-fb12-9d7f-d5d6987da21b",
    "sequenceNumber": 1,
    "commitTimestamp": 1714411236000,
    "commitNumber": 72780631358,
    "commitUser": "005ZM000000M6o1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
}

```

```

    "WhoId": null,
    "WhatId": null,
    "Subject": "Meeting with Jane",
    "Location": "San Francisco",
    "IsAllDayEvent": false,
    "ActivityDateTime": 1715104800000,
    "ActivityDate": 1715040000000,
    "DurationInMinutes": 60,
    "Description": "One-on-one meeting with Jane.",
    "AccountId": null,
    "OwnerId": "005ZM000000M6o1YAC",
    "Type": null,
    "IsPrivate": false,
    "ShowAs": "Busy",
    "IsChild": false,
    "IsGroupEvent": true,
    "GroupEventType": "1",
    "CreatedDate": 1714411236000,
    "CreatedById": "005ZM000000M6o1YAC",
    "LastModifiedDate": 1714411236000,
    "LastModifiedById": "005ZM000000M6o1YAC",
    "RecurrenceActivityId": null,
    "IsRecurrence": false,
    "RecurrenceStartDateTime": null,
    "RecurrenceEndDateOnly": null,
    "RecurrenceTimeZoneSidKey": null,
    "RecurrenceType": null,
    "RecurrenceInterval": null,
    "RecurrenceDayOfWeekMask": null,
    "RecurrenceDayOfMonth": null,
    "RecurrenceInstance": null,
    "RecurrenceMonthOfYear": null,
    "ReminderDateTime": null,
    "IsReminderSet": false,
    "IsRecurrence2Exclusion": false,
    "Recurrence2PatternText": null,
    "Recurrence2PatternVersion": null,
    "ActivityRecurrence2Id": null,
    "ActivityRecurrence2ExceptionId": null
  }

  // Change event generated for the child calendar event record for the invitee.
  {
    "ChangeEventHeader": {
      "entityName": "Event",
      "recordIds": [
        "00UZM00000165Pt2AI"
      ],
      "changeType": "CREATE",
      "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
      "transactionKey": "000a4b85-7abf-fb12-9d7f-d5d6987da21b",
      "sequenceNumber": 2,
      "commitTimestamp": 1714411237000,

```

```

    "commitNumber": 72780631358,
    "commitUser": "005ZM000000M6o1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "WhoId": null,
  "WhatId": null,
  "Subject": "Meeting with Jane",
  "Location": "San Francisco",
  "IsAllDayEvent": false,
  "ActivityDateTime": 1715104800000,
  "ActivityDate": 1715040000000,
  "DurationInMinutes": 60,
  "Description": "One-on-one meeting with Jane.",
  "AccountId": null,
  "OwnerId": "005ZM000000MJSoyAO",
  "Type": null,
  "IsPrivate": false,
  "ShowAs": "Busy",
  "IsChild": true,
  "IsGroupEvent": true,
  "GroupEventType": "1",
  "CreatedDate": 1714411237000,
  "CreatedById": "005ZM000000M6o1YAC",
  "LastModifiedDate": 1714411237000,
  "LastModifiedById": "005ZM000000M6o1YAC",
  "RecurrenceActivityId": null,
  "IsRecurrence": false,
  "RecurrenceStartDateTime": null,
  "RecurrenceEndDateOnly": null,
  "RecurrenceTimeZoneSidKey": null,
  "RecurrenceType": null,
  "RecurrenceInterval": null,
  "RecurrenceDayOfWeekMask": null,
  "RecurrenceDayOfMonth": null,
  "RecurrenceInstance": null,
  "RecurrenceMonthOfYear": null,
  "ReminderDateTime": null,
  "IsReminderSet": false,
  "IsRecurrence2Exclusion": false,
  "Recurrence2PatternText": null,
  "Recurrence2PatternVersion": null,
  "ActivityRecurrence2Id": null,
  "ActivityRecurrence2ExceptionId": null
}

// Change event generated for the EventRelation record representing the
// relationship between the invitee and the calendar event.
{
  "ChangeEventHeader": {
    "entityName": "EventRelation",
    "recordIds": [

```

```

    "0REZM000000005o4AA"
  ],
  "changeType": "CREATE",
  "changeOrigin": "com/salesforce/api/soap/61.0;client=SfdcInternalAPI/",
  "transactionKey": "000a4b85-7abf-fb12-9d7f-d5d6987da21b",
  "sequenceNumber": 3,
  "commitTimestamp": 1714411237000,
  "commitNumber": 72780631358,
  "commitUser": "005ZM000000M6o1YAC",
  "nulledFields": [],
  "diffFields": [],
  "changedFields": []
},
"RelationId": "005ZM000000MJSOYAO",
"EventId": "00UZM00000165Ps2AI",
"IsWhat": false,
"IsParent": false,
"IsInvitee": true,
"AccountId": null,
"Status": "New",
"RespondedDate": null,
"Response": null,
"CreatedDate": 1714411237000,
"CreatedById": "005ZM000000M6o1YAC",
"LastModifiedDate": 1714411237000,
"LastModifiedById": "005ZM000000M6o1YAC"
}

```

SEE ALSO:

[Object Reference for Salesforce and Force.com: EventRelation](#)

Updating Recurring Calendar Events

If a critical change is made to a recurring calendar event, such as changing the recurrence pattern or the recurrence start date, the series is deleted and recreated.

If a recurring calendar event contains many invitees and many occurrences, a critical change can lead to many change events. For example, updating the recurrence start date of a calendar event with 100 occurrences and 100 invitees results in deleting and recreating 10,000 child Event records (100 records x 100 occurrences) and 10,000 EventRelation records. A high volume of changes in a single transaction could generate overflow events. For more information, see [Overflow Events](#).

SEE ALSO:

[Object Reference for Salesforce and Force.com: EventRelation](#)

Shared Activities and Parent Records for Tasks and Events

If Shared Activities is enabled, the relationships between a task and its parent records (for example, contacts and lead), which correspond to TaskRelation objects, are tracked through change events.

Similarly, the relationships between a calendar event and its parent records, which correspond to `EventRelation` objects, are tracked. You can receive change events for task relationships on the `/data/TaskRelationChangeEvent` channel, and for event relationships on the `/data/EventRelationChangeEvent` channel.

When Shared Activities is not enabled, `EventRelation` objects associate a calendar event with invitees only.

SEE ALSO:

[Salesforce Help: Enable Shared Activities](#)

[Object Reference for Salesforce and Force.com: EventRelation](#)

[Object Reference for Salesforce and Force.com: TaskRelation](#)

Change Events for Person Accounts

Because a person account record combines fields from an account and a contact, changing a person account results in two change events: one for the account and one for the contact, provided that both objects are selected for change data capture. The two change events are generated for all changes to a person account, including create, update, delete, and undelete operations.



Note: To receive change events for person account records, enable both Account and Contact for change data capture. If only Account is selected and a person account is updated, the account change event doesn't contain the fields that stem from the contact. Examples of such fields are `PersonAssistantName`, which corresponds to the contact `AssistantName` field, or a contact custom field. This behavior doesn't apply when creating or undeleting a person account—the account change event contains the contact fields even if Contact isn't selected for capture.

IN THIS SECTION:

[Creating and Undeleting a Person Account](#)

When creating or undeleting a person account, the account change event contains both account and contact fields. It contains account record fields and some fields from the contact record. The contact fields that the account change event includes are all custom contact fields and some standard contact fields, which start with the Person prefix.

[Updating a Person Account](#)

When updating a person account, two change events are generated, one for the account and one for the contact, regardless which fields changed. Salesforce always updates the `LastModifiedDate` system field in both the account and contact even if the field updated is only in one of the underlying records.

[Converting an Account](#)

If a person account is converted to a business account through the API by modifying the record type ID, a change event for the account is generated. This change event contains the new record type ID of the account.

[Deleting a Person Account](#)

When deleting a person account, two change events are generated: one for the deleted account and one for the deleted contact. The change events don't contain record fields. They contain only event header fields.

SEE ALSO:

[Salesforce Help: Person Accounts](#)

[Select Objects for Change Notifications in the User Interface](#)

[Select Objects for Change Notifications with Metadata API and Tooling API](#)

Creating and Undeleting a Person Account

When creating or undeleting a person account, the account change event contains both account and contact fields. It contains account record fields and some fields from the contact record. The contact fields that the account change event includes are all custom contact fields and some standard contact fields, which start with the Person prefix.

The contact change event contains all contact standard and custom fields. The contact change event doesn't contain the account fields of the person account.

The `Name` field is included in both the account and contact change events for a new or undeleted person account.



Example: These two change events are generated when a person account is created. The first event is for the Account standard object. The account change event contains the `Name` field, which is the person account name. This field is also included in the contact change event. The second event is for the Contact standard object. The contact has a custom field named `CustomContactField__c`, which is part of both the contact and account change events. In the account change event, the contact custom field ends with the `__pc` prefix.

```
// Change event for Account
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "001ZL000001QS6mYAG"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "000a6a7b-b859-8ba4-2ab8-1a1babf66ff0",
    "sequenceNumber": 1,
    "commitTimestamp": 1714431097000,
    "commitNumber": 71542214535,
    "commitUser": "005ZL000000M0x1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "Name": {
    "Salutation": "Ms.",
    "FirstName": "Martha",
    "LastName": "Brown"
  },
  "Type": null,
  "RecordTypeId": "012ZL0000008Td4YAE",
  "ParentId": null,
  "BillingAddress": {
    "Street": "415 Mission Street",
    "City": "San Francisco",
    "State": "CA",
    "PostalCode": "94105",
    "Country": "US",
    "Latitude": null,
    "Longitude": null,
    "GeocodeAccuracy": null
  },
  "ShippingAddress": null,
  "Phone": "(415) 555-1212",
```

```

    "Fax": null,
    "AccountNumber": null,
    "Website": null,
    "Sic": null,
    "Industry": null,
    "AnnualRevenue": null,
    "NumberOfEmployees": null,
    "Ownership": null,
    "TickerSymbol": null,
    "Description": null,
    "Rating": null,
    "Site": null,
    "OwnerId": "005ZL000000M0x1YAC",
    "CreatedDate": 1714431097000,
    "CreatedById": "005ZL000000M0x1YAC",
    "LastModifiedDate": 1714431097000,
    "LastModifiedById": "005ZL000000M0x1YAC",
    "PersonContactId": "003ZL000001EDrPYAW",
    "PersonMailingAddress": null,
    "PersonOtherAddress": null,
    "PersonMobilePhone": null,
    "PersonHomePhone": null,
    "PersonOtherPhone": null,
    "PersonAssistantPhone": null,
    "PersonEmail": null,
    "PersonTitle": null,
    "PersonDepartment": null,
    "PersonAssistantName": null,
    "PersonLeadSource": null,
    "PersonBirthdate": null,
    "PersonHasOptedOutOfEmail": null,
    "PersonHasOptedOutOfFax": null,
    "PersonDoNotCall": null,
    "PersonLastCURequestDate": null,
    "PersonLastCUUpdateDate": null,
    "PersonEmailBouncedReason": null,
    "PersonEmailBouncedDate": null,
    "PersonIndividualId": null,
    "PersonPronouns": null,
    "PersonGenderIdentity": null,
    "Jigsaw": null,
    "JigsawCompanyId": null,
    "AccountSource": null,
    "SicDesc": null,
    "CustomContactField__pc": "ABC"
}

// Change event for Contact
{
  "ChangeEventHeader": {
    "entityName": "Contact",
    "recordIds": [
      "003ZL000001EDrPYAW"
    ]
  }
}

```

```

    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "000a6a7b-b859-8ba4-2ab8-1a1babf66ff0",
    "sequenceNumber": 2,
    "commitTimestamp": 1714431097000,
    "commitNumber": 71542214535,
    "commitUser": "005ZL000000M0x1YAC",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "AccountId": "001ZL000001QS6mYAG",
  "IsPersonAccount": true,
  "Name": {
    "Salutation": "Ms.",
    "FirstName": "Martha",
    "LastName": "Brown"
  },
  "OtherAddress": null,
  "MailingAddress": null,
  "Phone": "(415) 555-1212",
  "Fax": null,
  "MobilePhone": null,
  "HomePhone": null,
  "OtherPhone": null,
  "AssistantPhone": null,
  "ReportsToId": null,
  "Email": null,
  "Title": null,
  "Department": null,
  "AssistantName": null,
  "LeadSource": null,
  "Birthdate": null,
  "Description": null,
  "OwnerId": "005ZL000000M0x1YAC",
  "HasOptedOutOfEmail": null,
  "HasOptedOutOfFax": null,
  "DoNotCall": null,
  "CreatedDate": 1714431097000,
  "CreatedById": "005ZL000000M0x1YAC",
  "LastModifiedDate": 1714431097000,
  "LastModifiedById": "005ZL000000M0x1YAC",
  "LastCURequestDate": null,
  "LastCUUpdateDate": null,
  "EmailBouncedReason": null,
  "EmailBouncedDate": null,
  "Jigsaw": null,
  "JigsawContactId": null,
  "IndividualId": null,
  "Pronouns": null,
  "GenderIdentity": null,

```

```

    "CustomContactField__c": "ABC"
  }

```

SEE ALSO:

[Salesforce Help: Person Account Fields \(by Label Names\)](#)

Updating a Person Account

When updating a person account, two change events are generated, one for the account and one for the contact, regardless which fields changed. Salesforce always updates the LastModifiedDate system field in both the account and contact even if the field updated is only in one of the underlying records.

Because a person account corresponds to one account and one contact, the timestamp fields of the account and contact records must match. If an account-only field is updated, such as the Industry field, the account change event contains the changed field and the LastModifiedDate field. The contact change event contains only the LastModifiedDate field. If the updated field stems from a contact, or is a custom contact field, both change events contain all changed fields and the LastModifiedDate field. In particular, if a person account's first name or last name is modified, the corresponding field is included in both change events.

Converting an Account

If a person account is converted to a business account through the API by modifying the record type ID, a change event for the account is generated. This change event contains the new record type ID of the account.

Conversely, if a business account is converted to a person account, a change event is generated for the account with the new record type ID.

Deleting a Person Account

When deleting a person account, two change events are generated: one for the deleted account and one for the deleted contact. The change events don't contain record fields. They contain only event header fields.

Change Events for Users

The user and email preferences in change events include only the preferences that are enabled (set to true) without their Boolean values. Preferences that are disabled (set to false) are not included in the event payload.

For a list of user and email preferences, see the [User Object](#) in the *Object Reference*.



Note: Preferences are stored in a 32-bit integer internal field in the database. When a preference is changed, a change is detected in the corresponding 32-bit integer field, and all enabled preferences that are represented by that internal integer field are published, whether or not they were changed.



Example: This change event is generated when a User record is created. Preferences are included under EmailPreferences and UserPreferences.

```

{
  "ChangeEventHeader": {
    "entityName": "User",
    "recordIds": [
      "0055f00000GBZR4AAP"
    ],

```

```

    "changeType": "CREATE",
    "changeOrigin": "",
    "transactionKey": "00068e80-1b23-ad90-c49d-ca6a5759661b",
    "sequenceNumber": 1,
    "commitTimestamp": 1714155903000,
    "commitNumber": 1100662266706,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "Username": "olga.brown@example.com",
  "Name": {
    "Salutation": null,
    "FirstName": "Olga",
    "LastName": "Brown"
  },
  "CompanyName": null,
  "Division": null,
  "Department": null,
  "Title": null,
  "Address": null,
  "Email": "katia_hage@hotmail.com",
  "EmailPreferences": [
    "AutoBcc",
    "StayInTouchReminder"
  ],
  "SenderEmail": null,
  "SenderName": null,
  "Signature": null,
  "StayInTouchSubject": null,
  "StayInTouchSignature": null,
  "StayInTouchNote": null,
  "Phone": null,
  "Fax": null,
  "MobilePhone": null,
  "Alias": "obrown",
  "CommunityNickname": "User17141558217316917617",
  "IsActive": true,
  "IsSystemControlled": false,
  "TimeZoneSidKey": "America/Los_Angeles",
  "UserRoleId": null,
  "LocaleSidKey": "en_US",
  "ReceivesInfoEmails": true,
  "ReceivesAdminInfoEmails": true,
  "EmailEncodingKey": "UTF-8",
  "ProfileId": "00e5f000002mIQ6AAM",
  "UserType": "Standard",
  "UserSubtype": null,
  "LanguageLocaleKey": "en_US",
  "EmployeeNumber": null,
  "DelegatedApproverId": null,
  "ManagerId": null,
  "LastLoginDate": null,

```

```

    "LastPasswordChangeDate": null,
    "CreatedDate": 1714155903000,
    "CreatedById": "0055f0000005mc66AAA",
    "LastModifiedDate": 1714155903000,
    "LastModifiedById": "0055f0000005mc66AAA",
    "NumberOfFailedLogins": null,
    "OfflineTrialExpirationDate": null,
    "OfflinePdaTrialExpirationDate": null,
    "UserPermissions": [
      "SFContentUser"
    ],
    "ForecastEnabled": false,
    "UserPreferences": [
      "ActivityRemindersPopup",
      "EventRemindersCheckboxDefault",
      "TaskRemindersCheckboxDefault",
      "DisableLikeEmail",
      "SortFeedByComment",
      "ShowTitleToExternalUsers",
      "HideSlBrowserUI",
      "LightningExperiencePreferred",
      "HideSfxWelcomeMat"
    ],
    "ContactId": null,
    "AccountId": null,
    "CallCenterId": null,
    "Extension": null,
    "FederationIdentifier": null,
    "AboutMe": null,
    "DigestFrequency": "D",
    "DefaultGroupNotificationFrequency": "N",
    "JigsawImportLimitOverride": 300,
    "WorkspaceId": null,
    "IsProfilePhotoActive": false,
    "IndividualId": null
  }

```

Change Events for Lead Conversion

Converting a lead results in the creation of an account, a contact, and optionally an opportunity, and also a lead update. When converting a lead, the change event for the lead update includes fields specific to the conversion.

These fields are included in the lead update change event for a lead conversion.

Field	Description
Status	The lead conversion status. Possible status values are in the LeadStatus standard object.
IsConverted	Indicates whether the lead was converted (<code>true</code>).
ConvertedDate	The date of the lead conversion. <code>ConvertedDate</code> doesn't include the time.
ConvertedAccountId	The ID of the account created in the lead conversion.

Field	Description
ConvertedContactId	The ID of the contact created in the lead conversion.
ConvertedOpportunityId	The ID of the opportunity created in the lead conversion.

The change event for the lead update doesn't include the `LastModifiedDate` field.

For an example lead update change event for a lead conversion, see [Lead Update Change Event](#) in the Example section.



Example: These example change events are generated when converting a lead. The order of the change events corresponds to the sequence of operations: the creation of an account, contact, opportunity, and the lead update. The `sequenceNumber` field in each change event denotes the sequence of the operations in the same transaction.

Account Create Change Event

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "0015f00002JUX1JAAX"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "00006948-0ac9-7da3-edb3-ea0e31b2334a",
    "sequenceNumber": 1,
    "commitTimestamp": 1714153779000,
    "commitNumber": 1100638524517,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "Name": "Cadinal Inc.",
  "Type": null,
  "ParentId": null,
  "BillingAddress": {
    "Street": null,
    "City": null,
    "State": "IL",
    "PostalCode": null,
    "Country": "USA",
    "Latitude": null,
    "Longitude": null,
    "GeocodeAccuracy": null
  },
  "ShippingAddress": null,
  "Phone": "(847) 262-5000",
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": null,
  "AnnualRevenue": null,
}
```

```

    "NumberOfEmployees": null,
    "Ownership": null,
    "TickerSymbol": null,
    "Description": null,
    "Rating": null,
    "Site": null,
    "OwnerId": "0055f000005mc66AAA",
    "CreatedDate": 1714153779000,
    "CreatedById": "0055f000005mc66AAA",
    "LastModifiedDate": 1714153779000,
    "LastModifiedById": "0055f000005mc66AAA",
    "Jigsaw": null,
    "JigsawCompanyId": null,
    "CleanStatus": "Pending",
    "AccountSource": "Web",
    "DunsNumber": null,
    "Tradestyle": null,
    "NaicsCode": null,
    "NaicsDesc": null,
    "YearStarted": null,
    "SicDesc": null,
    "DandbCompanyId": null,
    "OperatingHoursId": null,
    "CustomerPriority__c": null,
    "SLA__c": null,
    "Active__c": null,
    "NumberofLocations__c": null,
    "UpsellOpportunity__c": null,
    "SLASerialNumber__c": null,
    "SLAExpirationDate__c": null
  }
}

```

Contact Create Change Event

```

{
  "ChangeEventHeader": {
    "entityName": "Contact",
    "recordIds": [
      "0035f00002EztxDAAR"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "00006948-0ac9-7da3-edb3-ea0e31b2334a",
    "sequenceNumber": 2,
    "commitTimestamp": 1714153779000,
    "commitNumber": 1100638524517,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "AccountId": "0015f00002JUX1JAAX",
  "Name": {
    "Salutation": null,

```

```

    "FirstName": "Brenda",
    "LastName": "Mcclure"
  },
  "OtherAddress": null,
  "MailingAddress": {
    "Street": null,
    "City": null,
    "State": "IL",
    "PostalCode": null,
    "Country": "USA",
    "Latitude": null,
    "Longitude": null,
    "GeocodeAccuracy": null
  },
  "Phone": "(847) 262-5000",
  "Fax": null,
  "MobilePhone": null,
  "HomePhone": null,
  "OtherPhone": null,
  "AssistantPhone": null,
  "ReportsToId": null,
  "Email": "brenda@cardinal.net",
  "Title": "CFO",
  "Department": null,
  "AssistantName": null,
  "LeadSource": "Web",
  "Birthdate": null,
  "Description": null,
  "OwnerId": "0055f000005mc66AAA",
  "HasOptedOutOfEmail": null,
  "HasOptedOutOfFax": null,
  "DoNotCall": null,
  "CreatedDate": 1714153779000,
  "CreatedById": "0055f000005mc66AAA",
  "LastModifiedDate": 1714153779000,
  "LastModifiedById": "0055f000005mc66AAA",
  "LastCURequestDate": null,
  "LastCUUpdateDate": null,
  "EmailBouncedReason": null,
  "EmailBouncedDate": null,
  "Jigsaw": null,
  "JigsawContactId": null,
  "CleanStatus": "Pending",
  "IndividualId": null,
  "Pronouns": null,
  "GenderIdentity": null,
  "Level__c": null,
  "Languages__c": null
}

```

Opportunity Create Change Event

```

{
  "ChangeEventHeader": {
    "entityName": "Opportunity",

```

```

    "recordIds": [
      "0065f00000UloqVAAR"
    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "00006948-0ac9-7da3-edb3-ea0e31b2334a",
    "sequenceNumber": 3,
    "commitTimestamp": 1714153779000,
    "commitNumber": 1100638524517,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "AccountId": "0015f00002JUX1JAAX",
  "IsPrivate": false,
  "Name": "Cadinal Inc.-",
  "Description": null,
  "StageName": "Prospecting",
  "Amount": null,
  "Probability": 10,
  "ExpectedRevenue": null,
  "TotalOpportunityQuantity": null,
  "CloseDate": 1719705600000,
  "Type": null,
  "NextStep": null,
  "LeadSource": "Web",
  "IsClosed": false,
  "IsWon": false,
  "ForecastCategory": "Pipeline",
  "ForecastCategoryName": "Pipeline",
  "CampaignId": null,
  "HasOpportunityLineItem": false,
  "Pricebook2Id": null,
  "OwnerId": "0055f000005mc66AAA",
  "CreatedDate": 1714153779000,
  "CreatedById": "0055f000005mc66AAA",
  "LastModifiedDate": 1714153779000,
  "LastModifiedById": "0055f000005mc66AAA",
  "LastStageChangeDate": null,
  "ContactId": null,
  "ContractId": null,
  "LastAmountChangedHistoryId": null,
  "LastCloseDateChangedHistoryId": null,
  "DeliveryInstallationStatus__c": null,
  "TrackingNumber__c": null,
  "OrderNumber__c": null,
  "CurrentGenerators__c": null,
  "MainCompetitors__c": null
}

```

Lead Update Change Event

```
{
  "ChangeEventHeader": {
    "entityName": "Lead",
    "recordIds": [
      "00Q5f000005bwLFEAY"
    ],
    "changeType": "UPDATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "00006948-0ac9-7da3-edb3-ea0e31b2334a",
    "sequenceNumber": 4,
    "commitTimestamp": 1714153780000,
    "commitNumber": 1100638524517,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": [
      "0x08F81000"
    ]
  },
  "Name": null,
  "Title": null,
  "Company": null,
  "Address": null,
  "Phone": null,
  "MobilePhone": null,
  "Fax": null,
  "Email": null,
  "Website": null,
  "Description": null,
  "LeadSource": null,
  "Status": "Closed - Converted",
  "Industry": null,
  "Rating": null,
  "AnnualRevenue": null,
  "NumberOfEmployees": null,
  "OwnerId": null,
  "HasOptedOutOfEmail": null,
  "IsConverted": true,
  "ConvertedDate": 1714089600000,
  "ConvertedAccountId": "0015f00002JUX1JAAX",
  "ConvertedContactId": "0035f00002EztxDAAR",
  "ConvertedOpportunityId": "0065f00000UloqVAAR",
  "IsUnreadByOwner": null,
  "CreatedDate": null,
  "CreatedById": null,
  "LastModifiedDate": 1714153780000,
  "LastModifiedById": null,
  "DoNotCall": null,
  "HasOptedOutOfFax": null,
  "LastTransferDate": null,
  "Jigsaw": null,
  "JigsawContactId": null,
}
```

```

"CleanStatus": null,
"CompanyDunsNumber": null,
"DandbCompanyId": null,
"EmailBouncedReason": null,
"EmailBouncedDate": null,
"IndividualId": null,
"Pronouns": null,
"GenderIdentity": null,
"SICCode__c": null,
"ProductInterest__c": null,
"Primary__c": null,
"CurrentGenerators__c": null,
"NumberofLocations__c": null
}

```

The `changedFields` bitmap field contains the fields that were changed for the lead record. These are the fields contained in `changedFields` after the Pub/Sub API client decoded this field.

```

2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class -
=====
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class -      Changed
Fields
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class -
=====
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class - Status
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class - IsConverted
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class - ConvertedDate
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class - ConvertedAccountId
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class - ConvertedContactId
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class -
ConvertedOpportunityId
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class - LastModifiedDate
2024-04-26 10:49:43,114 [grpc-default-executor-1] java.lang.Class -
=====

```

Change Events for PricebookEntry

The Create Change Events payload does not include the system fields (`sCreatedBy`, `CreatedDate`, `LastModifiedById`, and `LastModifiedDate`). This is due to the highly customized nature of the `PricebookEntry` object, which prevents these specific fields from being captured at the time the event is generated.

Change Events for Fields

Learn about the change event characteristics for fields.

IN THIS SECTION:

[Sending Data Differences for Fields of Updated Records](#)

To reduce the event payload size and improve performance, Salesforce sometimes sends data differences of updated text values. For large text fields, such as `Description` or `Long Text Area` fields that contain at least 1,000 characters, only the data differences might be sent. Data differences use the unified diff format.

Change Events for Custom Field Type Conversions

When you change the type of a custom field, a change event or gap event is generated for data changes for some conversions. Other conversions, such as those that preserve or truncate field values, don't generate events.

Sending Data Differences for Fields of Updated Records

To reduce the event payload size and improve performance, Salesforce sometimes sends data differences of updated text values. For large text fields, such as Description or Long Text Area fields that contain at least 1,000 characters, only the data differences might be sent. Data differences use the unified diff format.

Differences are computed for each line in the text value. The diff algorithm breaks the field value into lines by using the line breaks found in the value.

If sending the diff for updates of large text fields does not reduce the field size, the entire value is sent. The diff value is **not** sent for the following conditions.

- The length of the field value is less than 1,000 characters.
- The difference between the old and new values is greater than 50% in length.
- More than 25% of the lines of the total of number of lines in the old and new values are changed.
- The diff's length is greater than the length of the new value.

For more information about the unified diff format and the diff utility, see the [Diff Utility](#) Wikipedia article.

The diff value includes an SHA-256 hash value that is computed on the entire updated value. Use the hash value to verify that the reconstructed value matches the original value before it was converted to a diff. To do so, compute the SHA-256 hash after expanding the diff value. Then compare the two hash values to ensure that they're equal. If the reconstructed content is different from the original content, the hash value is different. To compute an SHA-256 hash value, you can use a utility such as the UNIX sha256sum command or the [DigestUtils](#) class from the Apache Commons library.

IN THIS SECTION:

[Data Differences in Event Fields](#)

When the updated text field value is sent as a diff, it contains the SHA-256 hash value and data differences in the unified diff format.

[How to Reconstruct a Field from Its Diff Value](#)

The value of a diff field is in the unified diff format. Use a diff utility to obtain the full field value from the diff.

[Considerations for Newline Characters and Computing the SHA-256 Hash](#)

The content that Salesforce uses to generate the SHA-256 hash might have newline characters transformed by the browser. Many browsers transform newline characters to `\r\n` in record field values before records are stored in Salesforce. Also, Salesforce trims leading and trailing white spaces in field values.

Data Differences in Event Fields

When the updated text field value is sent as a diff, it contains the SHA-256 hash value and data differences in the unified diff format.

The field whose data is sent as a unified diff contains a value in this format.

```
"--- \n+++ <hash_value>\n
(Changes) "
```



Example: In a Pub/Sub API client, the field contains the unified diff value in this format.

```
"<Field_Name>": "--- \n+++ <hash_value>\n
(Changes)"
}
```

And the fields whose values are sent as a unified diff are listed in `diffFields` in `ChangeEventHeader`.

In a Streaming API (CometD) client, the field contains the `diff` subfield, which contains the unified diff value.

```
"<Field_Name>": {
  "diff": "--- \n+++ <hash_value>\n
(Changes)"
}
```

This change event is received in a Pub/Sub API client after the Description field with more than 1,000 characters is updated for an account. The Description field contains the hash value after the `+++` prefix followed by the data differences.

```
{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "0015f00002JUWMVAA5"
    ],
    "changeType": "UPDATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "0000654a-b07c-5852-d65f-47dc5ecd631d",
    "sequenceNumber": 1,
    "commitTimestamp": 1714149392000,
    "commitNumber": 1100583868458,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [
      "0x010000"
    ],
    "changedFields": [
      "0x410000"
    ]
  },
  "Name": null,
  "Type": null,
  "ParentId": null,
  "BillingAddress": null,
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": null,
  "AnnualRevenue": null,
  "NumberOfEmployees": null,
  "Ownership": null,
  "TickerSymbol": null,
  "Description": "--- \n+++
682b8747ccdb93b546e7bbe479b27d26ec7c38ccabb76cdd8308c6595492bffc\n@@ -2,1 +2,1
```

```

@@\n-Business applications are moving to the cloud. It's not just a fad-the shift from
  traditional software models to the Internet has steadily gained momentum over the
  last 10 years.\n+Business apps are moving to the cloud. It's not just a fad-the shift
  from traditional software models to the Internet has steadily gained momentum over
  the last 10 years.\n@@ -7,1 +7,1 @@\n-As cloud computing grows in popularity, thousands
  of companies are simply rebranding their non-cloud products and services as "cloud
  computing." Always dig deeper when evaluating cloud offerings.\n+As cloud computing
  grows in popularity, thousands of companies are simply rebranding their non-cloud
  products and services as "cloud computing." Always dig deeper when evaluating cloud
  offerings. And keep in mind that if you have to buy and manage hardware and software,
  what you're looking at isn't really cloud computing but a false cloud.",
  "Rating": null,
  "Site": null,
  "OwnerId": null,
  "CreatedDate": null,
  "CreatedById": null,
  "LastModifiedDate": 1714149392000,
  "LastModifiedById": null,
  "Jigsaw": null,
  "JigsawCompanyId": null,
  "CleanStatus": null,
  "AccountSource": null,
  "DunsNumber": null,
  "Tradestyle": null,
  "NaicsCode": null,
  "NaicsDesc": null,
  "YearStarted": null,
  "SicDesc": null,
  "DandbCompanyId": null,
  "OperatingHoursId": null,
  "CustomerPriority__c": null,
  "SLA__c": null,
  "Active__c": null,
  "NumberofLocations__c": null,
  "UpsellOpportunity__c": null,
  "SLASerialNumber__c": null,
  "SLAExpirationDate__c": null
}

```

The updates made to the diff field are the following.

- In the paragraph starting with "Business applications," the word "applications" was replaced with "apps."
- In the paragraph starting with "As cloud computing grows," one sentence was appended at the end of the paragraph: "And keep in mind that if you have to buy and manage hardware and software, what you're looking at isn't really cloud computing but a false cloud."

This event for the account creation shows the original and full values in the Description field before it was updated. If you generate the SHA-256 hash on the full value, you get the same value sent in the account update event (682b8747ccdb93b546e7bbe479b27d26ec7c38ccabb76cdd8308c6595492bf6fc).

```

{
  "ChangeEventHeader": {
    "entityName": "Account",
    "recordIds": [
      "0015f00002JUWMVAA5"
    ]
  }
}

```

```

    ],
    "changeType": "CREATE",
    "changeOrigin": "com/salesforce/api/soap/60.0;client=SfdcInternalAPI/",
    "transactionKey": "0000653e-941c-2eab-33e9-5459293f4b89",
    "sequenceNumber": 1,
    "commitTimestamp": 1714149340000,
    "commitNumber": 1100583170994,
    "commitUser": "0055f000005mc66AAA",
    "nulledFields": [],
    "diffFields": [],
    "changedFields": []
  },
  "Name": "Acme",
  "Type": null,
  "ParentId": null,
  "BillingAddress": null,
  "ShippingAddress": null,
  "Phone": null,
  "Fax": null,
  "AccountNumber": null,
  "Website": null,
  "Sic": null,
  "Industry": null,
  "AnnualRevenue": null,
  "NumberOfEmployees": null,
  "Ownership": null,
  "TickerSymbol": null,
  "Description": "Everyone is talking about “the cloud.” But what does it
mean?\r\nBusiness applications are moving to the cloud. It’s not just a fad—the shift
from traditional software models to the Internet has steadily gained momentum over
the last 10 years.\r\nCloud computing: a better way\r\nWith cloud computing, you
eliminate headaches because you’re not managing hardware and software—that’s the
responsibility of an experienced vendor like Salesforce. The shared infrastructure
means it works like a utility: you only pay for what you need, upgrades are automatic,
and scaling up or down is easy.\r\nCloud-based apps can be up and running in days or
weeks, and they cost less. With a cloud app, you just open a browser, log in, customize
the app, and start using it.\r\nBusinesses are running all kinds of apps in the cloud,
like customer relationship management (CRM), HR, accounting, and much more. Some of
the world’s largest companies moved their applications to the cloud with Salesforce
after rigorously testing the security and reliability of our infrastructure.\r\nAs
cloud computing grows in popularity, thousands of companies are simply rebranding their
non-cloud products and services as “cloud computing.” Always dig deeper when evaluating
cloud offerings.",
  "Rating": null,
  "Site": null,
  "OwnerId": "0055f000005mc66AAA",
  "CreatedDate": 1714149340000,
  "CreatedBy": "0055f000005mc66AAA",
  "LastModifiedDate": 1714149340000,
  "LastModifiedBy": "0055f000005mc66AAA",
  "Jigsaw": null,
  "JigsawCompanyId": null,
  "CleanStatus": "Pending",
  "AccountSource": null,

```

```

    "DunsNumber": null,
    "Tradestyle": null,
    "NaicsCode": null,
    "NaicsDesc": null,
    "YearStarted": null,
    "SicDesc": null,
    "DandbCompanyId": null,
    "OperatingHoursId": null,
    "CustomerPriority__c": null,
    "SLA__c": null,
    "Active__c": null,
    "NumberofLocations__c": null,
    "UpsellOpportunity__c": null,
    "SLASerialNumber__c": null,
    "SLAExpirationDate__c": null
}

```

How to Reconstruct a Field from Its Diff Value

The value of a diff field is in the unified diff format. Use a diff utility to obtain the full field value from the diff.

For example, you can use the [Java Diff Utilities library](#).

The following code sample shows the order of operations and the library tools used. The `toLines()` method, which you implement, splits the diff value into a list of lines. The `BufferedReader` Java object determines how the newline character is represented, so you don't need to pass in the `newLine` value.

Next, patches are obtained from the diff lines through the Java diff utility method `DiffUtils.parseUnifiedDiff()`. The patches are the changes applied to the content. The `toLines()` method is called again to split the original content into lines. The patches are then applied to the original lines using the `DiffUtils.patch()` method.

You implement the `combineLines()` method to combine the updated lines into one string variable. The `newLine` variable is passed to `combineLines()` to reintroduce the original line breaks in the text. Set the `newLine` variable to the newline character sequence that was in the original content (`\r\n` or `\n`). For more information, see [Considerations for Newline Characters and Computing the SHA-256 Hash](#). The revised string variable is the reconstructed value from the diff that contains the updates.

The final step is to generate a SHA-256 hash value to validate that the original updated value matches the reconstructed value. To generate a hash, use the [Apache Common DigestUtils library](#). After the hash is generated, compare it to the one sent in the event and ensure that both hash values are equal.

```

public void BuildOriginalValueFromDiff(String original, String diff, String newLine) {
    // Split diff value into lines and get patches.
    List<String> diffLines = toLines(diff);
    Patch<String> patch = DiffUtils.parseUnifiedDiff(diffLines);

    // Split original text into lines.
    List<String> originalLines = toLines(original);

    // Apply patches to original lines, then combined lines.
    List<String> revisedLines = DiffUtils.patch(originalLines, patch);
    String revised = combineLines(revisedLines, newLine);

    // Generate SHA-256 hash on reconstructed value.
    String checksum = DigestUtils.sha256Hex(revised);
}

```

```
// Extract hash from the event diff field.

// Compare extracted hash with generated hash and verify they are equal.
}
```

The following are examples of what to implement to split lines and combine lines.

```
private List<String> toLines(String s) {
    BufferedReader rd = new BufferedReader(new StringReader(s));
    return rd.lines().collect(Collectors.toList());
}

private String combineLines(List<String> lines, String newLine) {
    StringBuilder sb = new StringBuilder();
    lines.forEach(l -> sb.append(l).append(newLine));
    sb.deleteCharAt(sb.length()-newLine.length()); // remove last newline added
    return sb.toString();
}
```

Considerations for Newline Characters and Computing the SHA-256 Hash

The content that Salesforce uses to generate the SHA-256 hash might have newline characters transformed by the browser. Many browsers transform newline characters to `\r\n` in record field values before records are stored in Salesforce. Also, Salesforce trims leading and trailing white spaces in field values.

Before you generate the SHA-256 hash value, ensure that the reconstructed content from the diff contains the same newline characters as the original content and that no new leading or trailing white spaces are added. For example, when you save the content in a file, the operating system can add a trailing white space character.



Note: If you used the API to create or update field values, the newline characters supplied by the application are honored and stored in Salesforce without further transformations.

Windows systems represent the newline character as a carriage return and line-feed character sequence (`\r\n`). UNIX and UNIX-based systems, like macOS and Linux, represent the newline character as a line-feed character (`\n`).

Change Events for Custom Field Type Conversions

When you change the type of a custom field, a change event or gap event is generated for data changes for some conversions. Other conversions, such as those that preserve or truncate field values, don't generate events.

IN THIS SECTION:

[Conversions That Generate a Change Event](#)

When converting a custom field type to another type that isn't compatible, field data is lost and is set to null in records corresponding to the object. One change event is generated for all the affected records, and the event message contains no record fields.

[Conversions That Generate a Gap Event](#)

A gap event is generated for all the affected records for some field conversions from Picklist. The change event header of the gap event message contains information about the records, including the record IDs and a change type of `GAP_UPDATE`.

[Conversions That Don't Generate Events](#)

No change or gap events are generated for custom field type conversions that preserve or truncate field data, and for conversions between Picklist and Text fields.

SEE ALSO:

[Salesforce Help: Change the Custom Field Type](#)

[Salesforce Help: Notes on Changing Custom Field Types](#)

[Salesforce Help: Custom Field Types](#)

Conversions That Generate a Change Event

When converting a custom field type to another type that isn't compatible, field data is lost and is set to null in records corresponding to the object. One change event is generated for all the affected records, and the event message contains no record fields.

Examples of incompatible field changes are:

- Changing a Date or Date/Time field to any other field type, and vice versa
- Changing a Checkbox field to any other field type
- Changing a Picklist (Multi-Select) field to any other field type

Because a field type conversion can affect many records, the `recordIds` header field value in the event message contains a wildcard value instead of a record ID array. The value starts with the three-character object ID prefix, followed by the wildcard character `*`. For example, if you make an incompatible field type change for an Account custom field, the `recordIds` field looks similar to the following.

```
"ChangeEventHeader": {
  "entityName": "Account",
  "recordIds": [
    "001*"
  ],
  ...
}
```

SEE ALSO:

[ChangeEventHeader Fields](#)

Conversions That Generate a Gap Event

A gap event is generated for all the affected records for some field conversions from Picklist. The change event header of the gap event message contains information about the records, including the record IDs and a change type of `GAP_UPDATE`.

These field type conversions generate a gap event.

- Changing a Picklist field to Checkbox
- Changing a Picklist field to Picklist (Multi-Select)

SEE ALSO:

[Gap Events](#)

Conversions That Don't Generate Events

No change or gap events are generated for custom field type conversions that preserve or truncate field data, and for conversions between Picklist and Text fields.

Compatible Field Types with No Data Change

When converting a field type to another type that is compatible, field data is unchanged, and no event is generated. For example, these conversions are compatible.

- Changing a Text Area, Email, Url, Phone, Autonumber, Number, Percent, or Currency field to a Text field
- Changing a Text field to a Text Area, Text Area (Long), Email, Url, Phone, or Autonumber field

Other Field Type Conversions

These field type conversions also don't generate events.

- Changing a Picklist field to a Text field
- Changing a Text field to a Picklist field
- Conversions that result in truncated data because the target field type has a smaller size, such as changing a Text Area (Long) field to a Text, Text Area, Email, Url, or Phone field